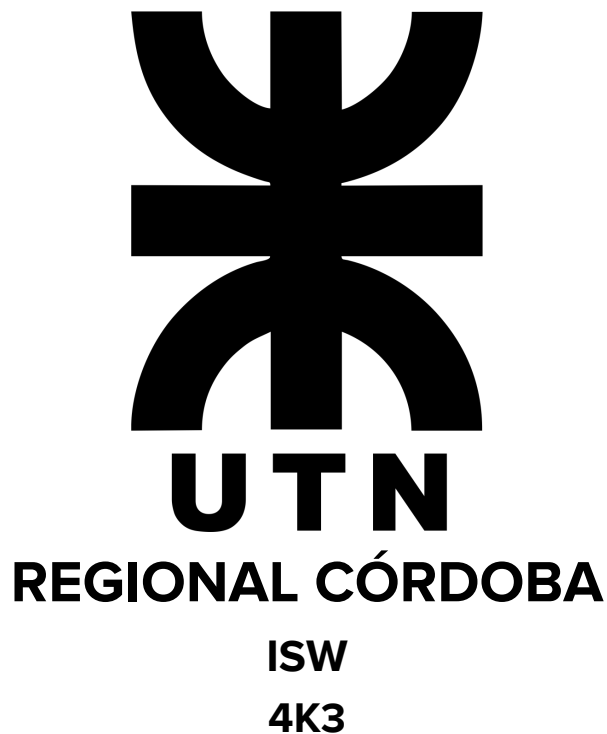


UNIVERSIDAD TECNOLÓGICA NACIONAL



RESUMEN FINAL Y ULTRAVIOLENTO

PROFESORES:

- Covaro, Laura Ines (Little Meles)
- Gonzalez, Georgina Gisela (La Morocha ♥ ♥ ♥)
- Massano, Maria Cecilia (Teniente Massano)

ALUMNOS:

- Bergesio, Genaro (MVP)
- Cuello, Tomas Marcelo (Cloaca Enjoyer)
- Floreano, Micaela (El último miembro)
- Galiano, Tomas (Silence)
- Liendo, Agustin (El Mítico)
- Nayi, Carlitooooooooosss (Closet Abierto)



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

ÍNDICE

INTRODUCCIÓN.....	6
Vencedores Vencidos.....	6
Introducción.....	8
Bitácora 07-10-2024.....	9
Bitácora 29-11-2024.....	10
TEMARIO.....	11
Unidad N° 1.....	11
Unidad N° 2.....	11
Unidad N° 3.....	11
Unidad N° 4.....	12
UNIDAD N° 1.....	12
Software, Ingeniería y ustedes.....	12
Definiciones.....	12
Reseña histórica.....	13
La mencionada Crisis.....	14
Disciplinas de la ingeniería.....	14
Éxitos y Fracasos en Proyectos.....	15
Contexto.....	15
Ciclos de vida y si vale la pena vivirlos.....	16
Modelo Secuencial.....	16
Modelo Interactivo-Incremental.....	16
Modelo Recursivo.....	17
Procesos Empíricos.....	17
Diferencias con definidos.....	17
Agilidad y Pilares.....	18
Ciclos de Vida.....	19
Code and Fix.....	19
Cascada puro.....	20
Cascada en V.....	21
Cascada con fases solapadas.....	21
Cascada con subproyecto.....	21
Incremental.....	21
Espiral.....	22
Evolutivo.....	22
Diseño para cronograma.....	22
Prototipo evolutivo.....	22
Proyectos.....	23
Roles dentro de un proyecto.....	24
Líder de proyecto.....	24
Equipo de proyecto.....	24
Stakeholders.....	24
Product Owner (PO).....	25



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

SCRUM Master.....	25
La triple restricción.....	25
Plan de proyecto.....	26
1. Definición de alcance.....	27
2. Definición del proceso y ciclo de vida.....	27
3. Estimación.....	27
4. Gestión de riesgo.....	27
5. Asignación de responsabilidades.....	28
6. Calendario.....	28
7. Métricas.....	28
8. Controles.....	28
Vinculo Proceso-Producto-Proyecto (Triple P).....	29
Sin Balitas de Plata (Tesis).....	30
UNIDAD N° 2.....	32
Gestión de Proyectos basados en procesos definidos.....	32
Filosofía Lean.....	33
7 Principios Lean.....	33
Manifiesto Ágil.....	34
Agilismo.....	34
Manifiesto Ágil.....	35
Valores Ágiles:.....	36
12 Principios Ágiles.....	36
Filosofía Ágil by JC.....	37
Requerimientos Ágiles.....	37
Introducción.....	37
BRUF.....	38
Just In Time.....	38
Tipos de Requerimientos:.....	39
Gestión de Requerimientos en Agile.....	39
Introducción.....	39
User Stories.....	41
Partes de una US.....	42
Story Points.....	43
Tipos de Historias.....	43
Modelo INVEST.....	44
Modelado de Roles.....	45
Product BackLog.....	45
Definition of Ready (DoR).....	46
Definition of Done (DoD).....	46
Estimaciones.....	46
Errores comunes en las estimaciones.....	47
Proceso de estimación.....	47
No Estimate:.....	49
Tips respecto de las estimaciones:.....	49
Estimaciones Ágiles:.....	50



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Tamaño vs Esfuerzo:	50
Poker Planning / Poker Estimation	51
Pasos (By Carlitos):	51
Fibo y sus amigos.	52
Velocidad	52
SCRUM	53
Introducción a la verdad.	53
Nosotros y el SCRUM	54
Eventos SCRUM	56
El SPRINT (Toda la iteracion)	56
Planning (Ceremonia)	57
Daily (Ceremonia)	57
Review (Ceremonia)	57
Retrospective (Ceremonia)	57
Refinamiento (Ceremonia)	57
Artefactos de SCRUM	57
Pila de producto (Product backlog)	58
La pila del sprint (Sprint Backlog)	58
Incremento	58
TimeBox	58
DoR (definición de listo)	58
DoD (Definición de Terminado)	58
Capacidad del equipo en un sprint	59
Herramientas del SCRUM	59
Tarjeta de tarea	59
Tableros	59
Gráficos del backlog	60
Sprint BurnDown Chart	60
Sprint BurnUp Chart	60
Combined Burn Chart	61
Los tiempos del SCRUM:	61
Métricas	61
Running Tested Features (Funcionalidades andando y validadas)	62
Capacidad	62
Velocidad	63
Framework para escalar SCRUM	63
Nexus	63
LeSS	64
Less Huge	65
Métricas Ágiles	65
Gestión de Productos de Software	66
Propuesta de Valor Única (UVP):	66
Producto Mínimo Viable (MVP):	67
Característica Mínima Viable (MVF):	67
Característica Mínima Comerciable (MMF):	68



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Producto Mínimo Comerciales (MMP):.....	68
Las Características Mínima de Release (MMR):.....	68
¿Cómo se relacionan todos estos conceptos?.....	69
¿Qué errores comunes podemos cometer con estos conceptos?.....	69
El dilema de la audacia cero.....	69
KANBAN.....	70
Introducción.....	70
Prácticas de kanban.....	72
Visualizar.....	72
Limitar el WIP.....	72
Gestionar flujos de trabajo.....	72
Hacer políticas explícitas.....	73
Establecer ciclos de retroalimentación.....	73
Mejora colaborativamente evolucionar experimental.....	73
Métricas Kanban.....	73
Metricas Lean (Kanban).....	74
Lead Time o Elapsed Time.....	74
Cycle time.....	74
Touch Time.....	74
Tablero Kan.....	75
Valores de KANBAN.....	75
Principios.....	75
LEAN UX.....	76
Introducción.....	76
Metodología Lean UX.....	76
Pensar.....	76
Hacer.....	77
Comprobar.....	77
Design Thinking.....	77
UNIDAD N° 3.....	78
Introducción.....	78
Configuración de software:.....	78
Conceptos del SCM.....	80
Ítem de Configuración de Software (SCI):.....	80
Versión:.....	81
Variante:.....	81
Línea Base.....	82
Repositorio:.....	82
Ramas.....	83
Actividades relacionadas con la Gestión de Configuración.....	83
Identificación de Ítems.....	84
Control de cambios:.....	84
Auditorías de Configuración de Software:.....	86
Informes de estado:.....	86
Gestión de Configuración en ambientes ágiles.....	87



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Prácticas continuas.....	87
Despliegue de producto. Estrategias para el despliegue de productos.....	88
Introducción.....	88
Tipos de despliegues.....	89
Consideraciones.....	90
UNIDAD N° 4.....	91
Conceptos generales sobre calidad.....	91
Pequeña anécdota sobre las instituciones.....	91
Aseguramiento de calidad.....	92
Principios.....	93
La gestión de la calidad.....	93
Administración de la Calidad del Software.....	94
Reporte del grupo de aseguramiento de calidad (GAC).....	94
Actividades asociadas.....	95
Principales Modelos de Calidad existentes.....	96
Introducción.....	96
CMMI.....	96
Diferencias contra métodos ágiles.....	98
Similitudes.....	99
Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad.....	100
Introducción.....	100
Tipos de Auditorías.....	101
Auditoría de proyecto.....	101
Auditoría de configuración funcional.....	101
Auditoría de configuración física.....	101
Auditoría externas de aseguramiento de calidad.....	101
Proceso de Auditorías.....	102
Procesos de auditorías.....	102
Resultados de una auditoría.....	102
Métricas de auditoría.....	103
Herramientas de auditorías.....	103
Calidad de Producto:.....	103
Técnicas y Herramientas para la realización de revisiones técnicas del software.....	104
Técnicas de revisión técnica del software.....	104
Herramientas para revisiones técnicas.....	104
Testing en ambientes Ágiles.....	105
Introducción a molestar a los desarrolladores.....	105
Planificación y control.....	107
Análisis y Diseño.....	107
Ejecución.....	108
Evaluación.....	108
Principios del Testing.....	108
Niveles de Testing.....	109
Pruebas Unitarias.....	109
Pruebas Integración.....	109



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Pruebas de Versión o Sistema.....	110
Pruebas de Aceptacion Usuario.....	110
Estrategias de Caja Negra.....	110
Basado en especificaciones: Partición de Equivalencias.....	110
Basado en especificaciones: Análisis de valores límites.....	112
Basados en la experiencia: Adivinanza de Defectos.....	114
Basados en la experiencia: Testing Exploratorio.....	114
Estrategias de Caja Blanca.....	114
¿Cuánto Testing es Suficiente?.....	115
Tipos de Testing.....	116
Smoke Test.....	116
Testing Funcional.....	117
Testing No Funcional.....	117
TDD.....	118
BIBLIOGRAFÍA.....	118
MIS MEMORIAS.....	120
Mil años de soledad.....	120



INTRODUCCIÓN

Vencedores Vencidos

Diez individuos se encuentran sentados en una habitación oscura, debatiendo quién es el culpable de un crimen cometido hace años.

El incidente ocurrió en Marzo del 97, hacía calor, los adolescentes tenían esperanzas de recibirse, ese día llovía, en resumen había humedad. Yo me encontraba con la espalda apoyada contra la pared mirando el aula 226, se olía un dejo a tristeza pero no sabía de donde venía.

Una señora se presentó frente mi:

- ¿Fuego tiene?

Sin mediar palabra le acerque un encendedor negro que tenia dando vueltas. En los 90 se podía fumar adentro, que buenos tiempos. Prendió el pucho y preguntó:

- ¿Sabe si se da DSI en el Aula 226?
- + Sí, como todos los años

Era sabido que la facultad transitaba un cambio de era, ingeniería en sistemas era nueva y aún había pie para cambiar el plan de estudio y las materias. DSI era una de estas asignaturas que todo el mundo hacía sin pensar. Se hablaba que la tecnología traería exámenes en computadoras en vez de los horribles diagramas en hojas A3 que estaban pasando de moda.

La curiosidad me invadió e indague

- + ¿Viene a clases?
- No, soy la nueva profesora de este curso
- + Que curioso, justo aprobé el final la semana pasada ¿La curso hace mucho?
- Hace 3 años, me recibí el año pasado y estoy buscando horas paralelo a mi trabajo
- + Déjeme adivinar, ¿analista no?
- ¿Por qué daría esta materia sino? De igual manera tengo planes para el futuro para que sea algo más
- + Algo como que?
- Un adelanto de la vida real, la facultad no te prepara para el laburo de enserio. Me parece que el tercer año de ingeniería es el mejor lugar para empezar, ¿no?
- + No estaría mal, pero un viaje cambiar tanto una cátedra
- No creo que sea para tanto



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Nos quedamos en silencio cada uno en su propio mundo, probablemente soñando cosas que no se iban a cumplir.

El compañero que estaba esperando salió del aula, me avisó que iba al baño y que terminara de fumar tranquilo. Espere unos segundos, tire el pucho al piso y aplaste el dejó de vida que le quedaba. Salude a la futura docente con un gesto de cabeza y me encamine por el vacío pasillo.

Al tiempo me enteré que fue la última vez que se vio a nuestra predispuesta ingeniería, quedando de ella un montón de papeles. Parece que era enserio lo que me había contado y que estaba rehaciendo el plan de la materia para que realmente fuera útil, buscando software para hacer diagramas de manera cómoda.

De manera sospechosa la persona que supuestamente la suplanto también pensaba en cómo cambiar la materia... pero de otra forma... Una joven Judith Meles se presentó como nueva docente del 3K1 de ese año y comenzó su estelar ascenso por los rangos de la universidad.

En el lejano tiempo de este recuerdo hasta ahora se ha buscado incansablemente el culpable de la desaparición de una **persona tan llena de sueños** que nunca más fue vista por las autoridades de la universidad. De las personas en la habitación existen diversas teorías, algunos dicen que era su hermana que en un ataque de celos de que ella ganara el concurso se propuso arruinar su sueño de hacer la materia más útil. Otros creen que solo estaba en el lugar correcto en el momento correcto y creen en su inocencia, como también que fue presa de la circunstancias por eso se hace todo en papel.

Pero yo sé la verdad, lo vi con mis propios ojos. La Ingeniera Meles es aquella que tuvo esos sueños de grandeza, que buscaba mejorar las cosas... Ahora solo queda un alma en desdicha, que atormenta a sus estudiantes aun después de DSI.

Acá estamos en el 2024 y vemos los restos de un montón de sueños destruidos mientras cursamos ISW.

Introducción

Bueno ahora dejando la pavada que acostumbro les comento la situación real de la materia este bellísimo 2024. En el corriente año se modificó la forma en que se toman los parciales por ende es una aventura que probablemente vivamos juntos de acá en adelante. Judith como lo mencionó previamente tiene su injerencia en esta materia pero no es tan horrible e inservible como es DSI, sino que tiene un componente de utilidad en las relaciones laborales.

La materia consiste de un torrente de trabajos prácticos (Menos que ARE) que resolveremos a lo largo del año y que tendremos que asistir prácticamente de manera “obligatoria” a todos. Yo tengo con la Ingeniera Covaro que por motivos ajenos a mi no da el 100% de las clases de los viernes sino que asiste Meles. La verdad es que Laura (Covaro) da una clase concisa, interesante y termina como 1 hora antes mientras que Judith da el mismo perno de DSI hasta las 23hs. Mi consejo es que no vayan



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

a las clases de la segunda, disfruten su viernes saliendo a tomar y viendo las clases de manera asincrónica.

Ahora para que se entienda, la materia es semestral, no es extremadamente difícil pero conlleva un conjunto de horas mayor a otras materias. Las clases prácticas no deberían faltar porque son de gran utilidad ya que resolvemos ejercicios que van al parcial.

La cuestión es así, hay que sacar 80 ir como al 80% de los TP, hacer un zoológico de plastilina (No fake), hacer un montón de User Stories y en general pasarla mal. Voy a hacer como hicieron otros antes de mí y voy a poner el temario completo más allá de si lo vimos en clases o no. No tenemos muchas opciones así que aquellas almas puras no temáis y uniros a mi cruzada.

Prácticamente me olvidé, pero este resumen es final y por ende es ultraviolento así que no queda otra que escuchar:



1, 2, Ultraviolento

“Caen Murallas, va comenzar la única justa de las batallas”

- La ex de Pique -

Bitácora 07-10-2024

Escribo esta bitácora como advertencia a aquellas almas que se adentren en este cruel mundo sin la información correcta. Rendimos el primer parcial y es una total falacia que es un parcial unificado, lo unificado es como te rompen la cola. Bueno tampoco para tanto, pero si rendis 2 parciales por separado el mismo día (Un sábado) por lo que encaren este día con mucha predisposición.

Los temas que nuestras docentes escribieron que van:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Unidad 1 completa (incluye paper No silver bullet)

- Introducción a la Ingeniería del Software. ¿Qué es?
- Estado Actual y Antecedentes. La Crisis del Software.
- Disciplinas que conforman la Ingeniería de Software.
- Ejemplos de grandes proyectos de software fallidos y exitosos.
- Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.
- Procesos de Desarrollo Empíricos vs. Definidos.
- Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software
- Ventajas y desventajas de c/u de los ciclos de vida.
- Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.
- Componentes de un Proyecto de Sistemas de Información.
- Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Unidad 2

- Gestión de Producto
- Requerimientos Ágiles
- User Stories
- Estimaciones Ágiles

Unidad 3

- SCM

Sobre el examen, **el examen teórico** consiste en 4 preguntas más o menos de este tipo:

En mi Tema:

Considerando la metodología de gestión de proyecto tradicional (Procesos Definidos) tenemos que:

1. Definir qué ciclo de vida usariamos, cuando serian los entregables y justificar
2. Usando la triple restricción como estimamos, como se hacen, quién las hace
3. Cómo se solicitan, como se controlan las desviaciones.
4. Ítems de configuración, identificar 3 y hacerle regla de nomenclatura

El otro tema son los mismo puntos pero con Ágil

Por otro lado, **el práctico** fue un enunciado sobre una aplicación del tipo spotify donde aclara lo que el cliente cree es lo mejor para el MVP. En mi caso decía que tenía que sí o sí tener listas de reproducción para cuentas Premium y que únicamente le diéramos importancia al Premium Familiar. También que pudiéramos loguear con Apple, teniendo después los siguientes puntos:

1. Identificar las User Story que van al MVP
2. Explicar el MVP (como en hacemos en clases)
 - a. Hipótesis
 - b. User Stories (Punto repetido)
 - c. Que no incluye



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

d. Justificar

3. Realizar la tarjeta completa de la User Story asociada al requerimiento “Crear lista de reproducción” y justificar.
4. Realizar la estimación y justificativo de la User Story que consideremos canónica (Sin tarjeta).

El examen en si no es super complicado, aun no vi mi nota pero si mas o menos le gusta lo que escribí no creo que desapruebe. Estamos a mitad de semestre y mi recomendación es que vayan los al práctico (miércoles) y únicamente entren a las clases teóricas (los viernes) de Covaro porque la Meles es **zzz** puro. Además nos tomaron lo que dio Covaro si mal no recuerdo, planificación de proyecto ágil y tradicional como también Estimaciones e ítems de configuración. Por otro lado los miércoles no falten nunca, porque es donde está el práctico 100% y se suelen hacer los TPs entregables.

Bitácora 29-11-2024

Materia cursada, en el día de la fecha estoy regular pero hice el recuperatorio del primero que me puede llegar a dar la copa Pistón (Ap Directa). Esto gracias a ciencia lo escribo desde Los Reartes, desde la tranquilidad de mi hogar y solo quedándome esta materia por aprobar (aparte de proyecto). Les comento un poco en retrospectiva, fueron muchos TP que tuvimos hasta la última semana de clases, el recu lo rendí el sábado 23 por ende estuve el último sábado del año de cursado rindiendo y dándolo todo (No quería seguir) . El recu fue fácil, fue menos complicado que el parcial y espero sacarme un 8.

Los temas del parcial no los adjunto porque la misma cátedra lo hace y creo que sería redundante. Lo único que les puedo decir es que no es tan difícil como DSI pero requiere su tiempo de cocinado. Desde mi punto de tipo chill de cojones (meme del momento) les digo que no le guardo tanto rencor a los docentes como con DSI, la verdad que sí hicieron algunas cosas más amenas, me parece que se equivocaron en algunas cosas pero no fue tan grave como el año pasado.

Bitácora 10-06-2025

Revisando este resumen por temas puramente académicos y de comentarles que en el recuperatorio me saque 8 mi teoría es porque gandhi en persona (La Georgi) me lo corrigió. La felicidad que sentí el día que me dieron la nota me permitió seguir con mi miserable vida porque ahora me la paso re feliz sin tener que ir a cursar y dando clases.

Desde ya muchas gracias, nos vemos en el próximo resumen.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

TEMARIO

Unidad N° 1

Título: Ingeniería de Software en Contexto

Contenidos:

- La disciplina de Ingeniería del Software. ¿Qué comprende la ingeniería de software?
- Estado Actual y Antecedentes. La Crisis del Software.
- Disciplinas que conforman la Ingeniería de Software.
- Ejemplos de grandes proyectos de software fallidos y exitosos.
- Ciclos de vida y su influencia en la Administración de Proyectos de Software.
- Procesos de Desarrollo Empíricos vs. Definidos.
- Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software
- Criterios para elección de ciclos de vida para proyectos de desarrollo de software.
- Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Unidad N° 2

Título: Gestión Lean-Ágil de Productos de Software

Contenidos:

- Gestión de Proyectos basados en procesos definidos. Planificación para el desarrollo y el mantenimiento de productos de software. Métricas y Estimaciones de Software
- Manifiesto Ágil/Filosofía Lean
- Requerimientos en ambientes lean ágil
- Requerimientos en ambientes ágiles - User Stories
- Estimaciones en ambientes ágiles
- Frameworks SCRUM a nivel equipo y escala - Métricas Ágiles
- Framework Kanban en el contexto del desarrollo de software – Métricas Lean
- Gestión de Productos de Software – Planificación de Productos – Herramientas para
- Definición de Productos de Software
- Lean UX
- Design Thinking

Unidad N° 3

Título: Gestión del Software como producto

Contenidos:

- Conceptos que conforman la disciplina Gestión de Configuración de Software.
- Planificación de la Gestión de Configuración de Software.
- Actividades relacionadas con la Gestión de Configuración.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- El rol de las líneas base y su administración.
- Elementos de configuración del Software.
- Identificación de ítems de configuración en la Configuración de un software.
- Gestión de Configuración en ambientes ágiles
- Prácticas continuas (Continuous Integration /Continuous Delivery/ Continuous deployment)
- Despliegue de producto. Estrategias para el despliegue de productos.

Unidad N° 4

Título: Aseguramiento de Calidad de Proceso y de Producto

Contenidos:

- Conceptos generales sobre calidad.
- Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
- Actividades relacionadas con el Aseguramiento de la Calidad del Software.
- Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).
- Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad. Proceso de Auditorías
- Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.
- Técnicas y Herramientas para la realización de revisiones técnicas del software.
- Testing en ambientes Ágiles.

UNIDAD N° 1

Software, Ingeniería y ustedes

Definiciones

Como buena materia de nuestra carrera recibimos un montón de definiciones al aire que pueden ir a un parcial (o final). Entre ellas tenemos la famosa palabra, software (Mercancía Blanda) que en esta cátedra recibe una definición particular. Un Software es un conjunto de programas que sumado a toda la documentación requerida permite definir, desarrollar y mantener los programas ejecutables una vez que se le entregan al cliente. Es por eso que nuestra querida carrera se basa no solo en un montón de programitas que funcionen sino en todo el proceso de desarrollo, en cómo planificamos, creamos, estimamos y gestionamos los recursos de un proyecto para llegar a un producto de software.

Debemos entender que en nuestra carrera estaremos viendo 2 tipos de software:

- **Los genéricos:**



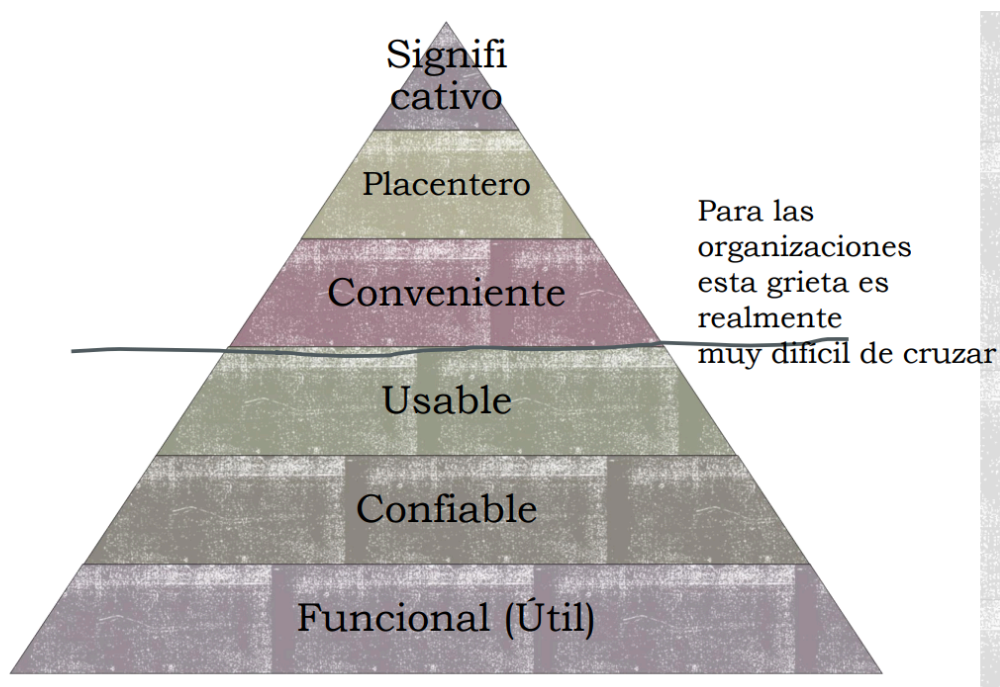
INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Básicamente todo lo que escuchaste nombrar como SAP, Tango, Manager 2, entre otros “enlatados”. Refiere a software que se crea pensando en un modelo de negocio pero no se ajusta a la realidad particular de la empresa. En general se vende por “módulos” que representan pequeñas partes y las mismas empresas brindan soporte.
- **A medida:**
 - Es como lo estudiamos en la carrera, es un producto en el cual estamos en contacto con el cliente y orientamos el mismo para que se ajuste a su realidad. Esto lleva a que sea “personalizado” que esté orientado a resolver las necesidades del cliente y como vimos en otras materias debemos tomar los requerimientos y charlar con nuestro benefactor.

Reseña histórica

En la antigüedad (cuando inventan las compus) existía el desarrollo de programas en pequeña escala, donde los equipos no superan las pocas personas ya que el hardware no había avanzado lo suficiente para que surja esa necesidad. Con el tiempo, a medida que evolucionó la tecnología teníamos la posibilidad de desarrollar cosas mucho más complejas y avanzadas. Acá es donde entramos en lo que vimos en DSI de cómo hacíamos software (Lineal, Cascada, etc) para crear producto de software. Nos encontramos entonces con una alta demanda de desarrollo de software, nuevas tecnologías pero poca capacidad para gestionar todos estos recursos. En algún momento se empieza a hablar de como tenemos un enfoque “tradicional” y uno “ágil” pero eso lo explico cuando hablemos de Scrum y mierdas así.





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

La mencionada Crisis

Inevitablemente la comercialización de los primeros PCs (Años 60) creó un nuevo mercado en auge que se empezaron a usar en muchos sectores. Esto evidenció una falta de problemas que tenía el desarrollo de software, que eran ad hoc (Artesana), carecían de rigor estructural y no podían gestionar adecuadamente el desarrollo para sistemas cada vez más complejos. Inevitablemente la bola de nieve que generó la demanda de software trajo los siguientes problemas:

- **Retrasos en la Entrega:** Se estiman mal los tiempos por la falta de experiencia
- **Sobrecostos:** A veces consecuencia de lo primero, para intentar llegar a tiempo se gastaba mas recursos de los planteados.
- **Calidad Deficiente:** Como cuando entregas un TP a las apuradas hecho por chat GPT, tiene dudosa calidad.
- **Mantenibilidad Limitada:** Al estar mal hecho pasaba que era difícil de darle mantenibilidad a largo plazo.
- **No cumplían los requisitos:** Pecado capital que desarrollamos un software que no necesitaba el cliente.

Aca el colectivo de señoras mayores, pendejas sin experiencia laboral y personas LGBT+ que conforman las cátedras de DSI e ISW les encanta recalcar como lo que ellas enseñan salvó el mundo.

Algunos de los avances que nos han permitido gestionar un poco la crisis son:

- Metodologías ágiles
- Automatización y herramientas avanzadas como por ejemplos los entornos integrados de desarrollo (IDE), y sistemas de gestión de versiones como GIT

Disciplinas de la ingeniería

Como ingenieros debemos entender que es una disciplina que se encarga de la producción de software desde la primera etapa de especificación del sistema hasta su mantenimiento post creación.

En cuanto a las disciplinas tenemos (Extracto de Jc):

“Sinceramente, tampoco es muy útil esto, pero lo pongo para que se entienda que carajo hacemos dentro de la ingeniería de software.

- Toma de requerimientos
- Análisis de requerimientos
- Diseño de software
- Implementación de software
- Despliegue de un software
- Documentación de software
- Capacitación de usuarios



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Disciplinas de gestión
- Disciplinas de soporte”

“La ingeniería de software es el establecimiento y uso de principios fundamentales de la ingeniería con objeto de desarrollar de forma economica software que sea confiable y que trabaje con eficiencia en máquinas reales”

- Resumen de Carlitos -

Éxitos y Fracazos en Proyectos

Contexto

En este punto de ejemplos podemos hablar de muchos fracasos (como yo) y de éxitos en nuestra industria. El tema es que a veces el fracaso financiero no significa lo mismo del proyecto ya que aunque este bien, se haya gestionado bien y hayamos tenido buen feedback no hicimos el software que la pegó en el mercado por otros factores. Si hubiera ido a clases le podría ofrecer un ejemplo más concreto de esto, pero lo que tienen que recuperar de las experiencias de nuestros antecesores en el rubro son la lecciones que nos dejaron. Recuerdo de clases de que uno de los libros está escrito por un tipo que le salió todo mal y registro todo el proceso para que nosotros nos podamos hacer una idea.

Entre cosas que noto es que cuando no estás llegando con el tiempo muchos tienden a contratar más personas para llegar antes o aumentar los salarios de manera estratosférica para retener a los que ya están. En lo que yo he visto muchas veces es que gente de rango superior interfiere en las decisiones de la área a la cual le estamos desarrollando el software. Ahí entramos en disruptiva en el cual la persona que te paga te pide una cosa pero la persona que sabe del negocio y le estamos el programa para el quiere otra. Mantener estos intereses separados es complicado debido a que en la diaria (sobre todo en Argentina) no se sigue al pie de la letra el reglamento o se hace cosas por fuera para después dibujar los números mejor.

Sin darle muchas más vueltas les dejo el listado de cosas que son clave de éxito y cosas en las cuales podemos pecar.

Claves de éxito

- Monitorea y Feedback
- Tener misión y valor claros
- Comunicación entre todos los actores

Claves de Fracaso

- Fallas al definir el problema
- Planificación basada en datos insuficientes
- Planificación realizada por ajenos al equipo



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- No hay seguimiento del proyecto
- Mala gestión de los recursos
- Estimación sin históricos
- No hay nadie a cargo

Ciclos de vida y si vale la pena vivirlos

Modelo Secuencial

Realización de actividades de forma lineal, las actividades no se pueden iniciar sin antes haber finalizado la linealmente anterior. Este modelo probablemente lo hayan escuchado nombrar en los casos del desarrollo en cascada. Sin ser extenso nos encontramos con una secuencia de pasos que debe completarse en su **TOTALIDAD** antes de arrancar la siguiente etapa.

En wikipedia encontramos esta serie de pasos:

1. Análisis de requisitos.
2. Diseño del sistema.
3. Diseño del programa.
4. Codificación.
5. Pruebas.
6. Despliegue del programa.
7. Mantenimiento.

Cuando lo elegimos:

- Modelos de cascada puro, minimizando los gatos.
- Productos con definiciones muy claras.
- No hay apuro de ver resultados medios, solo se busca el resultado final

Dificultades:

- Raramente un proyecto de la vida real tenga flujos secuenciales
- El cliente solo ve en que vas cuando terminas
- Encontrar defectos generalmente cuando lo entregaste
- Los tipos de controles son tan cercanos al plan que se vuelven burocráticos

Modelo Interactivo-Incremental

Este es el predilecto por la cátedra, el hijo bobo que mencionan todo el santo rato para que creas que es lo único en la vida. Según este modelo es la aplicación sucesiva de iteraciones de forma **escalonada** que genera un incremento en software a medida que va generando “entregables”.

Lo bueno:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Fácil de adaptar para requerimientos cambiantes (ósea en los proyectos de la vida real)
- El cliente es parte del proceso de desarrollo.
- Hay entregas más rápidas de partes útiles para el cliente.
- La especificación, desarrollo y validación están.

Lo malo

- Invisibiliza el proceso
- Alto costo de documentar
- La estructura suele a degradar

Lo feo

- A la Meles le encanta hablar

Modelo Recursivo

De este tema no tengo noción de donde salió así que paso a copiar así plano lo de otros compañeros.

Recursividad utilizada para la gestión de los riesgos en sistemas complejos y a gran escala, requiere de gran intervención del cliente, es como un espiral recursivo con 4 etapas:

1. Determina objetivos
2. Identificar y resolver riesgos
3. Desarrollar y probar
4. Planificar la próxima iteración

Dificultades

- Puede ser costoso
- La tecnología puede ser obsoleta
- Puede crecer demasiado mantenimiento o documentación
- La versión es recién al final.

Procesos Empiricos

Diferencias con definidos

Temita que les guste a las docentes que sepamos cual mantra, se repita y se entienda. Se suele relacionar mucho con la crisis del software y escenarios inciertos que presentan dificultades a la hora de estimar los recursos.

En la producción “física” que surge de la industrialización (manufactura) tenemos que cada pieza busca ser homogénea, los tiempos son predecibles, nos basamos en **procesos definidos** dictados por el mismo determinismo y como mucho en un rango de probabilidades acotado dando siempre el **mismo producto**. Por otro lado en el software no desarrollamos 2 veces el mismo programa (porque sino lo copias y ya), esto hace que sean inciertos y caóticos. La mayoría de las



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

veces ya que estábamos experimentando con cosas nuevas llevando a que se usen **procesos empíricos** para poder tener una noción de los tiempos y recursos.

Expandiendo sobre los procesos definidos, se habla de un proceso donde se conoce todo lo que debemos hacer, tenemos un alcance fijo el cual asumimos que el cliente no se olvido nada. Para procesos que no son de software este punto de vista puede ser útil ya que se entiende que tenemos que hacer y hasta dónde debemos llegar. En los proyectos de sistemas esto no pasa, casi nunca vamos a saber la totalidad del alcance apenas arrancamos, sino que se completará a medida que avanzamos.

Definición:

*“El **proceso de desarrollo** es una secuencia de pasos ejecutados para un propósito, aplicamos este concepto en el software y podemos decir que es un conjunto de actividades, métodos, prácticas y transformaciones que se usa para desarrollar o mantener un software.”*

Agilidad y Pilares

Desde el enfoque ágil (que veremos más adelante) se basa en el empirismo y asume que los requisitos no son fijos estando sujetos a cambios. Es por eso que el empirismo se basa en la recolección de experiencias de manera constante, tanto individual como colectivo (equipo) generando conocimiento en el proceso. El conocimiento es más que la información ya que es la aplicación de la misma en el contexto dado. Por lo que la experiencia que juntemos en un equipo de trabajo, en una empresa concreta es aplicable pero no extrapolable a todas las empresas y equipos.

A ver, acá las viejas no aclaran (creo) pero uno puede extrapolar su experiencia personal o buscar una generalización a la hora de realizar estimativos. Lo aclaro porque decir que la experiencia no es extrapolable es ser un termo (opinó).

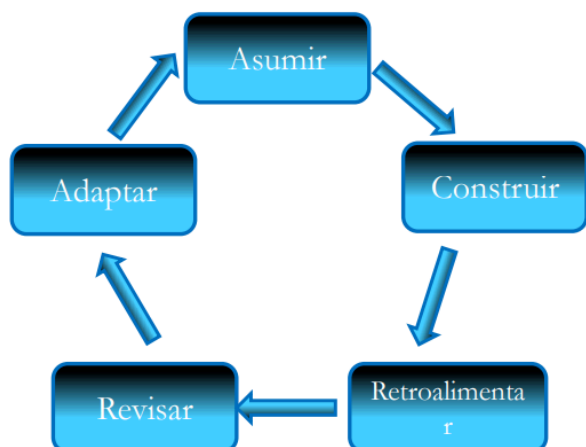
El empirismo tiene 3 pilares:

1. **Transparencia:** Mostrar nuestras acciones de manera clara, siendo sincero con todas las partes del proyecto y tener un diálogo abierto en lo posible.
2. **Inspección:** Los equipos deben identificar los errores mediante evaluaciones periódicas y fomentar la mejora constante.
3. **Adaptación:** Obviamente la inspección nos va a mostrar los errores que debemos ajustar en función de los conocimientos adquiridos. A medida que los equipos descubren nueva información y entienden mejor la dinámica.



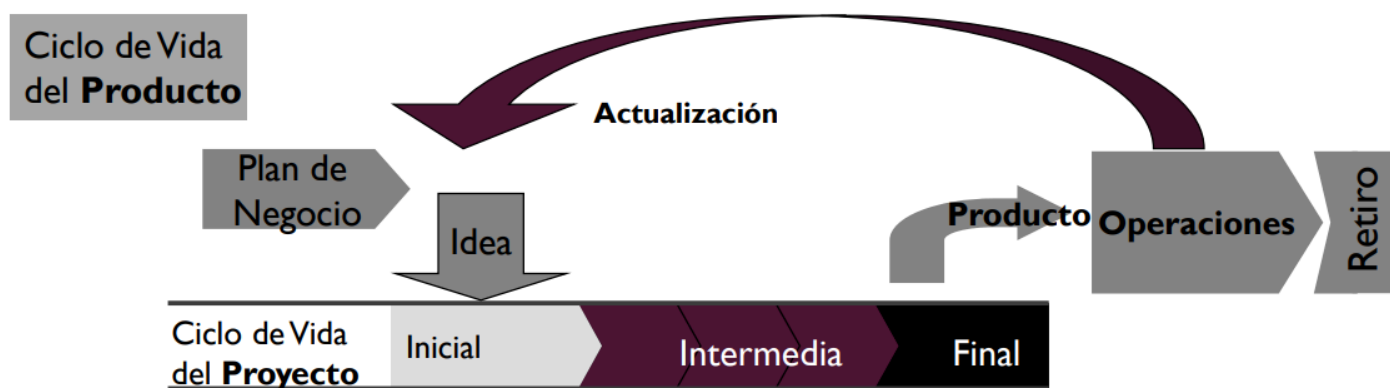
INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Ciclos de Vida

El ciclo de vida de un proyecto de software representa un proceso, una descripción del mismo desde una perspectiva en particular. Para ello debemos aclarar las fases del mismo, en que orden se van a llevar y qué características van a tener.



Code and Fix

Se desarrolla sin especificaciones o diseño y se modifica hasta que el cliente esté satisfecho. Los cambios se realizan durante el mantenimiento, lo que es muy claro.

Este modelo tiene como ventajas:

1. No se dedica tiempo de “análisis”.
2. Requiere poca experiencia.
3. Tano Like This

Desventaja:

1. No es escalable



INGENIERIA Y CALIDAD DE SOFTWARE

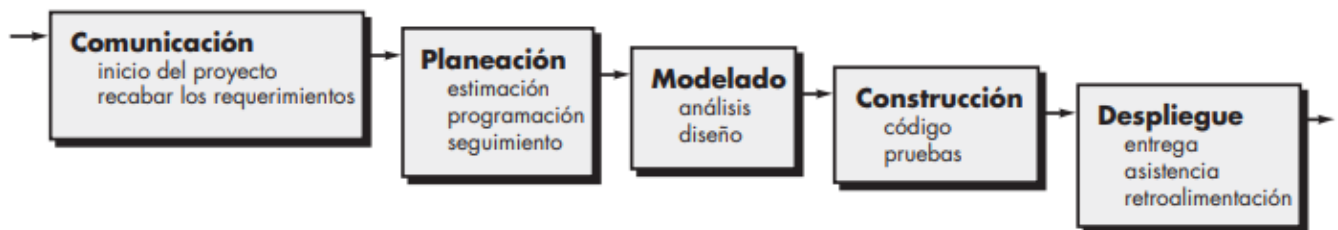
RESUMEN FINAL Y ULTRAVIOLENTO

2. Caro de mantener
3. No está documentado
4. Madurar es darse cuenta que el análisis sirve

Estamos hablando de una forma de programar “reactiva” a veces esta forma se usa para sacar prototipos lo más rápido posible para cubrir una necesidad inicial del cliente y terminas teniendo una arquitectura espagueti (**Big ball of Mud**).

Cascada puro

Modelo de la cascada



El modelo de la cascada a veces llamado clásico o tradicional establece un enfoque sistemático y secuencial para el desarrollo de software. Como cada parte debe ser revisada antes de pasar a la siguiente etapa hace que se reduzcan las cantidad de errores en el producto final. Ahora bien la realidad del software es cambiante y este ciclo de vida al ser rígido no permite volver atrás para cambiar un requerimiento previamente planteado. El otro defecto que pasó durante muchos años es que el cliente solo verá el software cuando se termine un ciclo de vida completo, osea que durante 6 meses laburamos y cuando entregamos el software el cliente dice que no le gusta. Esto hace que no nos paguen y encima trabajamos al pedo, ocurría mas de lo que se imaginan.

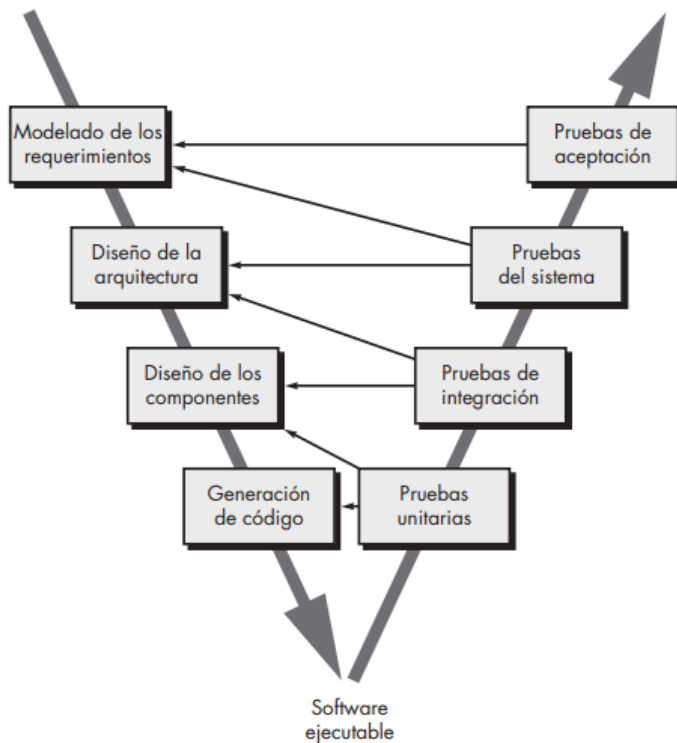
Tiene sus consideraciones, para equipos con poca experiencia y con pocos recursos es probablemente la forma mas cómoda de desarrollar. No se recomienda en proyectos grandes, escalables o que tenga requerimientos inciertos.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Cascada en V



No escapa mucho a su contraparte en “I” pero tenemos una representación más gráfica de un proceso donde del lado izquierdo creamos y en el lado derecho validamos con la posibilidad de rehacer trabajo de manera más cómoda.

Cascada con fases solapadas

En términos generales es lo mismo que vimos antes nomas que las tareas pueden empezar sin que esté el 100% completado.

Cascada con subproyecto

Mantiene la secuencialidad, pero pero, se subdivide en proyectitos de manera lineal hasta el diseño arquitectónico y de ahí proyectitos x módulos.

Incremental

Vemos algo tangible de salida, es útil para entregas funcionales de urgencia, y con alto grado de modificación. Solo funciona con buena planificación.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Espiral

Minimizamos riesgos mediante el análisis de riesgo, buscamos alternativas para iniciar cada fase, una vez cerrado el cada ciclo se planifica el siguiente ciclo. En palabras de los docentes, hacemos mini proyectitos.

Características

Desgraciadamente tenemos que profundizar en este modelo porque hay que saberlo:

- Permite evaluar la facilidad de un nuevo producto.
- Ayuda a la gestión de riesgo. (reduce costos potenciales)
- Adecuado para modelos de vida largo.
- El costo aumenta a su vez que el riesgo disminuye.
- Se fijan objetivos o hitos.

Evolutivo

Se fija tiempo o alcance, cuando se elige uno el otro se va modificando, sirve cuando los requerimientos no son claros y se realizan entregas tempranas al cliente.

Diseño para cronograma

No se asegura versión final, aunque si una entrada del producto para la fecha definida. Tiene que entregar al menos un mínimo negociado, es útil cuando no se conocen las capacidades de programación.

Diseño para herramientas

La funcionalidad está soportada por herramientas de software existentes. Se usa en proyectos muy sensibles al tiempo.

Prototipo evolutivo

Lo vimos en DSI, se usa para cambios de requerimientos rápidos y clientes poco comprometidos con los requerimientos.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Proyectos

Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Se adapta el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.

Características

1. Son **ÚNICOS**, lo que significa que son irrepetibles, por más parecido o igual que sean sus elementos.
2. Pueden ser **ORIENTADOS A OBJETIVOS**, para esto, no tienen que ser ambiguos, deben ser medibles, deben ser alcanzables, tener una duración limitada y las tareas interrelacionadas entre esfuerzo y recursos.
3. Tienen una duración limitada, un proyecto es una unidad de organización temporal que cuando el objetivo se cumple se da por finalizado.

Administración de proyectos de software

Contiene básicamente dos actividades: “Planificación” y “Monitoreo y Control”

Otras tareas puntuales son identificar requerimientos → establecer objetivos claros y alcanzables → adaptar las especificaciones, planes y enfoques a la vista de los interesados.

Metas más importantes de la administración

1. Entregar en el tiempo acordado
2. Mantener los costos dentro de lo presupuestado
3. Entregables a la altura de las expectativas del cliente
4. Mantener un equipo de desarrollo óptimo

Complejidades

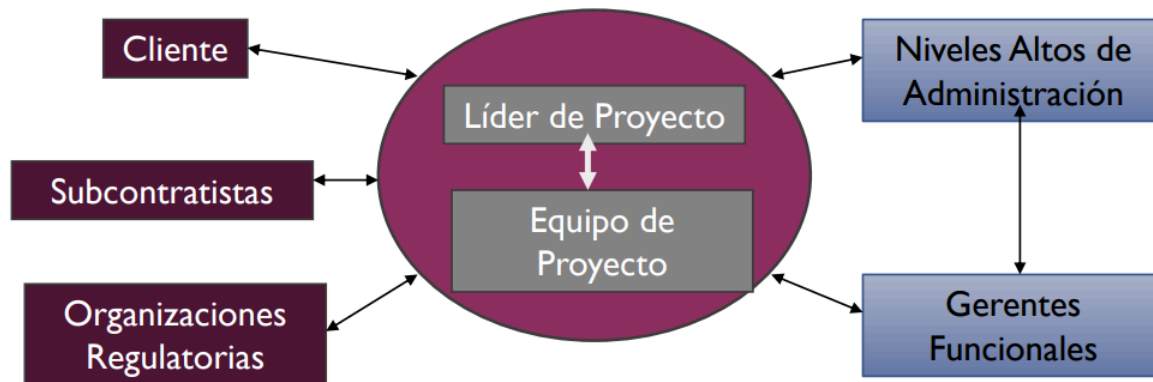
1. El producto es intangible
2. El producto por lo general es único y excepcional
3. Los procesos de desarrollo son variables.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Roles dentro de un proyecto



Líder de proyecto

Para los pibes, es un tipazo, tiene que tener motivación para que todo el equipo e interesados estén motivados a seguir el proyecto, debe organizar para que todo salga bien, tiene que buscar siempre cosas que permitan creatividad y la comodidad para esto, debe ser empático y entender a los otros actores y sus contextos, posee buena comunicación y transparencia y además es un líder tremendo.

Además de todo esto tiene sus grandes habilidades técnicas dentro del rubro sistemas como administrativos y si no sabe consigue el número de quien sí sabe.

Equipo de proyecto

Es el grupo que se encarga de desarrollar el proyecto, tienen un objetivo común y se deben sentir todos mutuamente responsables. Tienen los conocimientos necesarios, suelen ser atómicos para una comunicación efectiva, trabajan con espíritu de grupo y se buscan metas grupales.

Stakeholders

Son los interesados de que el proyecto salga, suele ser accionistas, gerentes, quienes ponen la plata y también incluyen a los clientes. Son básicamente aquellos actores que se van a ver afectados por el desarrollo del producto.

Agrego un par que son propios del SCUM como para no explicarlos después



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Product Owner (PO)

Es la persona dedicada a maximizar el valor del producto, tiene la visión de cómo debería ser el software, que funcionalidades, características y descomponerlo en el product Backlog. Idealmente tiene que ser una persona familiarizada con el SCRUM (no pasa nunca) que tenga el conocimiento óptimo del negocio y **que pueda tomar las decisiones**. Esto es importante ya que si cada vez que tiene que generar un requerimiento le tiene que preguntar a alguien más puede haber errores. Esto es porque la otra persona puede tener otra visión y en muchos casos si es un cargo superior no esta en la diaria del proceso.

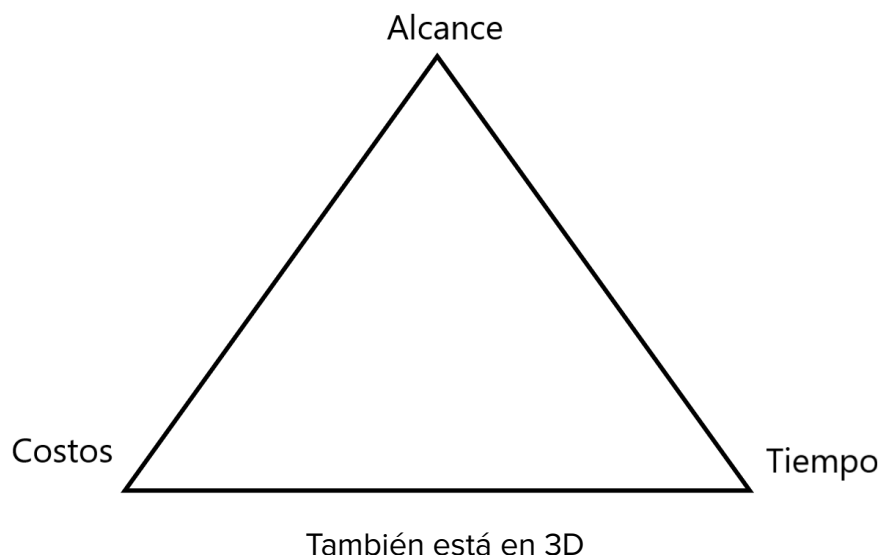
SCRUM Master

Este payaso es el que organiza el circo alrededor del SCRUM, vela por seguir las ceremonias pactadas (que veremos más adelante) y molesta a todo el mundo con las reuniones. En general esta persona debe ser experto en SCRUM y puede ser flexible en algunas cosas, aunque no te lo diga puede cambiar las ceremonias de acuerdo a la situación. Esto no lo escriban en el parcial porque le da asco que no seamos unos ortodoxos. Podría hacer esto que diga porque está velando por el dinamismo del equipo y seguir con la ideología ágil, si una reunión no aporta valor ese día por algún evento se puede no hacer.

La triple restricción

Tiempo, costo y alcance. Hace muchos años esos elementos vivían en armonía, pero todo cambió cuando la nación del SCRUM atacó... Solo el scrum master, maestro de todas las ceremonias podía detenerlos. Pero cuando el mundo más lo necesitaba, renuncio.

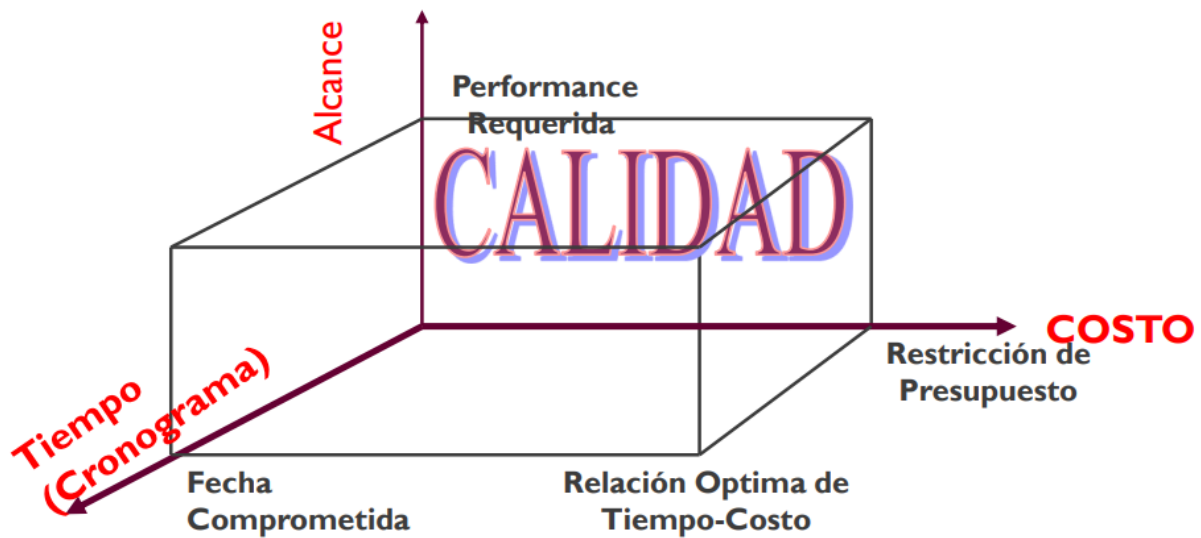
El balance de estos 3 factores afectan directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Si bien es un tema dentro de administración de proyectos de software, lo pongo como título ya que es muy importante tanto para parciales como para el final.

Como vemos en el gráfico, el triángulo que se forma al medio es la triple restricción, debemos definir qué vamos a hacer, esto le llamamos alcance, después el tiempo que nos va a llevar y el costo que tenemos. Todo esto tenemos que definir para un proyecto exitoso, el costo lo suelen poner con el presupuesto total sumado a una buena gestión de riesgo, el alcance se basa en que necesidades de la organización que se desean atender con este producto de software, y el tiempo es definido por la urgencia con la que tenemos que resolver con este producto.

Plan de proyecto

Es un artefacto de gestión que funciona como “hoja de ruta”, donde se documenta cada decisión, se define en procesos, asignación de responsabilidades, formación de equipos, calendario de entregables y actividades de soporte. Es diferente el plan de un producto o el alcance del mismo con respecto al del proyecto, ya que el último no solo abarca la creación del software sino todas las decisiones, reuniones, configuración y pautas que vamos a seguir siguiendo hasta el final del proyecto.

El plan debe documentar:

1. ¿Qué es lo que hacemos?
2. ¿Cómo lo hacemos?
3. ¿Qué riesgos existen?
4. ¿Quién lo va a hacer?
5. ¿Cuándo lo hacemos?
6. ¿Cómo sabemos que está bien hecho? (Mejorar)



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Ahora que sabes que es un plan de proyecto, vamos a profundizar en cada paso que tenemos que realizar para generar este artefacto.

1. Definición de alcance

Es la búsqueda que tiene que lograr el producto fruto del proyecto, se suele hacer en función de **ERS** (Especificación de Requerimientos) y llevándolo en perspectiva de los objetivos del proyectos. Si los requerimientos a resolver coinciden con el objetivo del proyecto, estamos por buen camino, en caso contrario debemos revisar que onda.

2. Definición del proceso y ciclo de vida

Retomamos un concepto que vimos hace unos temas, teniendo en cuenta los recursos y el contexto del proyecto y sobre todo el tiempo y presupuesto que brinda el cliente. Una vez que tenemos todo en cuenta se elige como serán los procesos y que ciclo de vida tomarán, es importante que sepamos que los ciclos de vida tradicionales tienen mayor flexibilidad de selección de ciclo de vida, en cambio las metodologías ágiles tiene que ser si o si iterativo-incremental.

3. Estimación

Consta en intentar predecir el valor de un elemento relacionado con el proyecto, el tiempo que va a llevar cada cosa, cuantos van a ser los actores designados. En un enfoque tradicional estas estimaciones las hace solo el líder de proyecto. No profundizamos ahora en las estimaciones ágiles así que voy a enumerar las que hace el líder de proyecto en el enfoque tradicional:

1. **Tamaño del producto**, se hace mediante los requerimientos que se conocen, es una de las estimaciones más valiosas porque es un dato fundamental para las otras estimaciones.
2. **Esfuerzo**, permite en función de qué tan grande sea el proyecto, la cantidad de horas de trabajo que se va a terminar necesitando.
3. **Calendario**, esto define las fechas de cada entrega o cantidad de proyecto que se hará en cierto periodo, esto se da en función de las anteriores 2 estimaciones.
4. **Costos** que depende de qué tan grande y cuánto esfuerzo requiere, pone precio al proyecto.

4. Gestión de riesgo

El riesgo es la posibilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.

Hacer gestión de riesgo es una gran clave de éxito, identificarlos en primera instancia es algo sumamente importante, después nos lleva a 3 instancias negarlos, reactivos o proactivos. En palabras que a mi me gustan: 1.hacernos los giles, 2.correctivos y 3.preventivos. Esto podemos relacionarlo a los ejercicios de simulación que nos demostraron que es más caro corregir las cosas cuando explotan que prevenir.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

5. Asignación de responsabilidades

Alguien tiene que hacer el trabajo, por ende llega el momento se debe definir quien va a hacer que, por ende se le dan lineamiento de que va a hacer cada persona dentro de los equipos de trabajo.

6. Calendario

Para presentar cualquier proyecto con todo lo anterior definido, establecemos fechas calendario, bajándolo a la realidad al proyecto, por lo general usamos herramientas como diagrama de gantt para mostrar esto.

7. Métricas

Una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, producto o proceso en un momento determinado del desarrollo. Cuando planificamos debemos definir de forma clarísima, ósea 0 ambigua, cada una de las métricas asociadas a cada dominio en conjunto a una forma de calcularlas. Las marcas seleccionadas nos deben dar información que beneficie a la practicidad del desarrollo y el seguimiento.

Las métricas más usuales son:

- Tamaño del producto
- Esfuerzo
- Calendario (Tiempo)
- Defectos
- Costos

Las métricas las podemos separar por el dominio o dimensión que cumplen:

Métricas de proceso: Saber en términos de organización la manera en la que estamos trabajando. Las métricas de proceso son una despersonalización de las métricas del proyecto ➔ Es decir que. todas las métricas que son del proyecto, también lo son del proceso

Métricas de proyecto: Son las que me permiten saber si un proyecto de software que está en ejecución se está cumpliendo de acuerdo a lo planificado o no

Métricas de producto: Directamente relacionadas con el producto de software que estamos construyendo, controlando que cumpla los objetivos y metas planteados .

8. Controles

Por último debemos ver cómo vamos a controlar el desarrollo del proyecto, se toma como base para controlar la herramienta ya predefinida, las métricas, verificándolas para que se busque un resultado deseado, también se define la periodicidad del control y la información necesaria para



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

llevarlos a cabo. Por fuera de estas actividades y de forma opcional también puede existir una definición del soporte del proyecto en caso de ser necesario, ya sea para asegurar calidad como con el plan de testing como para darle mantenimiento.

EL SUEÑO DEL PIBE...

• Desarrollador

1. Esfuerzo
2. Esfuerzo y duración estimada y actual de una tarea.
3. % de cobertura por el unit test
4. Numero y tipo de defectos encontrados en el unit test.
5. Numero y tipo de defectos encontrados en revisión por pares.

• Organización

1. Tiempo Calendario
2. Performance actual y planificada de esfuerzo.
3. Performance actual y planificada de presupuesto
4. Precisión de estimaciones en Schedule y esfuerzo
5. Defectos en Release

• Equipo de Desarrollo

1. Tamaño del producto
2. Duración estimada y actual entre los hitos más importantes.
3. Niveles de staffing actuales y estimados.
4. Nro. de tareas planificadas y completadas.
5. Distribución del esfuerzo
6. Status de requerimientos.
7. Volatilidad de requerimientos.
8. Nro. de defectos encontrados en la integración y prueba de sistemas.
9. Nro. de defectos encontrados en peer reviews.
10. Status de distribución de defectos.
11. % de test ejecutados

Vinculo Proceso-Producto-Proyecto (Triple P)

El proceso de desarrollo de software es variado debe incluir cuatro actividades fundamentales:

1. **Especificación de software:** Tienen que definirse tanto la funcionalidad del software como las restricciones de su operación
2. **Diseño e implementación del software:** Debe desarrollarse el software para cumplir con las especificaciones
3. **Validación del software:** Hay que validar el software para asegurarse de que cumple lo que el cliente quiere
4. **Evolución del software:** EL software tiene que evolucionar para satisfacer las necesidades cambiantes del cliente

Momento de condensar y relacionar todo lo que vimos en esta unidad, es importante ya que si nos evalúan respecto a cualquier tema de la unidad, podemos nuclear ese tema con esta síntesis. Los



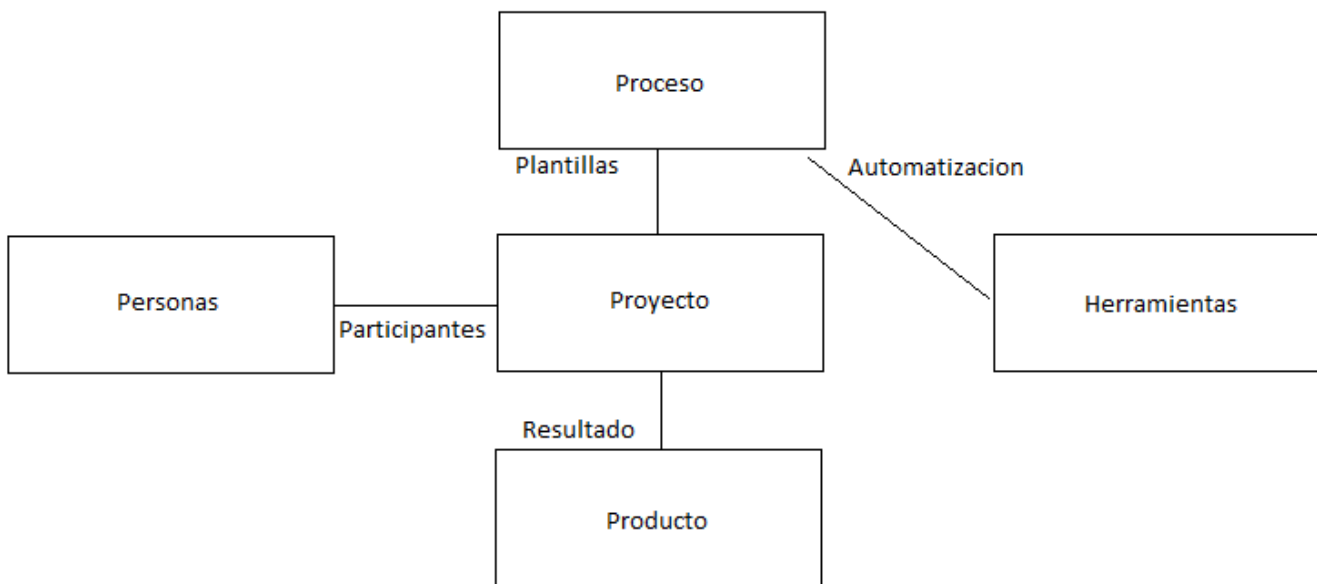
INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

procesos son automatizados mediante herramientas utilizadas, los procesos se adaptan al proyecto, al cual se le incorporan personas y de este proyecto se obtiene como resultado un producto.

En un enfoque tradicional para definir el proyecto es necesario determinar los objetivos y el ámbito del producto para de esa manera estimar de forma coherente. Los objetivos serán las metas globales del producto, el ámbito del producto define las funciones y comportamientos que toma el usuario al tomar el producto. Para obtener este resultado es necesario adaptar un proceso, donde definiremos actividades, métodos, prácticas y transformaciones que utilizamos para obtener el software. El proyecto está integrado por un equipo de trabajo, los cuales deben estar los roles bien definidos. Esto es lo más importante de todo, procesos adaptados, en proyectos que persiguen un objetivo centrado en la gente que labura para llegar a este. Adicionalmente se agregan herramientas que facilitan los procesos adaptados.

Si tuvieron clases con la Meles probablemente vieron en amplio resumen es cómo se conecta un proyecto con los demás conceptos que aprendemos en la materia, proceso, personas, productos y herramientas.



Sin Balitas de Plata (Tesis)

El artículo "No Silver Bullet: Essence and Accidents of Software Engineering" de Frederick P. Brooks, Jr. aborda las dificultades inherentes al desarrollo de software y argumenta que no existe una solución mágica ("silver bullet") para mejorar la productividad o la simplicidad del software de forma significativa.

Es un ensayo que distingue dos tipos diferentes de complejidad; la accidental y la esencial. La accidental (Como tener hijo a los 15), se refiere a problemas que los ingenieros crean y pueden solucionar, por ejemplo, los lenguajes modernos de programación son tan abstractos que sacan los detalles de optimización de código base (Assembler). Por otro lado, la **complejidad esencial** (Es invisible a los ojos) está causada por el dominio del problema que queremos solucionar y nada lo



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

puede sacar. Por poner un ejemplo, todo el sistema de validación legal que tiene un rubro comprende una complejidad esencial ya que es inherente a lo que queremos construir.

1. Esencia y Accidentes del Software:

- Brooks divide las dificultades en "esenciales" y "accidentales".
- Las **dificultades esenciales** se refieren a la complejidad inherente al diseño conceptual del software (complejidad, conformidad, cambiabilidad e invisibilidad).
- Las **dificultades accidentales** son las que se presentan en la implementación, como errores de sintaxis, que han sido parcialmente mitigadas con avances como lenguajes de alto nivel y entornos de desarrollo.

2. Las Propiedades Inherentes del Software:

- **Complejidad:** El software es intrínsecamente más complejo que la mayoría de las construcciones humanas, debido a su naturaleza abstracta y su gran cantidad de interacciones internas.
- **Conformidad:** El software debe ajustarse a restricciones externas, como hardware y protocolos, lo que añade más complejidad.
- **Cambiabilidad:** Es más fácil cambiar el software que productos físicos, pero esta facilidad conlleva que esté sujeto a más modificaciones, lo que lo vuelve propenso a errores.
- **Invisibilidad:** No se puede visualizar el software de manera geométrica, dificultando la comprensión y la comunicación.

3. Falta de un Progreso Radical:

- Brooks argumenta que no existe ninguna tecnología emergente que pueda proporcionar mejoras de productividad en un orden de magnitud, como sí ocurrió en hardware con avances como los transistores.
- Las técnicas actuales como programación orientada a objetos, verificación formal, o inteligencia artificial pueden mejorar aspectos, pero no resuelven la esencia del problema.

4. Avances en la Eliminación de Accidentes:

- El autor reconoce que ha habido mejoras en la eliminación de dificultades accidentales mediante lenguajes de alto nivel, entornos de desarrollo integrados, y técnicas de inteligencia artificial.
- Sin embargo, estos avances no eliminan las dificultades esenciales que conforman el núcleo de los problemas del software.

5. Mejores Prácticas Propuestas:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- **Prototipos rápidos y refinamiento de requisitos:** Es difícil establecer especificaciones completas de antemano; por tanto, es mejor construir prototipos para iterar con los usuarios.
- **Desarrollo incremental:** Desarrollar software en partes pequeñas y evolucionar a partir de un núcleo funcional.
- **Creación de grandes diseñadores:** El autor enfatiza que la diferencia entre buen y excelente software radica en la habilidad de los diseñadores. Los mejores programadores producen software más rápido y de mejor calidad.

6. Conclusión:

- No hay un "bala de plata" que revolucione el desarrollo de software, y Brooks cree que es poco probable que la haya. Los avances deben centrarse en mejorar prácticas, herramientas y diseñadores, y en enfrentar la esencia de la complejidad del software con mejores enfoques.

En general, el artículo concluye que las dificultades del software son profundas y estructurales, y que no se deben esperar soluciones revolucionarias, sino mejoras incrementales y esfuerzos continuos en formación y metodología

UNIDAD N° 2

Gestión de Proyectos basados en procesos definidos

Esto un poco lo vimos cuando hablamos de Empirismo, es una forma de modelar el mundo basado en las líneas de montajes industriales. Propone que conocemos la totalidad de lo que queremos desarrollar en nuestro proyecto y que no podemos cambiar los requerimientos (a menos que sea una emergencia).

Como se habló en la unidad anterior, los procesos definidos nos hablan de que tenemos un conocimiento completo del software y sus particularidades. El proyecto en sí mismo es todo el proceso desde que arrancamos con la idea, tomamos los requerimientos, diseñamos, desarrollamos, probamos, implementamos y realizamos un plan de mantenimiento.

Les dejo de clases lo que la Covaro planteó como pasos en proyectos "tradicionales" (definidos):

1. **Definir Objetivos:** El proyecto debe tener un objetivo: claro y alcanzable
2. **Alcance del proyecto:** Es la sumatoria de tareas desde que empezamos hasta que finalizamos.
3. **Determinar el equipo:** Tenemos que elegir quienes conforman el equipo en base a las necesidades.
4. **Proceso y Ciclo de Vida:** Definimos qué procesos vamos a realizar y qué tipo de ciclo de vida vamos a utilizar.
5. **Estimaciones:**



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- a. **Tamaño:** Cantidad de CU * Complejidad
 - b. **Esfuerzo:** Horas Persona(Always)
 - c. **Tiempo:** Horarios, calendario
 - d. **Costo:** Licencias, Salarios, cara de cliente (Entre más pavos mas caro)
6. **Riesgos:** Probabilidad de que algún error ocurra. El problema es algo que tiene 100%. Por otro lado cada riesgo tenemos la exposición que es la (Probabilidad * Impacto)
- a. Pueden ser cambios de requerimientos
 - b. Se va la gente
 - c. Error cuando estimamos
7. **Calendarización:** Organizar todo con US significativas
8. **Métricas:** Es una toma de valores a lo largo del proyecto para entender el desarrollo de nuestro proyecto
9. **Reuniones e Informes:** Tenemos que planificar con quien charlamos y a quien le reportamos
10. **Plan de Soporte:** Es el conjunto de actividades que brindan soporte en el futuro y cómo definimos la gestión de configuración.
11. **Plan Iteración(Opcional):** En caso de que sea un ciclo de vida iterativo.

“PLANNING IS EVERYTHING. PLANS ARE NOTHING.”
—FIELD MARSHAL HELMUTH GRAF VON MOLTKE

Filosofia Lean

“El Lean es un método innovador que busca optimizar los procesos de gestión y productivos de la empresa que lo ponga en práctica. De este modo se utilizan menos recursos, por lo que cualquier proceso se convierte en más eficiente. Su máxima está en reducir la inversión, el tiempo y el esfuerzo”

7 Principios Lean

1 Eliminar desperdicios

Es quizás el más importante, debemos tener un proceso que sea eficiente y le sume valor a todos los pasos del proceso. Todo paso o parte que no contribuye a la generación de valor produce un desperdicio.

1. Características extras [producir productos extras]
2. Trabajos a medias [acumulación de stock]
3. Proceso extra
4. Búsqueda de información
5. Defectos
6. Espera [costos logísticas]
7. Cambios de tareas [Multitasking]



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

2 Amplificar el aprendizaje

Es relativo a la transparencia y transformación del conocimiento de lo implícito en explícito, con acceso a todos lo cual fortalece el equipo de trabajo.

3 Embeber la integridad conceptual

La calidad del producto no se negocia, el producto tiene que satisfacer las expectativas del cliente, es la principal prioridad y ningún otro aspecto debe desviar de ese foco.

4 Diferir compromisos hasta el último momento responsable

La toma de decisiones se hace cuando se tiene la información realmente necesaria para hacerlo.

5 Dar poder al equipo (Autogestión)

Otorga libertad de decisión y poder de acción al equipo, esto les permite ser autogestionados, con sus miembros capacitados y motivados esto permite una mayor entrega hacia el valor agregado del producto final.

6 Ver el todo

Una visión holística del todo por encima de las partes, para que los objetivos particulares estén alineados con los objetivos globales.

7 Entregar lo antes posible

Busca salida rápida del software para entregar valor de un producto aunque sea mínimo y poner en competencia al cliente.

Manifiesto Ágil

Agilismo

Hace más de 20 años se juntaron un conjunto de referentes de la industria del software y generaron un acuerdo de “principios” que dieron forma al Manifiesto Ágil. Según la profe el agilismo es un movimiento que se gestó desde los desarrolladores (a chequear).



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



¿Pero qué es el agilismo?

Esto refiere a una ideología, es una forma de vida (Hakuna Matata) y no está relacionado a una metodología o proceso único. Es el conjunto de ideas que buscan un mundo feliz para mejorar el rendimiento de los equipos de trabajo.

Los equipos ágiles buscan:

- Planificación Continua
- Equipos autoorganizados
- Entregas frecuentes
- Practicas de ingeniería disciplinadas
- Integración Continua
- Testing Concurrente
- No es hacerlo por partes

La agilidad busca que exijamos menos documentación, reuniones, lo orientemos a las personas y sean adaptables a cualquier situación. Busca un balance en no generar documentación al vicio sino aquella que aporte valor al proyecto, que busquemos hacer las cosas solo si nos conviene y no por “tradición” o repetición. Esto hace que los proyectos ágiles no hagan las mismas cosas sino que se adapten de acuerdo a lo que haga falta.

Manifiesto Ágil

Lo que escribieron nuestros amigos en el congreso en Utah provocó una especie de culto en todas las cornudas que están en el análisis (No únicamente de la cátedra) ya que se escudan en estas pelotudeces para tomar las decisiones menos ágiles posibles. Saliendo de la bardeada, los gurús del software establecieron una serie de valores y principios que paso a enunciar:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

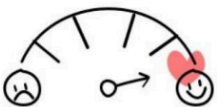
Valores Ágiles:

Plantea que ciertas cosas son más importantes que otras, si no recuerdan los cocodrilos (“>”) apunta a las cosas de mayor valor.

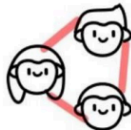
- **Individuos e Interacciones** >> los procesos y herramientas
 - Es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, comunicación fluida, pero corto de herramientas, a tener un equipo individuos alejados, desmotivados con todas la herramientas.
- **Software Funcionando** >> Documentación Extensiva
 - No es que no documentamos manga de gorriados sino que es preferible enfocar los esfuerzos en completar un entregable o hacer que funcione lo que desarrollamos (para que el cliente tenga algo que ver). La documentación debe existir pero no es fundamental a la hora de entregar.
- **Colaborar con el Cliente** >> Negociación Contractual
 - Una pelotudez, donde primamos llevarnos bien con el cliente que sacarle la plata. Aca hay un equilibrio complicado donde dependerá del cliente y de las exigencias. Ya que si nos provoca demasiado retrabajo por cosas que pautamos de manera concreta podemos reducir su “incertidumbre” diciéndole que le vamos a cobrar por realizar el cambio.
- **Responder al Cambio** >> Seguir un plan
 - Básicamente que no seas ese jefe pelotudo que si no está escrito en el plan no se puede hacer. La flexibilidad con las tareas y horarios provee la mayor comodidad para todos.

12 Principios Ágiles

- 1 Nuestra mayor prioridad es **satisfacer al cliente.**



- 4 Los responsables de negocios, diseñadores y desarrolladores deben **trabajar juntos** día a día durante el proyecto.



- 7 El **software funcionando** es la principal **medida de éxito.**



- 10 La simplicidad es esencial.



- 2 **Aceptar** que los requisitos **cambien.**



- 5 Desarrollamos proyectos en torno a **individuos motivados.**



- 8 Los procesos ágiles promueven el **desarrollo sostenible.**



- 11 Las mejores arquitecturas, requisitos, y diseños emergen de **equipos auto-organizados.**



- 3 **Entregar** software funcional **frecuentemente.**



- 6 El método más eficiente de comunicar información es **conversaciones cara a cara.**



- 9 La **atención continua** a la **excelencia técnica** y al **buen diseño** mejor la Agilidad.



- 12 A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto **ajustan su comportamiento.**





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

En sí mismo me parecen que están buenos estos principios porque no hablan solo del desarrollo de software sino que es una forma de negocios muy interesante. Plantea un “*el cliente tiene la razón*” entendiendo que en nuestro rubro es más importante que en todos los demás. No hay cosa más horrible que un software desarrollado que no se usa, representa una pérdida de recursos por la negligencia de nuestro líder de proyecto.

Filosofía Ágil by JC

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan cuando se varía mucho en requerimientos, involucrando siempre al cliente para conseguir una retroalimentación más rápida, esto se debe a que es imposible predecir cómo un sistema afectará sus prácticas operacionales.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla en una sola unidad sino que mediante incrementos, esto hace frente al PUD o RUP, que si bien utiliza procesos iterativos-incrementales, son procesos más definidos.

Los incrementos en cortos períodos se ponen a disposición del cliente, esto no solo nos dice que es parte del proceso, sino que es capaz de entender cómo se direccionan las decisiones dentro del desarrollo, además reduce la cantidad de documentación redundante o innecesaria, la que le baja al cliente pasa a ser una comunicación mucho más periódica.

Requerimientos Ágiles

Introducción

Los requerimientos es realmente el mejunje de la cuestión, es la traducción de las necesidades de nuestro cliente para lograr un software cohesivo que cumpla con las expectativas. Puede darse el peor de los casos, que lo que construyamos no sea lo que el usuario quería o necesitaba para que esto suceda. Retomando con la agilidad tenemos que lo mejor siempre es hablar cara a cara con el cliente siendo la comunicación el éxito o fracaso de un proyecto/producto. Tenemos situaciones donde solo una de las partes sabe comunicarse y lleva a diferentes desperfectos en la planificación como también pérdida en el valor agregado que ofrece nuestro software.

Hablando de clientes y valor, se plantea en el principio 3 que hagamos entregas parciales, esto por varios motivos:

1. Mostramos compromiso con el cliente
2. Podemos ajustar el rumbo
3. Ganamos experiencia de manera modular
4. Sumamos valor agregado en cada entrega
5. El cliente puede ir operando modularmente

Dentro de los requerimientos tenemos 3 pilares mas (Que hartante con los pilares):



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

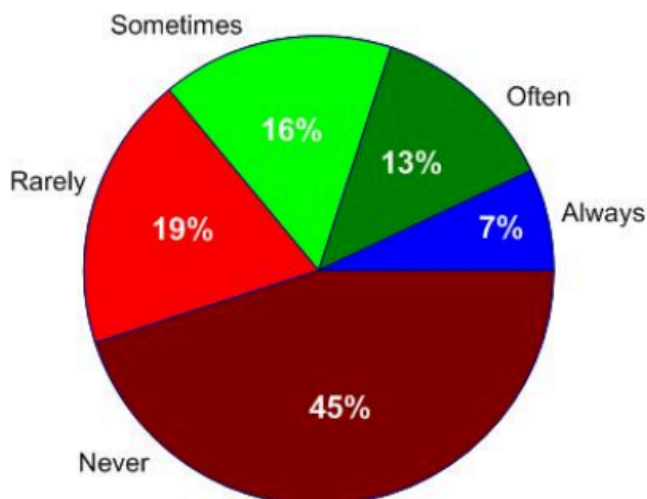
1. **Usar el valor para construir el producto correcto**, esto nos quiere decir que lo único importante es dejarle un software funcionando al cliente, que cumpla con sus expectativas y lo ayude en su objetivo organizacional.
2. **Determinar que es “solo lo suficiente”**, ósea que los requerimientos se van encontrando a de a poco, se inicia con los mínimos indispensables para iniciar un proyecto de software, luego se aprovecha el descubrimiento de los otros para aplicar el empirismo y la mejora continua.
3. **Usar historias y modelos para mostrar que construimos**, esto atiende al principio de técnicos y no técnicos trabajando a la par, el desarrollo de los requerimientos armando con alguien que tenga pleno conocimiento del negocio, del producto y trabaja junto al equipo para reflejar de la mejor manera posible.

BRUF

(Big Requirements Up front o Los requerimientos más grande primeros)

Es básicamente una falencia que tenía el modelo tradicional (Definido) donde ponían las cosas más pesadas primero consumiendo todos los recursos. Esto hace que muchas veces tengamos una visión sesgada de las necesidades reales del cliente. Es como que pones todo primero e intentas ser lo más exhaustivo en ese contexto. Esta metodología es poco adaptable a los cambios y responde muy mal a la implementación ya que gastamos una mayoría de recursos en algo que va a tener un alto riesgo.

Porcentaje de funcionalidad entrega que fue utilizada cuando se lo tomó de manera seria y fue utilizado de manera “correcta” en nuestro proyecto.



Just In Time

En contraposición al BRUF el “justo a tiempo” (JIT) es buscar detallar y documentar los requerimientos que vamos a necesitar en el momento minimizando los recursos a utilizar. Esto hace que podamos utilizar menos cantidad de gente a medida que se desarrollan las funcionalidades sin



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

saturar con un millón de cosas que capaz no usamos. Responde a lo que vimos en DSI como YAGNI (You Ain't Gonna Need It o “No lo vas a necesitar”) siendo una forma óptima de encarar proyectos.

Si hacemos una analogía con el hardware el BRUF es “centralizar” todo, donde para ser más rápido necesitas más gente en un solo proyecto coordinado de manera burocrática en un solo lugar. Por el otro lado el JIT tiende a la distribución donde pequeños grupos toman un parte del todo y se buscan autogestionar. Su reflejo en servidores podría ser visto así, donde podemos juntar más potencia en un “MainFrame” con más controles o distribuirlo en nodos con pequeñas funcionalidades cada uno.

Tipos de Requerimientos:

De Negocio:

Reducir un X% los tiempos de atención al cliente

De Usuario:

Realizar consultas en línea del estado de cuenta de los clientes

Funcional:

Generar reportes de saldos y recibir notificaciones por email

No funcional:

Generar documentación en algún formato. Cumplir con Estándares (Ej: ISO)

Implementación:

Necesitamos un servidor en la nube para desplegar el aplicativo.

Dominio del problema: Abarca todo lo relacionado con el contexto en el cual surge la necesidad del software. Aquí es donde se definen los requerimientos en términos de lo que el negocio y los usuarios necesitan.

- Req. de negocio
- Req. de usuario

Dominio de la solución: Abarca las decisiones sobre cómo se implementará el software para cumplir con los requerimientos definidos en el dominio del problema.

- Req. del software

Gestión de Requerimientos en Agile

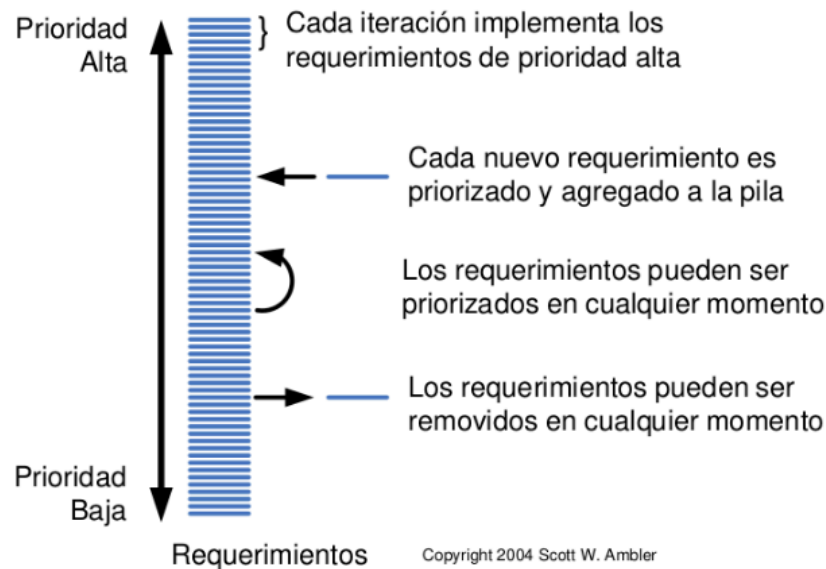
Introducción

La gestión Agile es diferente a la tradicional ya que tiene otros pasos, define otro orden de prioridades y también tiene otros actores. El que da prioridad aca no es el líder de proyecto sino que es el cliente que toma el rol de Product Owner (idealmente hablando). Es nuestro **PO** el que define el conjunto de requerimientos cambiantes que se tomarán a lo largo del proyecto, el mismo le asigna el orden y puede cambiar de opinión según que quiera en ese entregable (Principio 1 y 3).

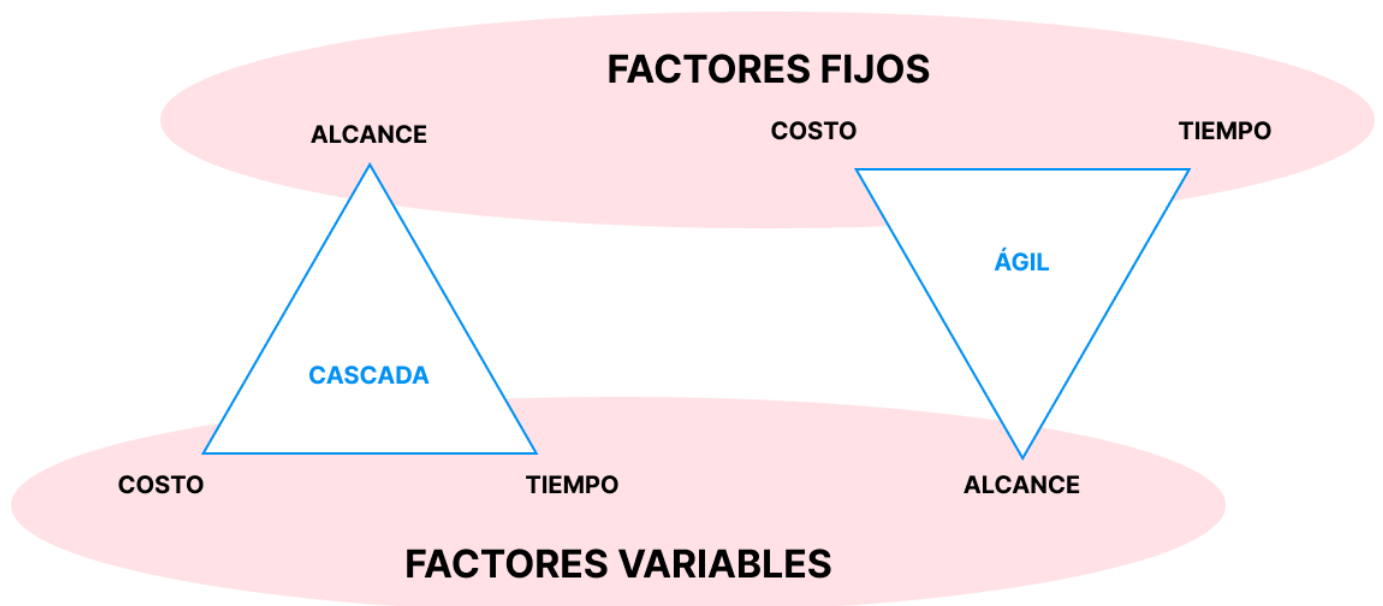


INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Otra cosa que debemos tener en cuenta es que en estos frameworks ágiles se da vuelta el triangulito de Alcance-Costo-Tiempo de la siguiente forma:



Como vemos “cascada” se refiere a los procesos tradicionales y en los frameworks ágiles esto se invierte. Lo que pasa es que buscamos tener un equipo fijo, que en el mismo pedacito de tiempo resuelva un alcance cambiante. Sucede esto porque los entornos ágiles van recolectando de a poco y a medida que lo necesite el cliente el trazado “fino” para que lo desarrollemos en un entregable. En la primera imagen vemos que existe la alta prioridad y la baja donde nuestro PO puede en cada iteración cambiar el orden o agregar alguno que se olvidó. Es inevitable que todo vaya cambiando a medida que pasa el tiempo y en el transcurso del proyecto.

La relacion que tiene con los principios agiles es:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Entregables iterativos que satisfacen al cliente (Principio 1 y 3)
- Recibir cambios en los Requerimientos en etapas avanzadas (Principio 2)
- Técnicos (Equipo) y no Técnicos (Cliente) trabajan juntos (Principio 4)
- Nos comunicamos cara a cara con nuestro PO y equipo (Principio 6)
- Los equipos son autoorganizados para resolver problemas mas pequeños (Principio 11)

Dejando en pocas palabras, nos tenemos que adaptar al cambio constante de nuestro requerimientos y las situaciones cambian sistemáticamente.

*“Cambia lo superficial
Cambia también lo profundo
Cambia el modo de pensar
Cambia todo en este mundo...”*

- Mercedes Sosa -

User Stories

Las **US** son una técnica fundamental de los ambientes ágiles que contienen una descripción corta de una funcionalidad de nuestro futuro producto y que está valorizada por nuestro usuario. Es esencial para crearlas que tengamos una conversación cara a cara y este documento sea un testamento de dicho intercambio. Idealmente las escribe nuestro PO (no pasa) y tiene un trazado más “grueso”, es decir no entra en los detalles completos de la funcionalidad.

Cumple con lo siguiente:

- Representar la necesidad del usuario.
- Conformar una descripción del producto.
- Ítem de planificación.
- Recordatorio para conversaciones a futuro del producto a construir.
- En conjunto al código, permite documentar el producto.

La documentación tradicional no da buenos resultados, sobre todo porque toman todas las decisiones al principio del proyecto, haciendo que los cambios sean difíciles, en cambio las decisiones se trasladan a lo largo del proyectos en las ágiles, **bajando la incertidumbre** y ganando información para lograr un mayor valor agregado.

En definitiva cada User representa un pedacito de nuestro software, una funcionalidad que queremos cumplir, para arrancar a realizarla debe estar completada y validada (sino la escribió el PO). Ahora cuando veamos las partes se va a entender mejor, a continuación veremos más tipos de User y



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

por lo que a mi me queda constancia es la forma en que tomamos en la iteración los “requerimientos ágiles” que queremos resolver.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita.
- Son comprensibles tanto para el cliente/usuario como para el equipo de desarrollo
- Tiene el tamaño justo para permitir planificar.
- Permite el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre US.
- No son apropiadas para equipos demasiado grandes (se vuelve difícil el tema de la conversación verbal).

Partes de una US

Tarjeta: es una descripción, utilizada para planificar y recordar. Tiene una sintaxis que debemos respetar en general para armar una “Frase Verbal” que no es obligatoria pero facilita bastante entender qué hacemos.

“Como [nombre del **ROL en el sistema**] yo puedo/quiero [acción concreta] de forma tal que [valor para el negocio].

En partes:

1. **Nombre del rol:** Aclaró “ROL” porque es el tipo de usuario en el software, puede o no ser semejante a la realidad del proceso. Por ejemplo, el rol “administrador” no solo puede usarse para los desarrolladores sino los altos cargos que organizan todo. Mientras que un rol como “cajero” aplica a la realidad y al software.
2. **Actividad:** Es la acción a realizar con el programa, por ejemplo, “registrar un usuario”.
3. **Valor del negocio:** Es lo que obtenemos al hacer esta acción. Cosas como “Para dar de alta una persona en el sistema”, “Transferir X cosa a alguien”, etc.

Criterios de aceptación: Diferentes criterios que permiten implementar, definen límites, dando una noción al equipo de estos factores lo que facilita el desarrollo y testeo.

Ejemplos:

- “En el campo de Tarjetas solo debe contener números”
- “El formato de las fechas tiene que ser dd/MM/yyyy”



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- “La fecha inicial debe ser menor a la final”

Pruebas de Aceptación: Lo que vimos iba en el frente mientras que esto va en la parte de atrás. Es la captura de los detalles manifestados en la conversación y determina la complejidad de la US. Además es muy útil para saber si el software esta andando correctamente.

1. **Conversación:** es la parte más importante de la US, ya que explicita la comunicación entre PO y el equipo de desarrollo, permite compensar la falta de detalles. El acuerdo por sobre la negociación del contrato y técnicos y no técnicos trabajando juntos. En resumen son lo que da detalles a la US.
2. **Confirmación:** Definición entre el PO y el equipo para demostrar características del producto que realmente se implementó, conforma la definición de las pruebas de aceptación. El PO decide si las pruebas son válidas, son las determinan la finalización de US.

Story Points

El Story Point es una unidad de medida con la cual medimos el tamaño de nuestra User Story. En las metodologías ágiles se intenta que ninguna historia sea más grande que la iteración que estamos utilizando. Es decir si usamos 2 semanas como periodo de tiempo no debemos tener US que cuesten más que eso.

Se mide con los siguientes parámetros:

- **Complejidad:** dificultad inherente a la funcionalidad, no permite dividir más la US.
- **Esfuerzo:** Es el tiempo en horas de trabajo que requiere implementación.
- **Incertidumbre:**
 - **Técnica:** No sabemos como resolver algo de manera programable o implementable, es una incertidumbre del equipo.
 - **De Negocio:** Desconocemos cómo funciona algo en el negocio, como una transacción, operación o regla de negocio.

Tipos de Historias

- **Epica:** Es una user grande como feminista, que en general tiende a ser desglosada en tareas mas pequeñas y manejables por Sprint. Cuando completamos todo lo que está adentro completariamos la Épica
- **Temas:** Parecido a lo anterior, son historias que están ligadas conceptualmente por algún aspecto y se agrupan como una sola entidad para facilitar la planificación.
- **Spikes (Investigación):** Son un tipo especial que se crean a partir de la incertidumbre, por esto mismo no tienen estimación y no cumple con la definición de listo que veremos más adelante. Una vez se resuelve la incertidumbre la Spike se convierte en una o más US con el tiempo estimado. Se clasifican en:
 - **Técnicas:** Debemos resolver como lo hacemos en el software, evaluamos la performance o cierta tecnología.
 - **Funcionales:** Nos quedan dudas de cómo los usuarios van a interactuar con nuestro software. Se asocian a los prototipos.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Lineamientos para los spikes:

- Estimables, demostrables y aceptables
- La excepción, no la regla
- Toda historia tiene incertidumbre y riesgos
- El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración
- Los spikes deben dejarse para incógnitas más críticas y grandes
- Utilizar spikes como ultima opcion
- Implementar la spike en una iteración separada de las historias resultantes
- Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

Modelo INVEST

El modelo INVEST significa:

Independiente
Negociable
Valuable
Estimable
Small
Testeable

No voy a explicar letra por letra porque creo que somos grandes y sabemos leer.

Ahora bien las User que responden a este modelo son más modulares y fáciles de manejar por la misma naturaleza.

Ejemplo:

“Como comprador agropecuario quiero poder cargar todas las verduras en un carrito, luego hacer una cotización para mandarla al exterior y comprar semillas por separado así me siento mejor conmigo mismo.”

Aunque una pelotudez lo que escribi es para mostrarles cuando no se cumple ciertas cosas:

1. Primero **no es independiente** porque hacemos todo lo de las verduras y además agrego la compra de semillas. Generalmente si ponemos “y” ya esta mal la User.
2. Segundo (Francia) **no es valuable ni estimable** ya que la respuesta lo mido en una especie de grado de felicidad conmigo mismo que no se entiende.
3. En base a lo anterior tampoco es testeable ya que capaz no todos nos sentimos mejor después de testearlo.

Otro ejemplo:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

“Como comprador agropecuario quiero cargar las verduras en un carrito para registrar una cotización en el sistema.”

Esto es una versión menos pava de la anterior que sí cumpliría con lo anteriormente dado.

Modelado de Roles

Es otro documento o tarjeta de rol de usuario donde dejamos escrito las características de aquellos que utilizan la aplicación.

Tenemos:

- **Rol Usuario:**
 - **Ejemplo:** Un **reclutador interno**, quien lo utiliza no es un experto en el software pero ha utilizado otras aplicaciones de este estilo. Cuando utilicemos nuestro software cargará los datos de manera masiva buscando la fiabilidad de la carga.
- **Personas:**
 - **Ejemplo:** Jean Paul Sartre es un reclutador de la empresa que está hace años. Posee un horario flexible y busca cargar la mayor cantidad de datos de la manera más rápida posible.
- **Personajes:**
 - **Ejemplo:** La aplicación apunta a ser utilizadas por personas mayores o empleados de la tercera edad de la compañía.
- **Representantes o Proxies:**
 - En este hay un abanico de categorías, este tipo de usuarios debería ser evitado por una cuestión de que no diseñamos para quien le usa el teléfono a su abuela para hacer un trámite.

Product BackLog

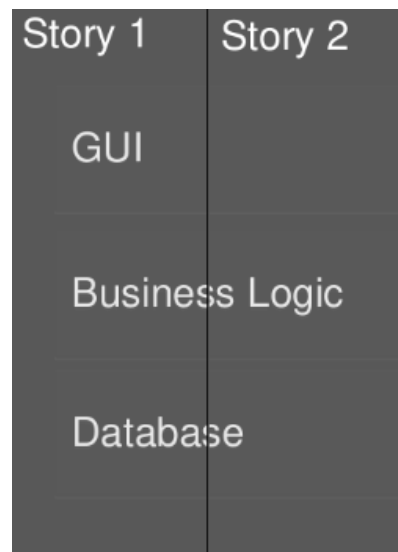
Es el conjunto de actividades, tareas que están en desarrollo y son ordenadas por el PO de manera tal que las más importantes están arriba. En las metodologías ágiles esta “pila” está confirmada por las User Stories que el usuario está creando. Algunas veces se pueden dejar US como “vacías” y a medida que nos acercamos las iremos detallando más. Esto se hace para no completar el 100 de las User's al principio, porque sería como el enfoque tradicional, sino que antes de ser tomadas en una iteración (Sprint en SCRUM) deben estar completas.

En clases los docentes ponen a las historias como partecitas “verticales” de diferentes capas que son horizontales y representan conceptos arquitectónicos



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Como vemos con cada User completamos la capa visual, la lógica de negocio y la base de datos.

Definition of Ready (DoR)

Es una “medida” de calidad que construimos en equipo para poder determinar qué Historias de usuarios están listas en condiciones para entrar en una iteración de desarrollo. Generalmente se utiliza de parámetro el modelo INVEST. En SCRUM vemos una plantilla de lo que podría ser:

- Valor de negocio claramente expresado
- Detalles suficientes comprendidos por el equipo para que se pueda tomar una decisión
- Dependencias identificadas o que no dependa de factores externos
- Fue debidamente estimado y puede ser completado en el Sprint (SCRUM)
- Los criterios son claros y testables
- De haber performance es claro y testeable
- Entendemos como lo podemos mostrar al PO.

Definition of Done (DoD)

Es básicamente las normas que ponemos como equipo para que una Historia de Usuario desarrollada sea ACEPTADA y dada como terminada. Esto hace que se saque del product backlog y sea considerada como terminada. En líneas generales se suele estipular que siga buenas prácticas de programación (Refactorizado, comentado, inspeccionado, etc), documentación actualizada, que fue probado por al menos 2 personas, que se integró con el Main y se encuentra en servidores productivos.

Estimaciones

“Estimar es el proceso de obtener una aproximación sobre una medida con objeto de predecir la completitud y gestionar los riesgos en el contexto, se relacionan con objetivos del negocio, compromisos y control.”



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Errores comunes en las estimaciones

Estimamos la necesidad de predecir la completitud y administrar los potenciales riesgos que surgen con la planificación. La regla es simple, entre más información y experiencia tenemos mejores van a ser nuestras predicciones.

1. Información imprecisa acerca del software.
2. Demasiado caos en el proyecto
3. Imprecisión generada por el proceso de estimación
4. Omitir actividades necesarias para la estimación.
5. Plantear no es estimar.
6. A mayor diferencia entre lo planeado y lo estimado, se traduce en mayor riesgo.
7. Las estimaciones no son compromisos.

Inevitablemente en las primeras iteraciones de un proyecto nos encontramos con errores ya que los equipos se forman por proyecto y solemos ser optimistas (que no hay que serlo)

Proceso de estimación

Tenemos las estimaciones tradicionales que ya la vimos en la unidad 1, por otro lado en las estimaciones ágiles tenemos varios métodos para utilizar alguno de estos más utilizados que otros pero el agile propone utilizar diferentes métodos y luego comparar, ya que todos tienen su factor de ajuste y calibrar la estimación dada, los métodos son:

1. Basados en la experiencia

- a. **Datos históricos:** consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones, con esto se busca una alternativa en la cual el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona. Esto choca con el hecho de que la experiencia no se extrapola entre equipos planteada por la gestión ágil.
 - i. Industry Data
 - ii. Historical Data
 - iii. Project Data

2. Juicio experto: Uno de los métodos más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Hay 2 técnicas para aplicarlo:

- a. **Juicio experto puro:** un experto estudia las especificaciones y realiza estimaciones, se basa en la experiencia del experto.
- b. **Juicio Wideband Delphi:** es similar al anterior pero grupal, un grupo de personas se reúne con la idea de debatir para generar una estimación en común, tanto en esfuerzo y duración. Hay un coordinador que reúne las estimaciones individuales, se opinan sobre las estimaciones ajenas, se vuelve a discutir, y se repiten hasta llegar a un acuerdo razonable, las bases del **Poker Planning**.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

3. **Analogía:** Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares. Este método suele tener mucho error dado que los conocimientos de un proyecto no son exportables.
4. **Otros:**
 - a. Basados exclusivamente en los recursos: consiste en analizar el personal que se tiene y de cuánto tiempo disponen, haciendo sólo estimaciones en función al tiempo.
 - b. Basados en el componente del producto:
 - i. **Bottom up:** se descompone el producto en unidades lo más pequeñas posibles. Se estima cada unidad y luego se hace mediante la estimación total
 - ii. **Top down:** se descompone el producto en grandes bloques y luego se estima cada uno de esos componentes.
5. **Métodos algorítmicos:** se ejecutan algoritmos que tienen como parámetros de entradas a medidas cuantitativas de tamaño, complejidad y conocimiento de dominio. Cada variable tendrá asociada un factor multiplicativo, no es certero por más matemático que sea.

Tamaño vs. Esfuerzo

Este tema lo traigo porque realmente el último renglón merece un libro, tenemos una seria tendencia de querer hacer que el tamaño es tiempo, y decir que una user de 8 StoryPoints nos va a costar 8 hs reloj. La complejidad que asignamos con tamaño y eso si se traduce con SP, después quien estime proyectará su esfuerzo viendo la cantidad total de actividades.

Velocidad

Es una **MÉTRICA** del progreso del trabajo de un equipo, el mismo se **CALCULA**, no se estima. El cálculo es mediante la suma de SP de las User completados en cada iteración. Testeado por el DoD y la aceptación del PO.

Es una métrica que se calcula de forma estrictamente **BINARIA**, ya que no se consideran historias a medias, siempre son aceptadas las que se contabilizan y entregadas.

Esta métrica permite retroalimentar el proceso de estimación ya que un equipo maduro y con mucho trayecto permite estimar cuántos puntos se pueden tomar en cada iteración.

Re estimación

Es aconsejable la reestimación de una US solo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que el proceso de estimación se han detectado otras US con tamaños más grandes o pequeños.

No debería hacer estimaciones en caso de que no se alcance de manera esperada (velocidad del equipo), ya que es precisamente en esta velocidad que va aportar el dato para ajustar.

Planificación de release



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Este proceso se da al inicio del proyecto, donde se analiza y determina que US, épicas o temas serán desarrollados en una nueva entrega del producto. Este plan se va actualizando mientras el proyecto transcurre de manera siempre refleja las expectativas actuales que se van a incluir en la entrega.

US priorizadas → Estimación de tamaño en SP → Derivar duración → Release Plan

Planificación de iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada. Esto se hace teniendo en cuenta el tiempo y la velocidad de trabajo del equipo.

Durante esta planificación se discute sobre las tareas que serán necesarias para transformar un US en código funcional y testeado.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí solo se discuten las tareas que se realizan durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

No Estimate:

Destacamos que las estimaciones ágiles se diferencian principalmente en que quien estima no es el PM, líder de proyecto, tu vieja o el (ex)marido de Pampita... NO, la estimación lo hace quien tiene asignada la tarea, siendo mucho más coherente pero más riesgoso para equipos no comprometidos (que la agilidad asume lo contrario).

Tips respecto de las estimaciones:

- Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización
- Lo más beneficioso en las estimaciones es el proceso de hacerlas
- La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no
- La estimación puede servir como una gran protección para el equipo



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Estimaciones Ágiles:

Las estimaciones ágiles, a diferencia de las estimaciones absolutas de la gestión tradicional, tienden a incorporar un método de estimación relativa basado en obtener la estimación a partir de la comparación.

Esto se debe a que:

- Las personas no saben estimar en términos absolutos
- Somos buenos comparando cosas
- ¿Cuántas veces más complejo es esto?
- ¿Cuántas veces más difícil es esto?
- Comparar es generalmente más rápido
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
- Se emplea mejor el tiempo de análisis que las stories

Con respecto al tamaño...

- “La palabra tamaño refiere a cuán grande o pequeño es algo”
- El tamaño es una medida de la cantidad de trabajo necesaria para producir una feature/story

El tamaño indica:

- Cuán compleja es una feature/story
- Cuan trabajo es requerido para hacer o completar una feature/story
- Y cuán grande es una feature/story

Tamaño vs Esfuerzo:

Las estimaciones basadas en tiempo son más propensas a errores debido a varias razones:

- Habilidades
- Conocimiento
- Asunciones
- Experiencia
- Familiaridad con los dominios de aplicación/negocio

El tamaño no es esfuerzo, hay cosas que involucran un gran alcance como sería cargar un montón de cosas en la base de datos. En teoría es fácil y no es super exigente sin embargo la carga va a requerir más tiempo. Por otro lado, podemos tener la misma estimación para una persona que realiza un ABM de un producto. El tamaño lo asociemos al alcance mientras que el esfuerzo a la complejidad.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Poker Planning / Poker Estimation

Es una técnica de estimación en ambientes ágiles, se realiza mediante el equipo estimando su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia el conocimiento de cada uno con la posibilidad de comprar con el resto del grupo. Recordemos que el PO está en la planificación pero no estima. Se hace una escala precisamente con cartas de póker, se toma la lista de actividades, y cada miembro propone una carta por actividad, teniendo en cuenta los 3 aspectos de las US. Luego procedemos al juicio experto definiendo mediante una exposición su elección, luego el resto del equipo da un feedback de lo expuesto, en caso que haya diferencia considerables se vuelve a hacer un nuevo ciclo de estimación y debate.

Pasos (By Carlitos):

1. Determinar la base story (la canónica) que será utilizada para comparar con las otras historias. Se usa como regla para medir el resto de las historias.
 - a. Suelen ser registros de entidades de negocio muy pequeñas
 - b. Nunca elegir transacciones como canónicas
2. La story a ser estimada se lee a todo el equipo
 - a. Los estimadores discuten la story, haciendo preguntas al product owner cuando lo necesiten
 - b. Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
 - c. Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
3. **Si todos los estimadores** (No pasa nunca) seleccionan el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos.
4. Se toma la próxima story, se discute con el producto owner
 - a. Cada estimador asigna a la story un valor por comparación contra la base story ¿Cuan grande/pequeña, compleja, riesgosa es está story comparada con la canónica?

En caso que las estimaciones no converjan, se hace un promedio o intermedio entre los extremos.

- Consideraciones
 - Criterio Done para ver que estén efectivamente terminas
 - El tiempo para realizar estimaciones es time boxing
 - La US por estimar tiene que ser las menos posibles
- Ventajas
 - Permite opiniones multidisciplinarias de todo el equipo, enriquece la estimación.
 - Al generar debates y discusiones, permite mayor comprensión del dominio para todos los miembros del equipo.
 - Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados que hacerlo de forma grupal.
 - Es una actividad que resulta divertida para el equipo.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Fibo y sus amigos

Generalmente para esto usamos una escala de Story Points siguiendo la serie de Fibonacci:

[0,1,2,3,5,8,13,21,34,55,89]

Que significan para referencia en una iteración:

- **0:** Deja la facu, que chota estás haciendo
- **1 - 2:** Funcionalidad pequeña (usualmente cosmética) o canónica
- **3:** Funcionalidad pequeña a mediana. Son partes más manejables de algo
- **5:** Funcionalidad media que nos interesa que se maneje así. Generalmente tener asignado 2 de estas es porque haces mucho.
- **8:** Funcionalidad grande, se suele asignar como mucho esta tarea a alguien y es porque es imposible su descomposición.
- (Límite Sano)
- -----
- (Enfermedad Mental)
- **13:** ¿Alguien puede explicar por qué no lo podemos dividir?
- **21:** Si no es una Épica probablemente estemos rozando algún tipo de enfermedad mental. Se divide porque no se puede sino.
- **34:** Nunca vi algo así, seguro que sos líder de proyecto?
- **55:** Ese título me parece que es de la Blas Pascal, con razón no entendes nada.
- **89:** No me pagan lo suficiente para aguantar esto.

Velocidad

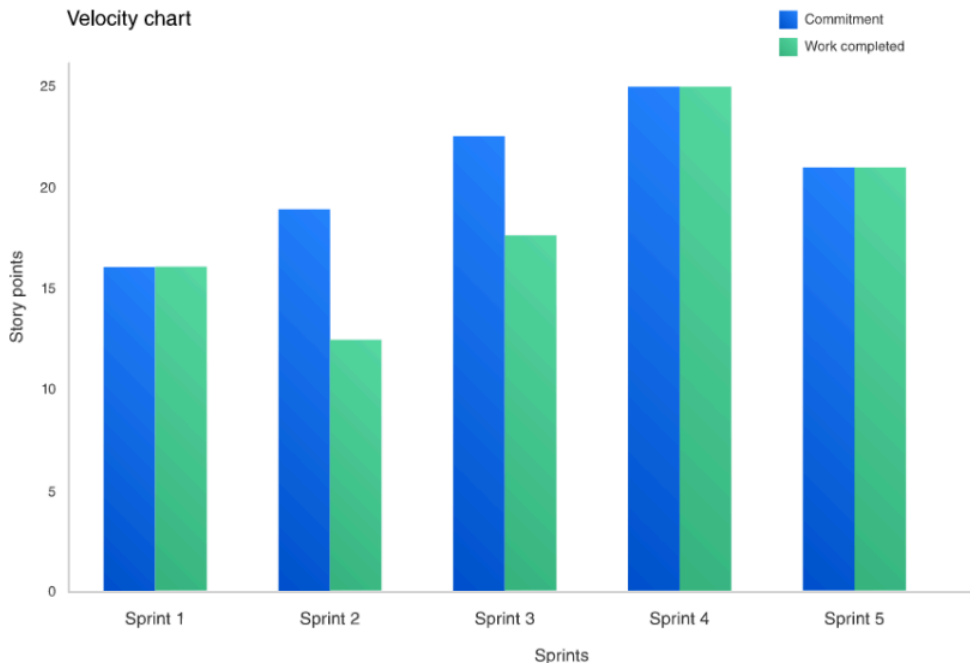
Es una de las métricas más importantes de las estimaciones ágiles y tiene que ver con cuántas Historias de Usuario pasamos a Definition of Done (DoD). Recordemos que esto significa que el PO vio lo que hicimos, le gustó y lo salió a publicar en redes. Para calcularlo debemos únicamente sumar la cantidad de Story Points de todas nuestras Historias completadas al final de una Iteración (Comúnmente Sprint).

Esta métrica está buena porque vos capaz hiciste un montón de historias pero ninguna al 100% de lo que se esperaba. Esto si bien no está mal, no se pueden descartar las User porque en algún momento lo debemos ver antes de entregarlo en producción. Por otro lado tomaste 2 historias chiquitas pero las completas en su totalidad y el PO te felicita porque probablemente tengas más materia gris que tus compañeros.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Con este tipo de gráfico vos le mostrás a tu “jefe” que el que mucho abarca poco aprieta.

SCRUM

Introducción a la verdad

Llegamos al famoso, al único, al más admirado, el SCRUM (Aplausos). Porque a las señoras que las gorrio el marido les gusta tanto esto no lo sé, pero como viste en lo que fui escribiendo lo íbamos mencionando. A mi me jode un poco porque desde DSI que se habla y no se porque no es una unidad de esa materia en vez de romper las bolas con los artefactos hediondos del PUD. Ahora, el SCRUM es un marco de trabajo ágil (Les encanta que digas eso) y conlleva un montón de “sugerencias” para que puedas completar tu proyecto.

Aclaró que se puede utilizar para cosas más allá del software pero no para todo, en aquellos contextos que no responda bien la gestión tradicional es una alternativa válida. De igual manera no todo desarrollo de software está obligado a hacerse en SCRUM o algo ágil.

Para poder ser uno con el SCRUM queremos de:

1. **Un propietario del producto (PO)**

- ordena el trabajo de un problema complejo en un Product Backlog. Como vimos lo ideal es que sea alguien que sepa de SCRUM y que además sepa del negocio. No pasa casi nunca, para mí una falencia del framework.

2. Francia

3. **El equipo de SCRUM**



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- a. Convierte una selección del trabajo en un incremento de valor durante un Sprint y que se sepan autogestionar.
4. Reuniones con sus las partes interesadas (stakeholders)
 - a. Inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint. No entiendo muy bien
5. Repetir.

Acá hago un break porque fui a buscar más info a las filminas y me encontré con esta tremendísima obra maestra, el pináculo de la agilidad.

Universidad Tecnológica Nacional
Facultad Regional Córdoba
Cátedra de Ingeniería y Calidad de Software
Docentes: Judith Meles – Laura Covaro



JUDITH MELES

SCRUM 2020

Claramente vemos como Maria Judith en su nivel de abstracción superior respeta todas las pautas que ella da para la presentación de algo, casi no escribe texto relevante, pone su nombre 2 veces y todo lo de arriba a la izquierda en el mismo tamaño. La imagen simboliza la estructura de su cátedra que está hecha por cañas (y se cae a pedazos) con uniones orgánicas por lo que representa que el todo es más que la suma de sus partes. Simplemente arte, cine 🎨.

Sacando el meme continuo

Nosotros y el SCRUM

La gente que lo desarrolló lo dejó “incompleto” para que cada uno con sus amigos rellene los espacios en blanco para formar vínculos duraderos. Además de que busca el minimalismo, no te obliga a escribir 20k de cosas sino que vos adaptas los principios y valores con el grupo para que encuentren algo que a todos les guste. Por ejemplo, se habla de ceremonias como weekly, daily, etc, pero no hace falta que sean como dice el SCRUM 2020 (última versión cuando escribo esto). Sino que uses aquellas ceremonias en el formato que el equipo de trabajo obtenga valor agregado y no sea un

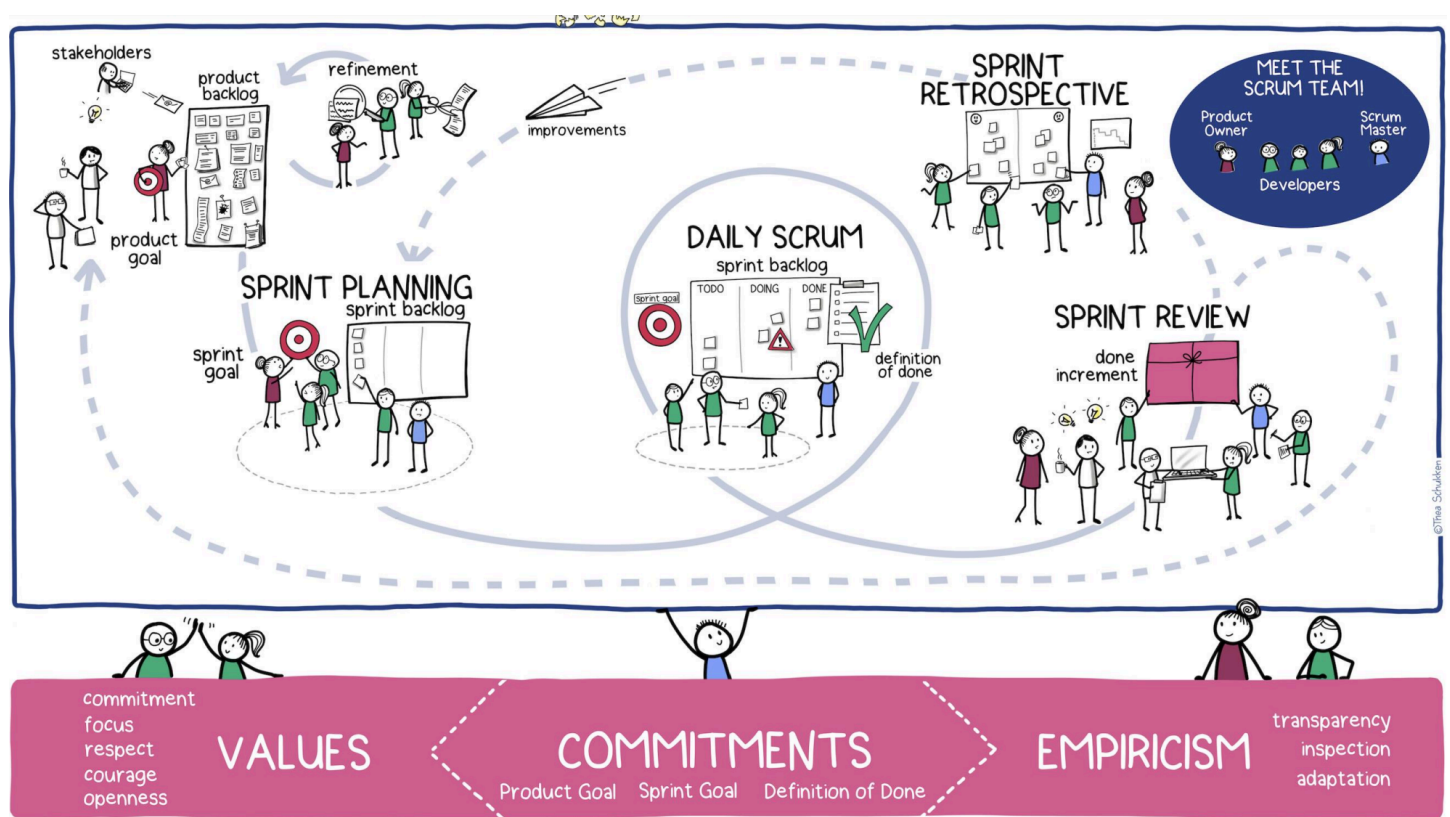


INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

desperdicio de tiempo. En esto todos los PM, líderes, Scrum masters son medios tarados (menos mi jefe JP Alessandrini que es una masa) porque te clavan 10 reuniones por día, que no te aportan nada y terminan retrasando a la gente. Mi consejo para la vida real, evitar darle reuniones a los desarrolladores, para ellos no es esencial ya que no les gusta hablar, prefieren trabajar y en general prefieren que les envíes un mail con lo pautado. Es como si a los obreros de una construcción les cortaras el trabajo cada 15 min para charlar de cómo quieren que siga la obra, es un trabajo para los ingenieros, arquitectos y maestro de obras.

Previamente vimos agilidad ya que es uno de los componentes que caracterizan a nuestro framework, a todo esto le sumamos las “recomendaciones” que hace nuestro software.



Esta es la “hoja de ruta” que tenemos en el Scrum y sus principales ceremonias. El sprint es una “iteración” o crecimiento del producto que tiene un objetivo cumpliendo una serie de pautas y tareas.

- Roles:
 - Scrum Master
 - PO
 - Desarrolladores (Cualquier posición del equipo)
- Reuniones:
 - Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective
 - Refinamiento del PB



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Artefactos
 - Product Backlog
 - Sprint Backlog
 - Versión del Producto
- Tiempos de cada ítem:
 - Sprint (2 semanas a un mes)
 - Sprint Planning (8 horas máximo)
 - Daily Meeting (15 minutos)
 - Sprint Review (4 horas máximo)
 - Sprint Retrospective (3 horas máximo)
 - Refinamiento del **P**roduct **B**acklog (10% del tiempo de un sprint)

Algunas cositas que vimos antes es la Definition of Ready (DoR) y Definition of Done (DoD) entran en el scrum.

Eventos SCRUM

Vamos a hablar de el contenedor de todo como el SPRINT, cada evento dentro del mismo sirve para inspeccionar y adaptar los artefactos de SCRUM. Estos eventos están diseñados para promover la transparencia, les solemos decir ceremonias y le damos cierta periodicidad para crear regularidad y establecer como reuniones necesarias, siendo las que resumen casi todos.

El SPRINT (Toda la iteracion)

La base de todo, es la idea de la iteración definida en un periodo establecido e inamovible de tiempo. Incluye en sí todas las ceremonias necesarias, es importante saber que:

- NO se hacen cambios que pongan en peligros el Objetivo del SPRINT
- La calidad no disminuye
- El trabajo pendiente del producto se refina según sea necesario
- El alcance se puede clarificar y negociar con el propietario del producto a medida y aprender más

El sprint nos ayuda a garantizar la inspección y adaptación del progreso hacia un objetivo del producto, como mínimo, una vez al mes en el calendario. Si alargamos más ese tiempo se puede perder el objetivo o volverse obsoleto, siendo un riesgo en aumento importante. Por eso se busca que sean cortos, aportando aprendizaje cada menos tiempo, esto lo podemos leer con gráficos burn downs y burn ups que exponen el progreso en sí mismo del sprint.

Solo es cancelable el sprint si el objetivo se vuelve obsoleto y el PO es el único que tiene potestad para hacerlo.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Planning (Ceremonia)

Se hace al iniciar cada SPRINT estableciendo que trabajo se va a realizar en el mismo, este plan resultante es creado por el trabajo colaborativo, PO se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importante y cómo se asignan al objetivo del producto.

- Se define pq el sprint va ser valioso
- Que se puede hacer en este sprint
- Cómo se realizará el trabajo elegido

Daily (Ceremonia)

El propósito es inspeccionar el progreso hacia el objetivo del sprint y adaptar el sprint backlog según sea necesario, ajustando el próximo trabajo planeado.

Es un evento de **15 minutos máximo** para todos los desarrolladores del equipo SCRUM, para reducir la complejidad, se lleva a cabo al mismo tiempo y lugar los días laborales del Sprint. Si el SCRUM Master y el PO está directamente implicado en el elemento de trabajo participan como desarrolladores.

Review (Ceremonia)

El propósito es revisar el resultado del sprint y determinar futuras adaptaciones, presentando a los interesados claves. Durante el evento el equipo visualiza los logros del sprint, no se debe limitar en tiempo ni forma para que no sea una simple presentación. Es el penúltimo evento del sprint y se utiliza en un plazo máximo de 4 hs para sprints de un mes, en sprint más cortos suelen ser menores.

Retrospective (Ceremonia)

El propósito de la retro es aumentar la calidad y la eficiencia, el equipo inspecciona como fue el último sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de hecho. Identifica los cambios más útiles, las mejoras se abordan lo antes posible.

Refinamiento (Ceremonia)

No describen mucho esto para el parcial, pero son revisiones del backlog con el fin de desechar aquellas funcionalidad o elementos de trabajos obsoletos, estimar nuevamente otros y ampliarlos o acomodar algunos.

Artefactos de SCRUM

Los artefactos de SCRUM representan trabajo o valor están diseñados para maximizar la transparencia de la información clave.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Para el trabajo pendiente del producto es el objetivo del producto.
- Para el Sprint Backlog es el Sprint Goal.
- Para el incremento es la definición de hecho.

Pila de producto (Product backlog)

El trabajo pendiente es una lista emergente y ordenada de lo que se necesita para mejorar el producto, es la única fuente de trabajo emprendida por el equipo de SCRUM. El compromiso asumido lo conocemos como product goal que describe el estado futuro del producto que pueda servir como objetivo del scrum contra el cual planificar (recordamos que las planificaciones son relativas)

La pila del sprint (Sprint Backlog)

Son el trabajo que se toma de la pila de producto y se asume como **desafío del sprint**, fijando los objetivos del mismo y el plan de como llegar al incremento de valor, lo elaboran los desarrolladores para ellos mismos. Una vez tomado lo debe estimar la persona que fue asignada a esa tarea.

Incremento

Es un paso hacia el objetivo final del producto, son aditivos entre ellos y compatibles, a medida que es mayor el número, mayor valor tiene el producto. Para saber que es un incremento válido usamos el término DOD (Definición de hecho) es una descripción formal del estado del incremento cuando cumple la calidad requeridas para el producto. La definición de hecho crea transparencia al proporcionar cuando se toma como completo un incremento. Si no cumple no debería presentarse al sprint.

TimeBox

Esto existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos.

DoR (definición de listo)

Es un criterio que define cuando una US esta lo suficientemente bien formulada como para **apodarse incluir en un SPRINT**.

DoD (Definición de Terminado)

Es un criterio colectivo que ayuda a determinar si una tarea está completada y se considera correcta su implementación.



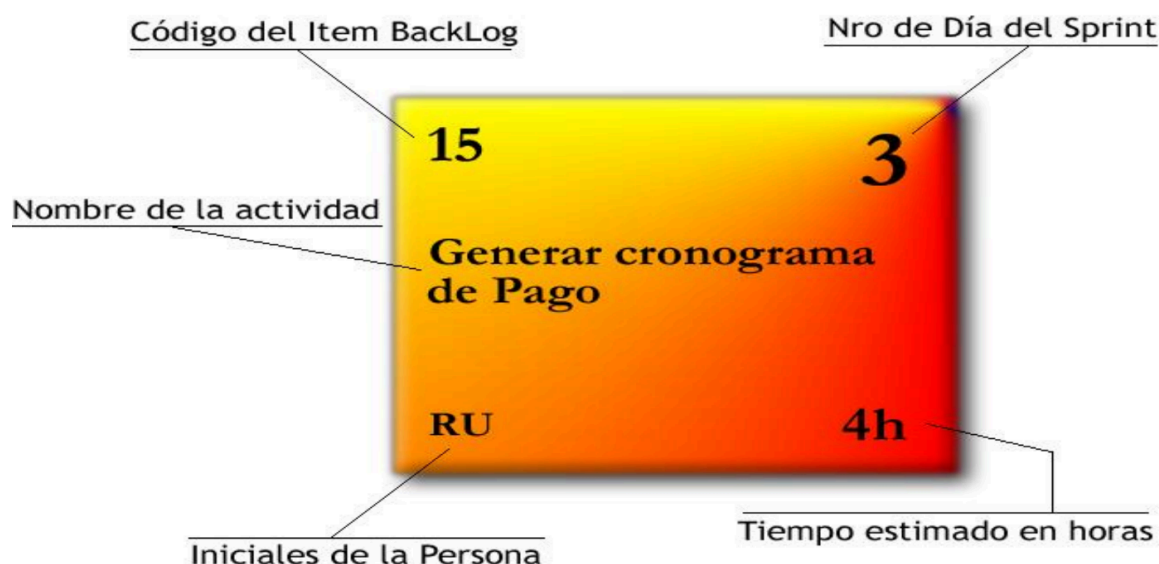
Es la métrica utilizada por el SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para determinado sprint, la capacidad se estima y se mide.

Tarjeta de tarea

Una tarjeta compuesta por 5 elementos → 1. Código de ítem 2. Nombre de la actividad 3. Iniciales del responsable 4. Número de día del sprint 5. Estimación del tiempo para finaliza la tarea.

Se da en 5 secciones, Stories, Para hacer, Trabajo en progreso, Para revisar y listo.

Story	To Do	In Process	To Verify	Done
As a user, I... 8 points	Code the... 9 Code the... 2 Test the... 8	Code the... DC 4 Test the... SC 8	Test the... SC 6	Code the... D Test the... SC 8 Test the... SC Test the... SC Test the... SC 6
As a user, I... 5 points	Code the... 8 Code the... 4	Code the... DC 8		Test the... SC Test the... SC Test the... SC 6





INGENIERIA Y CALIDAD DE SOFTWARE

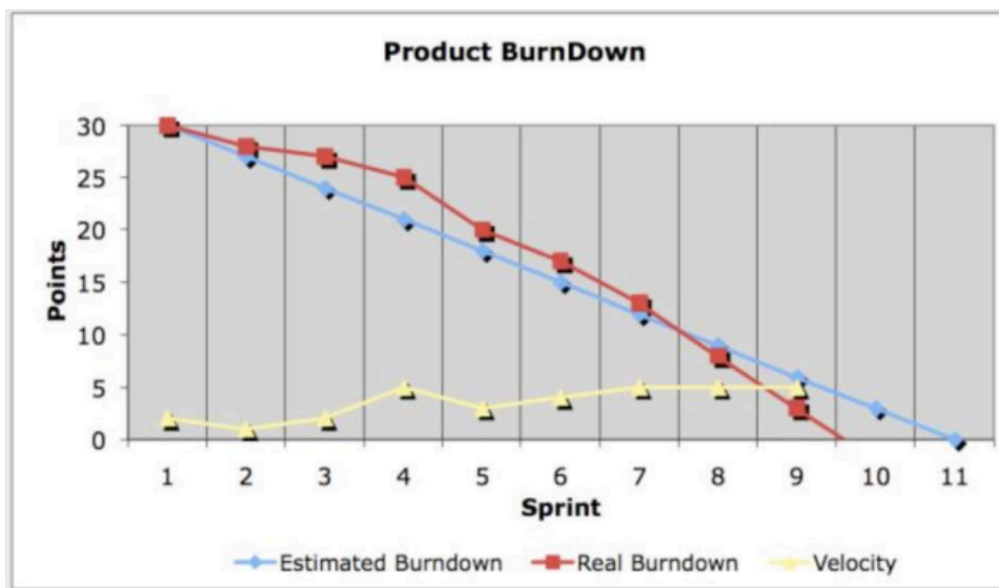
RESUMEN FINAL Y ULTRAVIOLENTO

Gráficos del backlog

Brindan información acerca del progreso de un sprint, de un release o del producto. El backlog de trabajo es la cantidad de trabajo que queda por ser realizado (en días) mientras que la tendencia del backlog compara la cantidad con el tiempo (en horas)

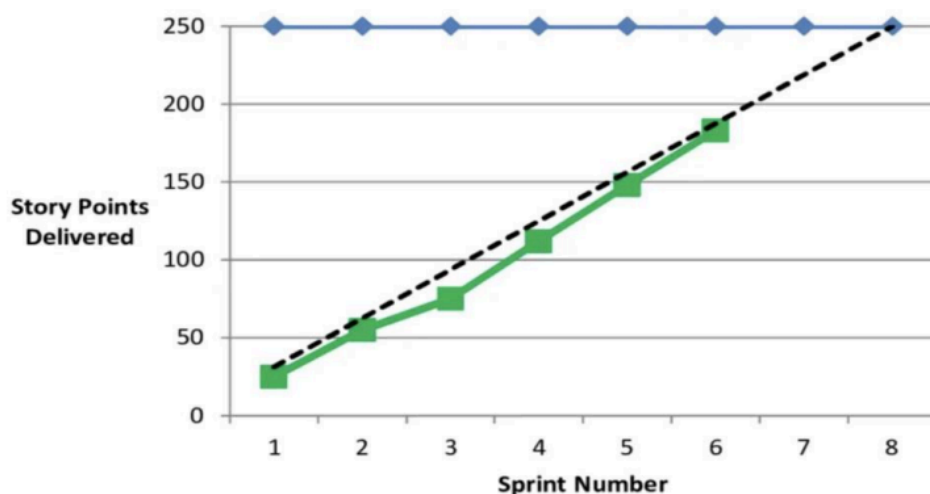
Sprint BurnDown Chart

Visualiza la curva descendente de cuánto trabajo queda para terminar, se hace en días del sprint.



Sprint BurnUp Chart

Visualiza curva ascendente de cuando trabajo se ha completado a lo largo del sprint, ahora los puntos de historia realizados a lo largo de cada sprint



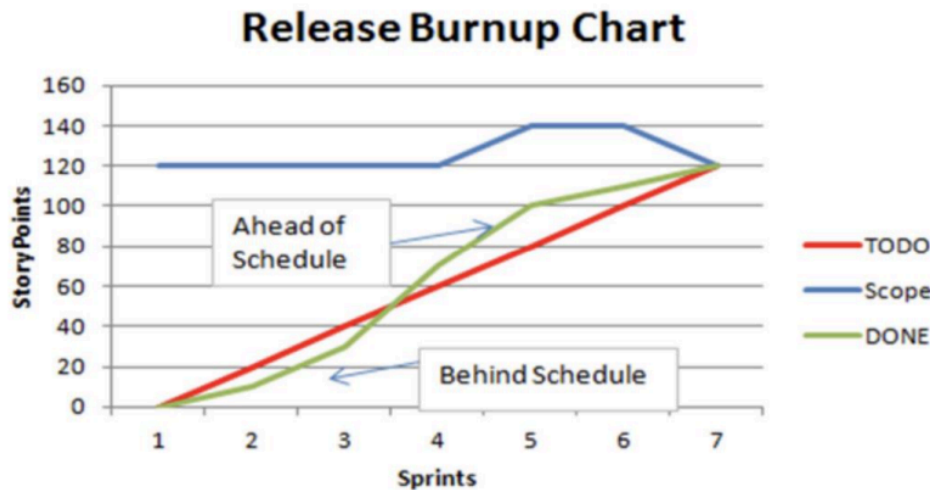


INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Combined Burn Chart

Es la visualización de trabajo por terminar y trabajo ya terminado.



Los tiempos del SCRUM:

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholders	Visión y evolución del producto a través del tiempo	Visión de Producto, Roadmap y características de alto nivel
Release	3 (o menos) a 9 meses	Equipo Scrum, Stakeholders	Balancear continuamente el valor de cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de Release
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum	Que aspectos entregar en el siguiente sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Equipo Scrum (al menos los que trabajan en IPB)	Cómo completar lo comprometido	Inspección del progreso actual y adaptación a la mejor forma de organizar el siguiente día de trabajo

Métricas

Las métricas en el SCRUM solo son 3, es minimalista como debería ser todo en esta vida tan compleja.



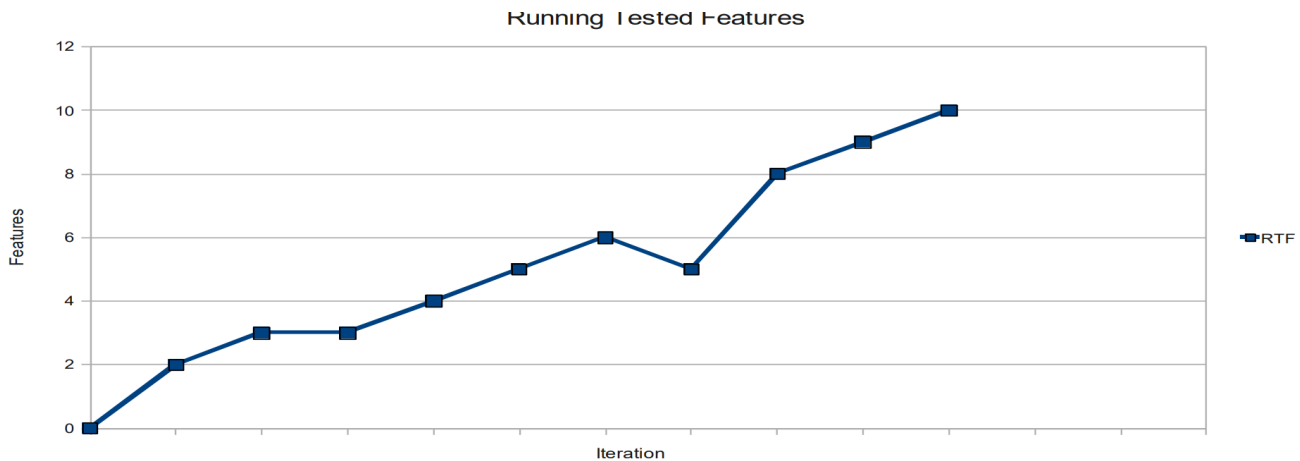
INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

"Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso."

Running Tested Features (Funcionalidades andando y validadas)

Es una forma de medir todas las cosas que puede hacer nuestro software graficado por cada sprint, como es evolutivo en general es aditivo a menos que cambie un requerimiento ya resuelto.

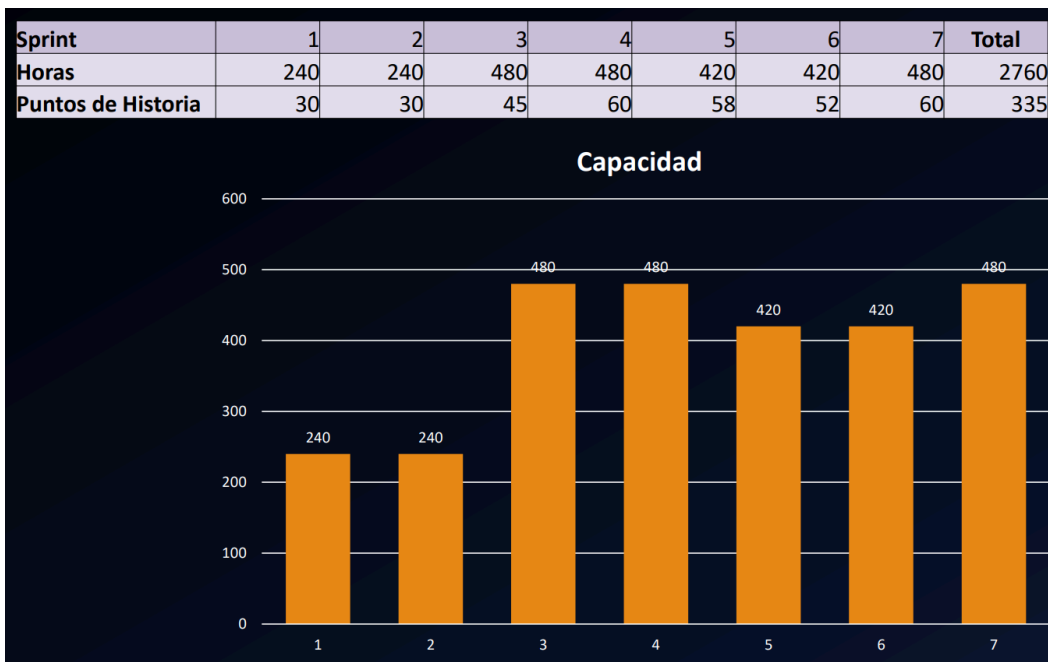


"El Software trabajando es la principal medida de progreso."

Capacidad

La capacidad son las horas de trabajo disponibles por día (WorkHour) multiplicada por la cantidad de días trabajados en esa iteración. Es decir que medimos nuestra capacidad sumando todas las horas del mes que van a trabajar los miembros de nuestro equipo.

Capacidad = WH * Días * Equipo





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Velocidad

Esta métrica es de progreso de nuestro equipo y se calcula sumando los números de story points asignados a las US que el equipo **COMPLETA** durante una iteración. Aquí es donde entra en juego la Definition of Done ya que este es el parámetro de aceptación para contar los Story Points. Aclaró que además tiene que ser aprobada por el PO no importa nuestro DoD porque es quien tiene la última palabra **SIEMPRE**.

Con esto podemos corregir nuestros errores de estimación, ya que nos muestra sólo aquellas tareas que completamos al 100% sin errores o tareas pendientes.

Framework para escalar SCRUM

Debemos considerar que el Scrum está hecho para equipos pequeños de 14 personas máximo que es autogestionado en una estructura “plana”. Como sabemos en la vida real los grandes proyectos van a necesitar expandir sus miembros y estar en diversos proyectos a la vez. Para lograr su escalabilidad existen muchas metodologías y marcos de los cuales mencionaremos un par:

Nexus

 Es un framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente 3 a 9 personas EQUIPOS DE SCRUM que trabajan sobre el mismo Product backlog común para la construcción de un incremento de un producto “terminado” y cuenta con un solo PO

Responsabilidades

- Nexus Integration Team:
 - Es quien toma la responsabilidad de velar por que se produzcan incrementos integrados por sprint, la integración propone nuevas restricciones y desafíos técnicos de los equipos.
- PO:
 - Es responsable de maximizar el valor del producto y del trabajo realizado e integrado, a su vez continúa con la responsabilidad del Product Backlog
- Scrum Master:
 - Responsable de asegurar que el marco de trabajo nexus se entienda y se promulgue como se describe en la guía de nexus.
- Uno o más miembros:
 - A menudo está formado por miembros de los diferentes equipos.

Eventos

- Nexus Sprint:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Los scrums teams producen una sola integración.
- Nexus refinamiento entre equipos:
 - El refinamiento entre equipos del trabajo del backlog reduce o elimina las dependencias entre equipos dentro de un Nexus, el product backlog debe descomponerse para que las dependencias sean transparentes.
- Nexus Daily Scrum:
 - Es la misma daily pero asiste un representante por equipo,
- Nexus Sprint Review:
 - Es una review de la integración y no de cada equipo.
- Nexus sprint retrospective:
 - El propósito es retroalimentar no solo cada equipo sino la estructura en conjunta de nexus.

Usa los mismos artefactos que SCRUM

LeSS

Es un marco de trabajo que se aplica a productos de entre dos y ocho equipos, con un solo PO y un SM cada uno a tres equipos, gestionando un único Backlog.

Principios

Scrum a gran escala es más scrum

1. Pensamiento sistémico
2. Pensamiento Lean
3. Control empírico
4. Transparencia
5. Mejora continua
6. Centrado en el consumidor
7. Enfoque en producto completo
8. Teoría de colas

Miembros del equipo

Son los mismos

Eventos

- **Sprint Planning One:** Es atendida por el PO y los representantes, se generará una lista tentativa
- **Sprint Planning Two:** Cada equipo decide una lista de ítems seleccionados para generar su sprint backlog
- **Daily:** Cada equipo tiene su propio daily.
- **Review:** Los stakeholders son parte



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- **Retro:** retro individual de cada equipo
- **Overall Retro:** Se reúne el LeSS y trabajan en lo bueno y malos del trabajo en equipo
- **Refinamiento del Backlog:** Igual al SCRUM
- **Sprint:** misma duración para todos los equipos

Mismos artefactos

Less Huge

Es para más de 8 equipos.

A diferencia del LeSS hay un área de PO, PM, Backlogs separados y uno general para lograr la separación de intereses necesarios.

Métricas Ágiles

Dominio de métricas

Se dividen en 3 dominios, el cual cada uno posee un foco distinto de medición:

Métricas de proceso:

Son métricas estratégicas a nivel organización, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software, son públicas, responsabilidad del ingeniero de procesos, quien realiza las mediciones y luego publica los datos para que todos los empleados de la organización puedan acceder a ellas.

Permite tener una visión profunda de la eficacia de un proceso, determinando que funciona de acuerdo a lo esperado y que no.

- Desviación organizacional de estimaciones
- Defectos por severidad en productos
- Errores previos a releases pro proyecto
- Defectos detectados por usuarios
- Esfuerzo realizado
- Tiempo de planificación promedio por proyecto
- Programación de errores de fase a fase

Métricas del proyecto:

Están enfocadas a los recursos que se dedican al proyecto, como costo, esfuerzos, estimaciones y tiempo. Son responsabilidad del líder de proyecto y permite al equipo adaptar el desarrollo de los proyectos y de la actividad técnica.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Se usan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzca riesgos potenciales y por lo tanto baja el costo total del proyecto

- Eficiencia de estimación del proyecto
- Costos estimados versus costos reales
- Esfuerzos/Tiempo por tarea del ingeniero en software
- Errores no cubiertos por hora de revisión
- Fechas de entregas reales versus programadas
- Cantidad de cambios y sus características

Métricas de Producto:

Están enfocadas en lo que se construye, son responsabilidades del equipo de desarrollo y de testing y son particulares de ese producto. Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficiencia del análisis, diseño, estructura de código, efectividad de los CP y calidad.

Se deben controlar los artefactos resultantes del proceso de desarrollo para garantizar:

1. Que se cumplan los requerimientos del cliente
2. Que se cumplan los requerimientos de calidad
3. Libre de errores
4. Que se realizaron bajo procedimientos de calidad

Se puede ver pero no usar de métrica:

- Cantidad de líneas de código del producto
- Defectos por severidad de un producto
- Promedio de métodos por clase
- Cantidad de métodos de cada clase
- Cantidad de casos de uso por complejidad

Gestión de Productos de Software

Propuesta de Valor Única (UVP):

Bueno todo empieza con este concepto. Una declaración clara y concisa que explica por qué un producto o servicio es diferente y mejor que el de la competencia. Es el factor que hace que los clientes elijan tu producto sobre otros disponibles en el mercado. Nosotros acá vamos a plantear una hipótesis que explique por qué nuestro producto / servicio será único



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Producto Mínimo Viable (MVP):

Eric Ries: Versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre clientes con el menor esfuerzo. Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos.

- Dirigido a un subconjunto de clientes potenciales
- Utilizado para obtener aprendizaje validado
- Más cercano a los prototipos que a una versión real funcionando de un producto
- Tiene el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo inicialmente
- Demuestra suficiente beneficio futuro para retener a los primeros usuarios
- Proporciona un ciclón de retroalimentación para guiar el desarrollo futuro

Justamente una de las cosas que nosotros queremos lograr con nuestro MVP es validar la hipótesis antes establecida acerca de por que nuestro producto / servicio es único

El MVP nos permite validar las diferentes hipótesis:

Hipótesis de valor: pruebas si el producto realmente está entregando valor a los clientes después de que comienzan a usarlos

Hipótesis de crecimiento: prueba como nuevos clientes descubrirá el proyecto.

Cómo cocinar un MVP:

1. Encontrar el nicho de mercado
2. **Roadmap:** hacer una definición de muy alto nivel de qué características del producto, para inversores.
3. Investigar la competencia: ver hasta donde llega, que hace, que fortalezas tiene.
4. Pre vender el MVP
5. Testear las suposiciones
6. Asegurar que el MVP resuelva el problema correcto
7. Enfocarse solo en las funcionalidades principales

Característica Mínima Viable (MVF):

El MVF incluye solo lo necesario para que la característica cumpla su propósito básico. Esto significa que no contiene extras o funciones adicionales que no sean fundamentales para la operación de la funcionalidad principal. Es como una versión en miniatura del MVP. Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Al igual que un MVP (Minimum Viable Product), la idea detrás del MVF es lanzar rápidamente una versión funcional de la característica para obtener retroalimentación de los usuarios reales. Esto permite validar si la nueva funcionalidad es útil, si satisface las necesidades de los usuarios y si vale la pena invertir más en su desarrollo.

Característica Mínima Comerciable (MMF):

Es una característica a pequeña escala que se puede construir e implementar rápidamente utilizando recursos mínimos, para una población objetivo para probar la utilidad y la adopción de una característica. El enfoque del MMF está en la comercialización. A diferencia del MVF, que puede ser más básico y enfocado en la validación, el MMF debe estar lo suficientemente completo como para que los usuarios lo perciban como una mejora valiosa del producto y, por lo tanto, justifique su promoción o venta.

Es parte de un MMR o un MMP

Ejemplo: Siguiendo con la aplicación de pedidos de comida, una MMF podría ser la integración de pagos en línea. Esta funcionalidad por sí sola agrega valor significativo al producto, ya que permite a los usuarios completar el proceso de compra de manera más conveniente.

Producto Mínimo Comerciales (MMP):

El MMP es una versión del producto que es lo suficientemente completa para ser lanzada al mercado y satisfacer a los primeros usuarios. Incluye todas las funcionalidades necesarias para que el producto sea comercializable y competitivo, aunque no necesariamente tenga todas las características finales planeadas.

Primer release de un MMR dirigido a primeros usuarios - early adopters - Focalizado en características clave que satisfagan a ese grupo clave

Las Características Mínima de Release (MMR):

¿Que tiene que tener como características mínimas un release de un producto para poder ser liberado?

Release de un producto que tiene el conjunto de característica lo más pequeño posible

Un incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales

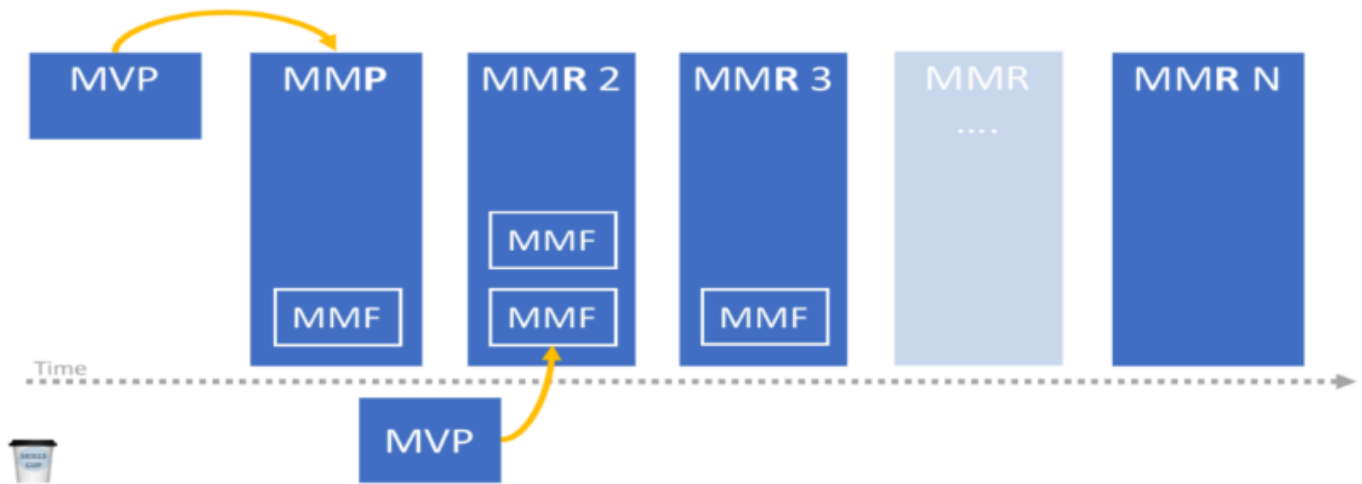
MMP = MMR1



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

¿Cómo se relacionan todos estos conceptos?



¿Qué errores comunes podemos cometer con estos conceptos?

Enfatizar la parte mínima del MVP con exclusión de la parte viable. El producto entregado no es de calidad suficiente para proporcionar una evaluación precisa de si los clientes utilizarán o no el producto

Entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.

Confundir un MVP que se enfoca en el aprendizaje, con una MMF o con un MMP que ambos se enfocan en ganar

El dilema de la audacia cero

El "dilema de la audacia cero" es un concepto que se refiere a la paradoja que enfrentan las organizaciones o individuos cuando, en un intento de evitar cualquier tipo de riesgo o fracaso, se vuelven tan cautelosos que terminan estancándose o siendo incapaces de innovar o progresar.

Si vos empezas a posponer la experimentación con el MVP, van a surgir algunos resultados desafortunados como:

- La cantidad de trabajo desperdiciado puede aumentar
- Se perderán los comentarios esenciales
- EL riesgo de que su startup construya algo que nadie quiere puede aumentar



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Compensaciones:

- ¿Preferiría atraer capital de riesgo y potencialmente derrocharlo?
- ¿O preferiría atraer capital de riesgo y utilizarlo sabiamente?
- Use un MVP para experimentar (inicialmente, en silencio) con los primeros usuarios del mercado

Verifique su concepto probando TODOS sus elementos, comenzando por los más riesgosos

KANBAN

Introducción

Es una palabra japonesa que significa “señal visual” siendo un enfoque de gestión de las formas de trabajo para obtener la mejora continua. Tu trabajo suele ser invisible e intangible por lo que utilizamos un tablero para visualizar para mostrárselo a los demás. Mediante el principio el principio se empieza por donde estés, planteando mejoras graduales a un proceso de desarrollo de software con una metodología de administración de proyectos ya existentes. Todo esto sirve para reducir la resistencia al cambio por parte de las personas fomentando gradualmente los procesos existentes.

Aprovecha los principios de LEAN (Minimalismo, ver el todo, ser eficiente, etc)

- Definiendo valor desde la perspectiva del cliente
- Limitando el trabajo en proceso
- Identificando y eliminando desperdicios
- Identificando y eliminando barrera de flujo que atrasa el proceso
- Cultura de mejora continua

Los desperdicios suelen ser 7+1:



Donde el “+1” es el potencial humano desperdiciado. El potencial humano que se desperdicia es el exceso de conocimientos que tiene un empleado o miembro del proyecto con respecto a su



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

tarea. Por ejemplo, un ingeniero en sistemas laburando de programador raso, un especialista en seguridad en mesa de soporte, etc. Es inevitable en muchos casos ya que a menos que tus personas se adapten al 100% a su puesto y no sepan ni un poquito mas no podemos evitar que excedan las capacidades. El tema es no llegar a un limite no sano donde la persona se siente en una situacion laboral poco “desafiante” que le interese cambiar por “nuevos desafios”.

Estos desperdicios de LEAN se adaptan inevitablemente a los sistemas o “basados en conocimiento” donde podemos ver la analogía con los originales.



Los siete desperdicios Lean (en servicios basados en conocimiento)

Desperdicios en Manufactura	Desperdicios en servicios basados en conocimiento
Inventario	Trabajo parcialmente terminado
Sobreproducción	Características extra
Proceso extra	Proceso extra
Transporte	Cambio de tareas Transferencia de conocimiento
Esperas	Esperas/Demoras
Movimiento	Cambio de contextos (Seteo)
Defectos	Defectos
Talento no utilizado	Talento no utilizado

Además Judith “La Matrona” Meles dice que según la bibliografía existen dos tipos.

El primero es aquel que es inevitable, necesario y producto indirecto de nuestro proceso. En este caso se busca minimizarlo para ser más efectivos. No pone ejemplos pero para referenciar mi laburo en provincia de Córdoba, existen tiempos de espera que se deben a la misma burocracia de seguridad (osea que no está mal). Qué ocurre, cuando necesito acceso a una estructura de datos tengo que solicitar permisos, se tiene que analizar si me corresponde para ese desarrollo y así darmelos. No está mal que exista, pero minimizarlo es lo correcto.

En segunda instancia tenemos aquellos que sí podemos eliminar del tipo II, como pueden ser defectos en el proceso de información que podemos optimizar y así mejorar.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Prácticas de kanban

Se visualiza el trabajo necesario por el cliente, divide el trabajo en piezas del mismo y las ubica en el tablero. Visualiza cada tarjeta en que parte del proceso está, representando un sistema de flujo, donde cada columna es una etapa, hay 2 tipos de etapas de producción o acumulación.

VISUALIZAR



Muestra el trabajo y su flujo.
Visualiza los riesgos.
Construye un modelo visual que refleje cómo se trabaja.

LIMITAR EL TRABAJO EN CURSO



¡Deja de empezar, empieza a terminar!
De derecha a izquierda. Limita el trabajo en el sistema a la capacidad disponible, basándote en los datos.

GESTIONAR EL FLUJO



El flujo es el movimiento del trabajo.
Gestiona el flujo para ser predecible y confiable.
Utiliza los datos.

HACER LAS POLÍTICAS EXPLÍCITAS



Tener políticas acordadas, visibles para todos los involucrados:
- Criterios de Pull (atracción/arrastre)
- Límites de WIP (trabajo en curso)
- Clases de servicio
- Otros que correspondan

ESTABLECER CICLOS DE RETROALIMENTACIÓN



Establece ciclos de retroalimentación con la cadencia adecuada. Fomenta la colaboración, el aprendizaje y las mejoras. Basado en datos.

MEJORAR COLABORATIVAMENTE, EVOLUCIONAR EXPERIMENTALMENTE



Usando el método científico.
El cambio impulsado por hipótesis.
Ejecutar experimentos para avanzar o aprender (*safe-to-fail experiments*).

Una aclaración de nuestra soberana es que KANBAN no trabaja con “iteraciones” es un solo flujo enorme que está todo el tiempo en mejora continua.

Visualizar

Es para tener un consenso general con todos los implicados en el proyecto para que puedan absorber toda la información en un corto periodo de tiempo. Nos enfocamos en crear modelos visuales que representen nuestras actividades.

Limitar el WIP

La abreviatura WIP significa **“Work In Progress”** o “trabajo en proceso” en español. En pocas palabras, WIP es la cantidad de tareas en las que un equipo está trabajando actualmente. Se debe asignar límites conocidos a la cantidad de piezas que pueden estar en cada columna, esto es para evitar cuellos de botellas por atrasos y optimizar recursos disponibles

Gestionar flujos de trabajo

El tablero puede ser compartido con otros proyectos y otros equipos, ya que se trabaja cada pieza de trabajo de forma individual. Relacionado al WIP intentamos no saturar los canales de comunicación ni los procesos, ya que el exceso de información puede abrumar a aquellos que continúan con la tarea. Siempre está bueno manejar un nivel de holgura para tener flexibilidad ante nuevas situaciones que ocurran.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Hacer políticas explícitas

Las políticas de proceso deben ser escasas, simples, bien definidas, visibles, de fácil aplicación y tienen que ser fácilmente modificables.



Establecer ciclos de retroalimentación

Propone mejoras al proceso a medida que lo va completando y reiniciando de acuerdo a las necesidades del equipo. Esencial en cualquier proceso de control que nos ayudan a realizar cambios evolutivos, se definen las frecuencia de reunión donde depende el contexto donde se presentan es la importancia del resultado

Mejora colaborativamente evolucionar experimental

Es aprender mediante las hipótesis y el método científico buscamos mejorar nuestro avance.

Métricas Kanban

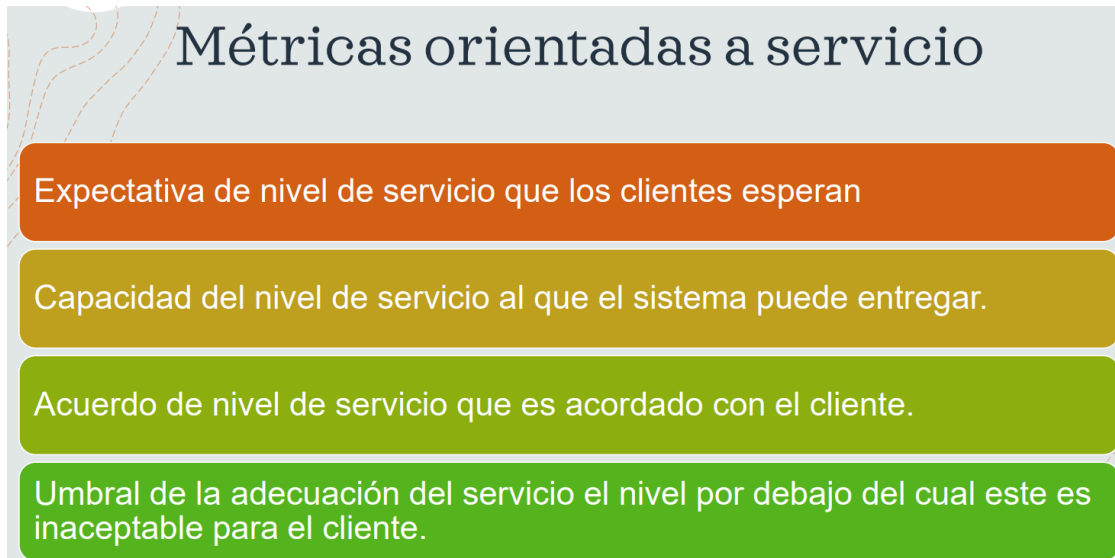
- Cycle time
- Lead time



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Touch time
- Eficiencia de ciclo de proceso



Metricas Lean (Kanban)

El foco de medición es el proceso ya que kanban, como veremos más adelante, propone trabajar sobre un flujo de trabajo, no existe un trabajo en sí de principio a fin. Es decir se mide el comportamiento del proceso en función al tiempo que se demoran las características.

Lead Time o Elapsed Time

Métricas de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que cliente pide algo hasta que se entrega

Cycle time

Mide el tiempo desde que el equipo empezó a trabajar en una funcionalidad y se entregó.

Touch Time

Mide los tiempos los cuales las personas efectivamente trabajaron sobre una unidad de trabajo, sin tener en cuenta los tiempos de espera

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Eficiencia del Ciclo de Proceso

$$\% \text{ Eficiencia ciclo proceso} = \text{Touch Time} / \text{Elapsed Time.}$$



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Tablero Kan

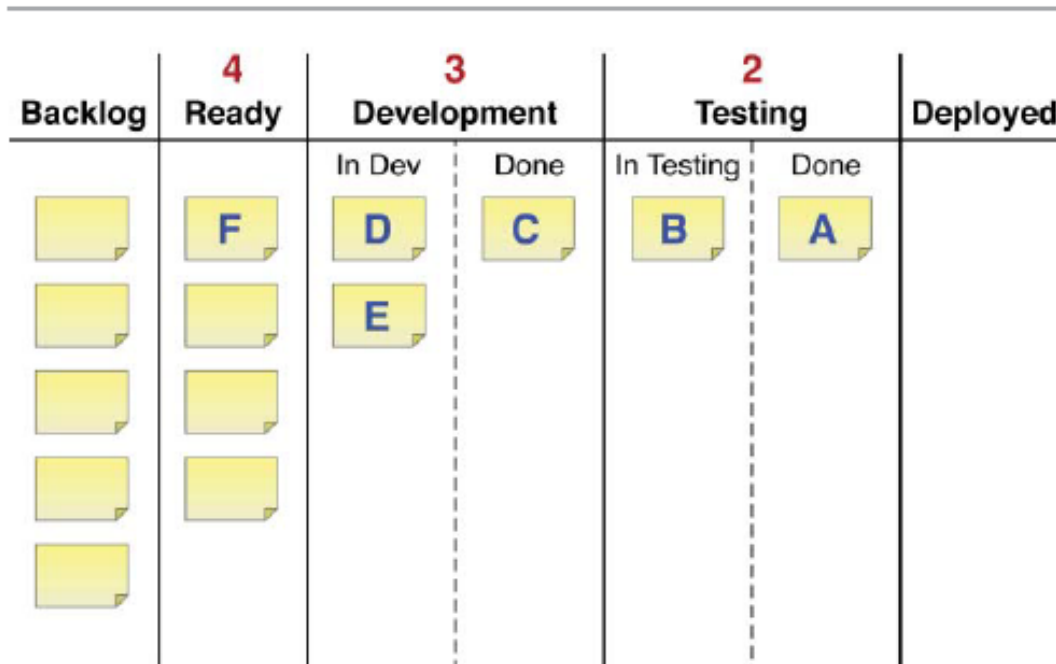


FIGURE 12.4 Kanban Board

Este es un ejemplo minimalista, lo que tienen que acordarse siempre es de ver el tema del WIP (número rojo arriba), que es la cantidad de tareas **MÁXIMAS** que podemos tener. Esto quiere decir que no puede haber más de x cantidad de tareas, por más que estén completadas (Done). Se usa para evitar cuellos de botellas y si notamos mucha capacidad ociosa es que estamos distribuyendo mal el personal en las diferentes partes del proceso. Otro detalle es que las tareas se pasan con un “PULL”, donde el área **SIGUIENTE** es la que trae a su parte las tareas finalizadas del anterior. Esto es importante ya que vos no podes decir, termine le pateó la tarea al que sigue. Vos terminas y si tu WIP te lo permite traes una tarea del proceso anterior, sino a mamarla.

Valores de KANBAN

1. Transparencia
2. Respeto
3. Equilibrio
4. Colaboración
5. Liderazgo
6. Acuerdo
7. **Flujo (Este es propio de KANBAN)**
8. Entendimiento
9. Foco en el cliente

Principios

- **Gestión de Cambios**



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Comenzar con lo que haces ahora
- Entender los procesos actuales tal y como están siendo realizados en la actualidad, Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo.
- Acordar la búsqueda de la mejora a través del cambio evolutivo
- Fomentar actos de liderazgo a todos los niveles
- **Entrega de Servicios**
 - Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
 - Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él
 - Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados

LEAN UX

Introducción

Es un modelo Lean donde se retroalimenta de la experiencia de usuario, sirviendo de pruebas pivotantes de bajo costo, nos permite equivocarnos en estadios tempranos y es barato reconstruir ahí la idea. Es la mezcla de metodologías ágiles con usabilidad, donde se busca darle más importancia a la experiencia que se genera con la creación de un producto que a los entregables que surjan de este. Lean UX nació para enfrentar las rápidas transformaciones que vive el mundo del desarrollo web.

The Build-Experiment-Learn feedback loop permite descubrir las necesidades del cliente y alinearlas metodológicamente.

Las fase de construir MVP:

- Ingresar lo más rápido posible un producto mínimo viable
- Un MVP varía en complejidad desde pruebas de humo extremadamente simples.

Metodología Lean UX

El proceso Lean UX se centra en tres fases clave: pensar, hacer y comprobar. Si bien puede ser más fácil verlos como 3 pasos que se llevan a cabo en línea recta, nada podría estar más lejos de la realidad. Lean UX es un proceso cíclico que continúa, con el producto mejorando y avanzando cada vez que el ciclo se completa y comienza de nuevo.

Pensar

Los equipos realizan una lluvia de ideas sobre las posibles áreas de mejora basándose en suposiciones sobre quiénes son sus usuarios, para qué van a usar el producto que está diseñando y



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

cuándo, cuáles serán las funciones y características más importantes, etc. A partir de aquí, desarrollan un planteamiento del problema y una hipótesis para poder decidir qué áreas quieren mejorar.

Un ejemplo de hipótesis puede ser:

Los usuarios abandonan el carrito de compras porque deben crear una cuenta para realizar el pedido. Eliminar este paso, puede disminuir el número de incidencias de «carrito abandonado» y aumentar las ventas. Para comprobarlo debemos tener en un mes, un aumento en las ventas del 10%.

Como ves es importante que, al escribir la hipótesis establezcas un objetivo medible sobre cómo vas a comprobar la hipótesis.

Hacer

Esta fase implica diseñar un Producto Mínimo Viable (MVP) que ayude a atraer a tu audiencia y permita iniciar un ciclo de retroalimentación de iteración. Un MVP es esencialmente la versión más básica que necesitarás para probar tu hipótesis y recopilar aprendizajes y comentarios iniciales. Puedes recurrir a diferentes métodos según tu hipótesis. Ya sea creando un prototipo de cómo sería el recorrido del usuario sin tener que crear una cuenta, para continuar con el ejemplo anterior, o en otros casos, a través de creación de estructuras alámbricas, landings, wireframes, etc.

Comprobar

La verificación es la última etapa del ciclo Lean UX. Aquí se recopilan comentarios del MVP para invalidar o validar la hipótesis original. Para ello puedes recurrir a diferentes herramientas como el test A/B, encuestas de satisfacción, análisis del sitio, entre otras. Si los clientes responden bien a la nueva función, pasa a formar parte del nuevo diseño. Si no mejora la experiencia del cliente, deberás volver a la fase de reflexión y probar algo nuevo

El éxito no es entregar un producto, el éxito se trata de entregar un producto y que el cliente lo use.

- Un sabio -

Design Thinking

Método destinado a la creación de servicios digitales centrada en el cumplimiento de las expectativas del usuario mediante un proceso de mejora continua que advierte las verdaderas necesidades de este.

Para llevar a cabo este proceso, utilizamos un esquema de pensamiento divergente y convergente. cuenta con 5 pasos para hacer esto:

1. **Empatizar:** comprender las necesidades del cliente
2. **Definir:** determinar lo valioso para el cliente a partir de lo recopilado durante la primera etapa, lo cual permitirá guiar el trabajo.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

3. **Idea:** articular las soluciones posibles.
4. **Prototipar:** consiste en materializar las ideas en un prototipo de solución propuesta.
5. **Probar:** validar que el prototipo convenza a los usuarios implicados en la solución, retroalimentar e iterar.

UNIDAD N° 3

Conceptos que conforman la disciplina Gestión de Configuración de Software.

Introducción

Entendemos a SCM como una disciplina de soporte → Es una actividad **paraguas**, transversal a todo el proyecto, relevante para el producto a lo largo de su ciclo de vida

Tiene sus orígenes a mediados de 1950 's, como CM (por Configuration Management) originalmente utilizado para desarrollo de hardware y control de producción, fue utilizado en el desarrollo de software.

SCM es una disciplina que aplica dirección y monitoreo administrativo y técnico a:

- Identificar y documentar las características funcionales y técnicas de los ítems de configuración
- Controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación
- Verificar correspondencia con los requerimientos

(ANSI / IEEE 828, 1990)

Configuración de software:

Es un conjunto de ítems de configuración con su correspondiente versión en un momento determinado

¿Por qué deberíamos gestionar la configuración del software?

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida.

Involucra para la configuración:

- Identificarla en un momento dado
- Controlar sistemáticamente sus cambios
- Mantener su integridad y origen



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

La Integridad del Producto, es importante porque:

- Satisface las necesidades de los usuarios
- Permite que el software sea completamente rastreado durante su ciclo de vida
- Satisface criterios de performance
- Cumple con sus expectativas de costo

¿Qué problemas surgen en el manejo de componentes?

- Pérdida de un componente
- Pérdida de cambios (el componente que tengo no es el último)
- Sincronía fuente - objeto - ejecutable
- Regresión de fallas
- Doble mantenimiento
- Superposición de cambios
- Cambios no válidos

Es importante entender que los desarrolladores de software en realidad son un conjunto complejo de entidades organizacionales que interactúan entre sí. Cuando nosotros emprendemos un proyecto de software, estos desarrolladores siempre se van a estructurar dentro de 3 disciplinas básicas:

Administración del proyecto: Las disciplinas de gestión de proyectos están dirigidas tanto hacia dentro como hacia fuera. Básicamente le sirven a la gestión general para ver qué es lo que sucede en el proyecto y asegurarse de que la organización desarrolle el producto con integridad. Además, estas disciplinas nos otorgan una visión interna del proyecto, en función de la asignación de los recursos del proyecto

Desarrollo: Representan las disciplinas tradicionalmente aplicadas al proyecto de software

- Análisis
- Diseño
- Ingeniería
- Producción
- TEsting
- Entre otras

Son básicamente lo que se requiere para llevar un concepto de un sistema hacia un ciclo de vida de desarrollo

Aseguramiento del producto: Estas disciplinas son utilizadas por los administradores del proyecto para ganar visibilidad dentro del desarrollo del proceso. Aquí se incluyen:

- **Gestión de configuración:** Esto es básicamente una disciplina que se encarga de identificar la configuración de un sistema a momentos discretos en el tiempo con el propósito de controlar sistemáticamente los cambios y mantener una trazabilidad a lo largo del ciclo de vida
- **Aseguramiento de calidad:**



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- **Validación y verificación:** Aquí lidiamos con el problema de que tan bien el software cumple con los requerimientos funcionales y de performance y que los requerimientos hayan sido interpretados correctamente
- **Testing y evaluación:** Es una disciplina impuesta por fuera del desarrollo del proyecto para que de manera independiente evaluar si el producto cumple o está en línea con los objetivos

El uso adecuado de estas disciplinas de aseguramientos de proyectos por parte del project manager son la base para el éxito del proyecto.

Conceptos del SCM

Ítem de Configuración de Software (SCI):

(Documentos de diseño, código fuente, código ejecutable, etc)

Se llama ítem de configuración (IC) a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.

Algunos ejemplos de SCI puede ser:

- | | |
|-----------------------------|-------------------------------|
| ❖ Plan de CM | ❖ Planes de Iteración |
| ❖ Propuestas de Cambio | ❖ Estándares de codificación |
| ❖ Visión | ❖ Casos de prueba |
| ❖ Riesgos | ❖ Código fuente |
| ❖ Plan de desarrollo | ❖ Gráficos, iconos, ... |
| ❖ Prototipo de Interfaz | ❖ Instructivo de ensamble |
| ❖ Guía de Estilo de IHM | ❖ Programa de instalación |
| ❖ Manual de Usuario | ❖ Documento de despliegue |
| ❖ Requerimientos | ❖ Lista de Control de entrega |
| ❖ Plan de Calidad | ❖ Formulario de aceptación |
| ❖ Arquitectura del Software | ❖ Registro del proyecto |
| ❖ Plan de Integración | |



INGENIERIA Y CALIDAD DE SOFTWARE

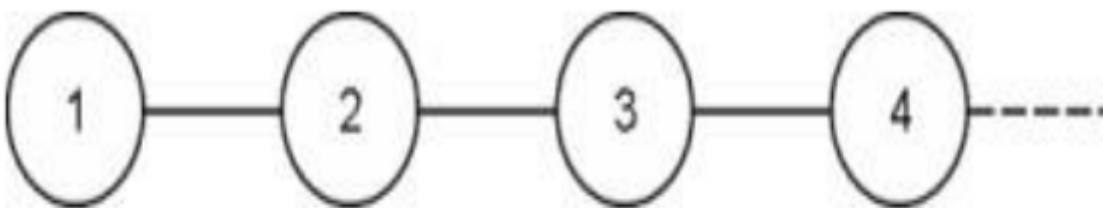
RESUMEN FINAL Y ULTRAVIOLENTO

Versión:

Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.

El control de versiones se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado.

La evolución puede representarse gráficamente en forma de grafo.



Evolución lineal de un ítem de configuración

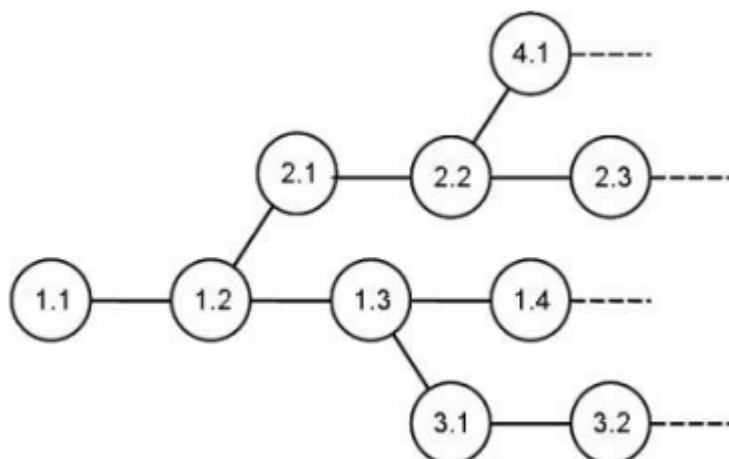
Variante:

Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado.

Las variantes representan configuraciones alternativas.

Un producto de software puede adoptar distintas configuraciones dependiendo del lugar donde se instale.

Por ejemplo, dependiendo de la plataforma (máquina + S.Ó.) que la soporta, o de las funciones opcionales que haya de realizar o no.



Variante de un ítem de configuración



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Línea Base

La línea base es un conjunto de ítems de configuración que están en una versión “estable”, lo que quiere decir que bajo una serie de estándares se definió que estos ítems representan un conjunto verificado. Sirve en líneas generales para que tengamos a donde volver en caso que algo ocurra más adelante o en todo caso si queremos implementar el producto en algún lado se utilice la versión correcta de todos los items ya que sabemos que funciona correctamente. En caso de querer plantearse un cambio o establecer una nueva línea base se debe hacer mediante un comité de cambios que audita la integridad del repositorio.

En resumen:

- Se utilizan etiquetas para marcar las baselines
- No debemos confundir con la versión del producto
- Entonces una línea base es una configuración que ha sido revisada formalmente y sobre la que se ha llegado un acuerdo.
- Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios.
- Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto
- Las líneas base puede ser:
 - De especificación DE (Requerimientos, Diseño)
 - De productos que han pasado por un control de calidad definido previamente

Repositorio:

Un repositorio de información conteniendo los ítems de configuración, mantiene el historial de cada SCI con sus atributos y relaciones. Usado para hacer evaluaciones de impacto de los cambios propuestos. Puede estar compuesta de una o varias bases de datos o repositorios.

Con respecto al funcionamiento de un repositorio, es bastante sencillo. Básicamente, tenemos el repositorio fuente, a partir del cual se puede realizar 2 acciones:

Extracción (Check out): Es el proceso mediante el cual un desarrollador toma una copia del código fuente desde el repositorio hacia su entorno local de trabajo. Esto sería análogo al **git pull**

Devolución (Check in): Una vez que el desarrollador ha realizado cambios en el código, devuelve esos cambios al repositorio. Esto sería análogo al **git push**

Entonces habría una actividad intermedia entre que uno construye los datos en su entorno de trabajo y los envía nuevamente al repositorio, lo cual sería básicamente hacer un git commit

Repositorios centralizados:

- Un servidor contiene todos los archivos con sus versiones
- Los administradores tienen mayor control sobre el repositorio



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Si falla el servidor estamos al horno

Repositorios descentralizados:

- Cada cliente tiene una copia exactamente igual del repositorio completo
- Si un servidor falla es solo cuestión de copiar y pegar
- Posibilita otros workflows, cosa que no está disponible en el modelo centralizado

Planificación de la Gestión de Configuración de Software debe incluir:

- Reglas de nombrado de los CI
 - EJ: CaspianSeaV1
- Herramientas a utilizar para SCM
 - EJ: Git
- Roles e integrantes del comité
 - Si es ágil tenemos un control de cambios por un comité de gente definido
- Procedimiento formal de cambios
 - Lo vemos más adelante
- Plantillas de formularios
 - Para que ingrese la información de manera homogénea
- Procesos de auditoría
 - Cada cuánto y cómo controlamos que esté todo en orden.

Ramas

Creación de ramas:

- Existen una rama principal (trunk, master)
- Sirven para bifurcar el desarrollo
- Pueden tener razones de creación con semántica
- Permiten la experimentación
- Pueden ser descartadas o integradas

Integración de ramas:

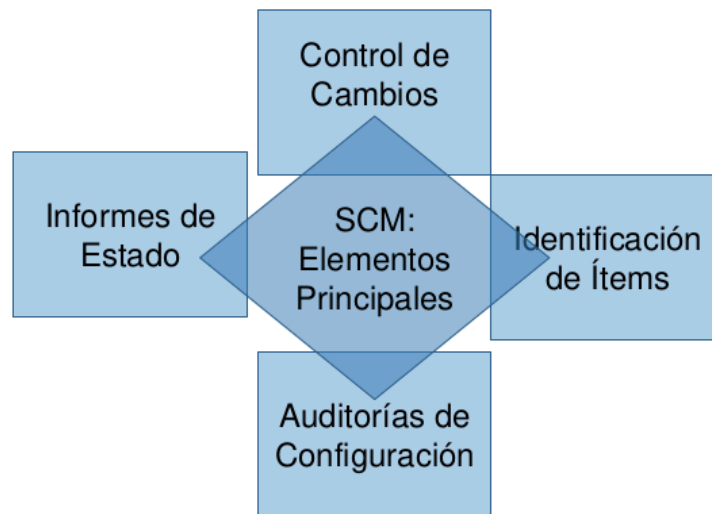
- La operación se conoce como merge
- Lleva los cambios a la rama principal
- Pueden surgir conflictos (resolverlos con diff)
- Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas

Actividades relacionadas con la Gestión de Configuración.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Lo que estás leyendo nada más que sintetizado en un bello diagrama y describimos las actividades

Identificación de Ítems

- Identificación unívoca de cada ítem de configuración
- Convenciones y reglas de nombrado
- Definición de la Estructura del Repositorio
- Ubicación dentro de la estructura del repositorio

Los IC puede ser de diferentes tipos:



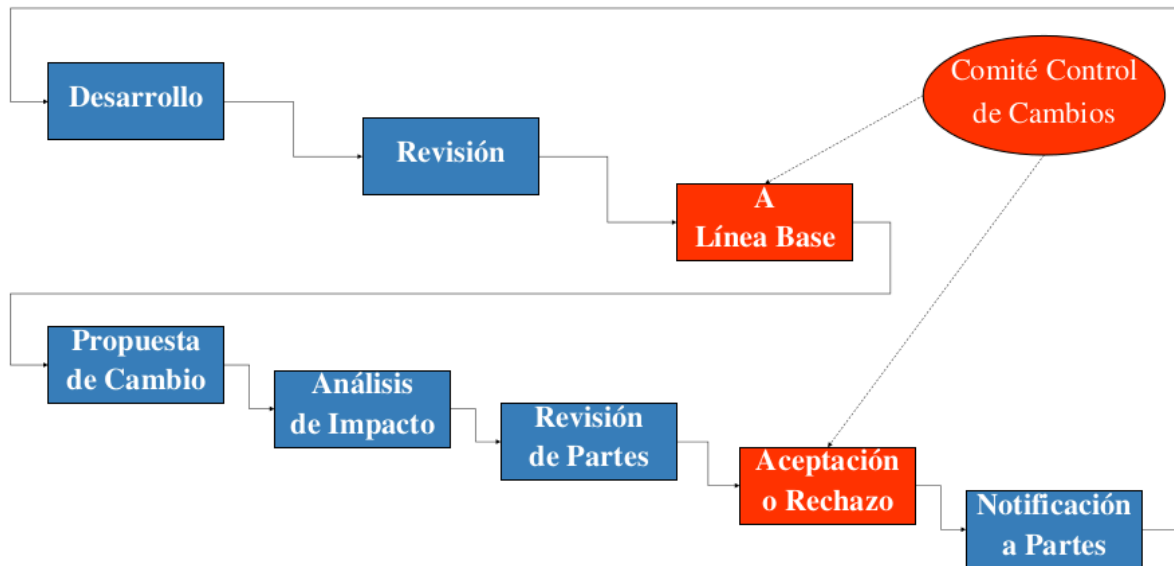
Control de cambios:

El proceso de control de cambios sería algo como esto:



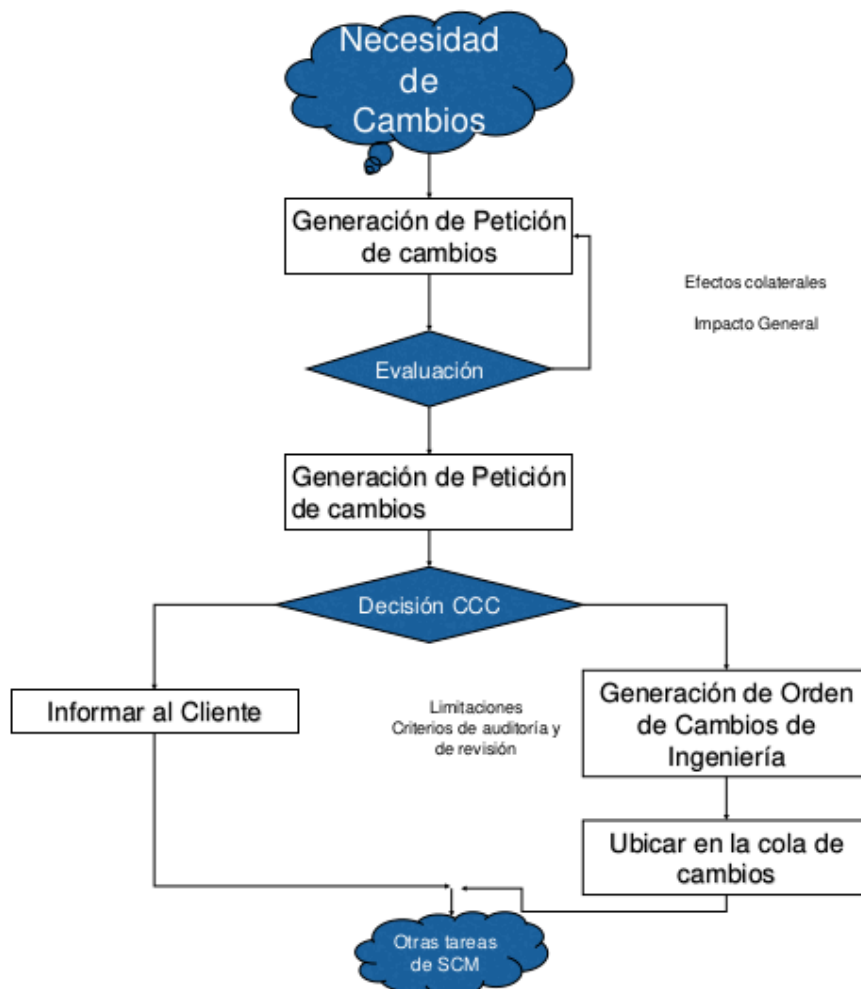
INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



El control de cambios tiene su origen en un requerimiento de cambio a uno o varios ítems de configuración que se encuentran en una línea base. Es un procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio

Un diagrama de flujo sobre la situación cuando un cliente solicita un cambio:





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

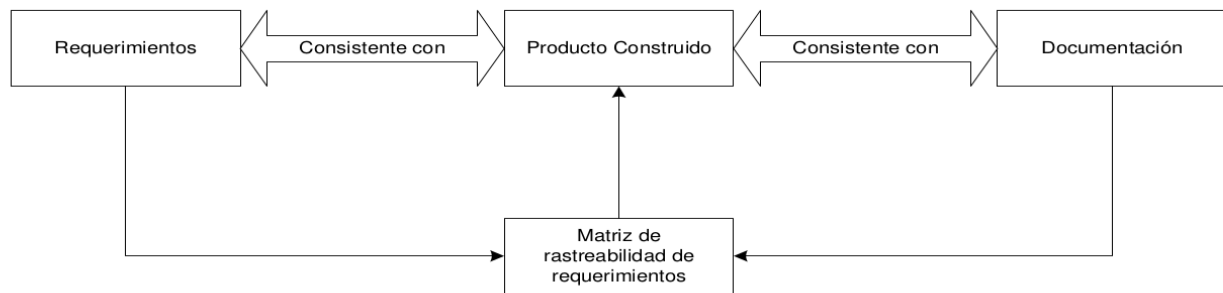
El comité de control de cambios está formado por representantes de todas las áreas involucradas en el desarrollo:

- Análisis, Diseño
- Implementación
- Testing
- Otros interesados

Auditorías de Configuración de Software:

Auditoría Funcional de Configuración

Auditoría Física de Configuración



Auditorías de Configuración:

- **Auditoría física de configuración (PCA):** Asegura que lo que está indicado para cada IC en la línea base o en la actualización se ha alcanzado realmente.
Consistencia entre lo que definimos en el plan de configuración para cada IC, con lo que tengo en el repositorio
Verificación: Asegura que un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base (línea base). Todas las funciones son llevadas a cabo con éxito y los test cases tengan status “ok” o bien consten como “problemas reportados” en la nota de release.
- **Auditoría funcional de configuración (FCA):** Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos.
Valida que el producto a construir sea consistente con los requerimientos identificados. Se usa una matriz de rastreabilidad para encontrar donde se implementan cada requerimiento
Validación: EL problema es resuelto de manera apropiada de manera que el usuario obtenga el producto correcto

Informes de estado:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

El registro e Informe de Estado:

- Se ocupa de mantener los registros de la evolución del sistema
- Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos
- Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida

Algunas preguntas que podría responder son:

- ¿Cuál es el estado del ítem?
- ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
- ¿Qué versión de ítem implementa un requerimiento de cambio aprobado (saber cuál es el componente que contiene la mejora)?
- ¿Cuál es la diferencia entre una versión y otra dada?

Gestión de Configuración en ambientes ágiles

- Sirve a los practicantes (equipo de desarrollo) y no viceversa
- Hace seguimiento y coordina el desarrollo en lugar de controlar a los desarrolladores
- Responde a los cambios en lugar de tratar de evitarlos
- Esforzarse por ser transparente y sin fricción, automatizando tanto como sea posible
- Coordinación y automatización frecuente y rápida
- Eliminar el desperdicio → no agregar nada más que valor
- Documentación Lean y Trazabilidad
- Feedback continuo y visible sobre calidad, estabilidad e integridad

Algunos tipos para SCM en Agile:

- Es responsabilidad de todo el equipo
- Automatizar lo más posible
- Educar la equipo
- Tareas de SCM embebidas en las demás tareas requeridas para alcanzar el objetivo del Sprint.

Prácticas continuas

(Continuous Integration /Continuous Delivery/ Continuous deployment)

Integración continua: Se refiere a la práctica de combinar frecuentemente y de manera automatizada los cambios de código realizados por diferentes miembros del equipo en un repositorio compartido. Esto permite detectar y solucionar errores de integración tempranamente, lo que ayuda a garantizar la calidad del código



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Entrega continua: es una extensión de la integración continua, que implica la automatización del proceso de construcción, prueba y empaquetado del software en cada cambio de código, y la entrega de estos paquetes a un entorno de pruebas o de producción para su evaluación. La entrega continua permite a los equipos de desarrollo detectar y corregir errores en etapas tempranas del ciclo de vida del software.

Despliegue continuo: es una extensión de la entrega continua, que implica la automatización de todo el proceso de despliegue del software en un entorno de producción. Esto permite que los cambios de código sean entregados a los usuarios finales con mayor rapidez y frecuencia, lo que reduce el tiempo de lanzamiento de nuevas funcionalidades y mejora la experiencia de los usuarios.

Despliegue de producto. Estrategias para el despliegue de productos

Nuestros docentes no lo pusieron cuando hablaron de SCM tampoco hay una filmina que tenga este título o tema. Por ende es mi consideración declarar este tema como tierra de nadie, cada uno lo estudia por su cuenta...

Les dejo mi copy paste e interpretación:

Introducción

Una estrategia de despliegue se refiere a la metodología utilizada para modificar o actualizar una aplicación con el objetivo de minimizar el tiempo de inactividad y garantizar una transición imperceptible para el usuario. Esto ocurre constantemente en la vida ya que es necesario para solucionar problemas, brechas de seguridad o adaptarse a las nuevas normativas.

En general hay diversas metodologías en el mercado, acá se resumen las actividades que suelen compartir las mismas:

1. Control de versiones:

- Implica el uso de un repositorio Git u otra herramienta similar para rastrear y gestionar los cambios realizados en el servicio. Esto garantiza un registro completo de las modificaciones. Relacionado al SCM y todas las prácticas vistas.

2. Revisión de código:

- Se trata de un sistema de supervisión de las alteraciones efectuadas en el código. Esto permite un control riguroso sobre las características y funcionalidades introducidas en el servicio, asegurando su correcto funcionamiento.

3. Integración continua:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- a. Este enfoque abarca una serie de procesos diseñados para gestionar la calidad y seguridad del código. Incluye la detección de vulnerabilidades, pruebas de capacidad de carga, construcción de paquetes y pruebas automatizadas. En suma, engloba todos los procedimientos que garanticen la calidad y el funcionamiento adecuado del servicio.

4. Automatización del despliegue:

- a. Implica la implementación de un proceso que permita la liberación de las nuevas características en el servicio sin necesidad de intervención manual. Esto acelera el proceso y reduce la probabilidad de errores humanos.

5. Supervisión de despliegues:

- a. Consiste en un sistema de monitoreo que asegura el óptimo funcionamiento del servicio. Proporciona una visión en tiempo real de su rendimiento y detecta problemas de manera proactiva.

Tipos de despliegues

Estas prácticas son fundamentales para el éxito de una estrategia de despliegue, ya que garantizan la estabilidad, la calidad y la eficiencia en la implementación de cambios en una aplicación. Esta estrategia se enfoca en la implementación de estrategias específicas para incorporar las nuevas funcionalidades en el servicio a desplegar. A continuación, presentamos varias estrategias que se utilizan con frecuencia.

Rolling Upgrade:

En este enfoque, se lleva a cabo una actualización secuencial del servicio, actualizando uno a uno los componentes. Este tipo de despliegue tiende a ser lento tanto en su ejecución como en su capacidad de volver atrás (rollback) y generalmente se emplea en combinación con otras estrategias.

Blue/Green:

Esta es una estrategia muy común que implica el uso de dos versiones del servicio: la versión nueva (blue) y la versión estable existente (green). Los usuarios continúan utilizando la versión verde mientras se prueba y evalúa la versión azul. Cuando se considera lista, los usuarios cambian a la versión azul. Si se detecta algún problema, es posible regresar a la versión verde.

Red/Black:

Similar a la estrategia blue/green, con la diferencia de que se desvía una pequeña porción del tráfico desde la versión negra (estable) hacia la nueva versión. Luego de evaluar su rendimiento, se realiza un desvío aún más grande del tráfico y, finalmente, se redirecciona todo el tráfico hacia la nueva versión. Se mantiene la versión negra en línea para permitir un rollback rápido si es necesario.

Dark Launcher:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Se implementa una nueva versión en paralelo a la versión estable existente. La particularidad aquí es que se duplica una parte del tráfico para observar cómo se comporta la nueva versión bajo una carga duplicada. Luego, se realiza un redireccionamiento parcial del tráfico hacia la nueva versión.

Canary Release:

Esta estrategia implica un despliegue parcial basado en el tiempo y el rendimiento de la nueva versión. Por ejemplo, se podría liberar el 10% del tráfico en intervalos de una hora, siempre y cuando los errores no superen el 5% del total de las respuestas. Esto permite una implementación gradual y controlada.

Consideraciones

En el ámbito de las estrategias de despliegue empresarial, resulta de vital importancia tener en cuenta una serie de factores esenciales con el fin de asegurar su efectividad y lograr los objetivos propuestos con éxito.

Gestión de conexiones de larga duración:

Es necesario abordar las conexiones de larga duración de manera cuidadosa durante la implementación.

Conversiones de Bases de Datos:

Las conversiones de bases de datos pueden ser complicadas y deben ejecutarse y revertirse junto con la aplicación, lo que requiere una planificación detallada.

Microservicios y componentes:

Si la aplicación combina microservicios y componentes tradicionales, es posible que se necesite un tiempo de inactividad planificado para completar la transición.

Requisitos de Infraestructura:

Asegurarse de contar con la infraestructura adecuada para respaldar la estrategia de implementación seleccionada.

Entorno de prueba aislado:

Si el entorno de prueba no está aislado, existe el riesgo de afectar tanto las versiones nuevas como las antiguas durante la implementación. Dado que los usuarios finales generalmente acceden a la aplicación a través de una ruta gestionada por un router, la estrategia de implementación puede centrarse en las funciones de configuración de implementación o en las funciones de enrutamiento.

Las estrategias que se centran en la configuración de la implementación afectan a todas las rutas utilizadas por la aplicación, mientras que las estrategias basadas en funciones de enrutador se aplican a rutas individuales.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Numerosas estrategias de implementación son compatibles con la configuración de implementación, y algunas también aprovechan las características del enrutador, incluyendo:

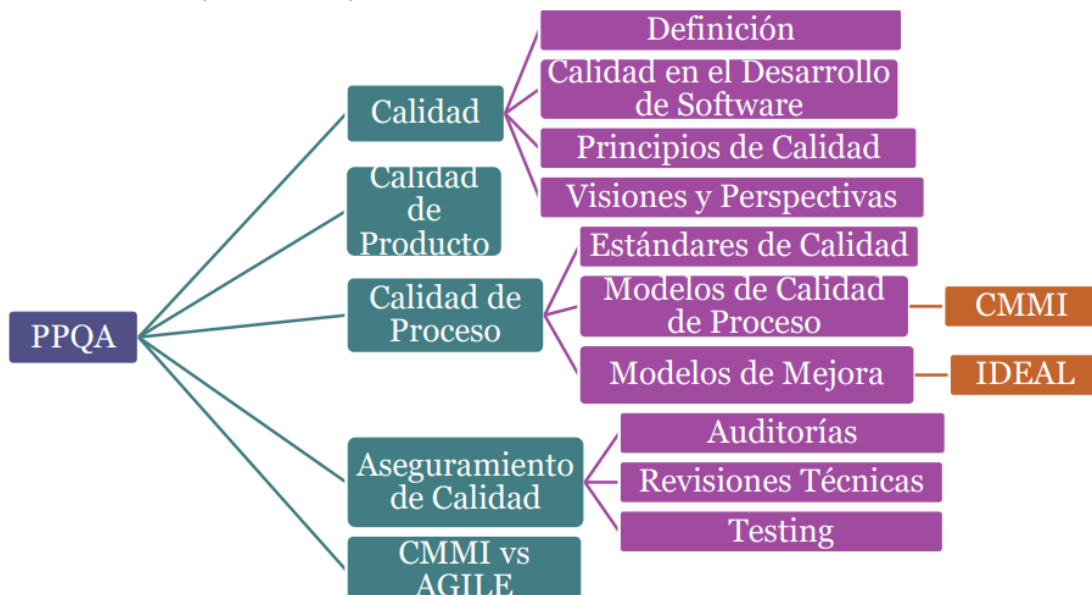
- Despliegues con Estrategia Rolling y Canary.
- Estrategia Recreate.
- Estrategia Personalizada.
- Despliegue Blue-Green con uso de rutas.
- Despliegues A/B y Canary con uso de rutas.
- Múltiples configuraciones de implementación.

UNIDAD N° 4

Conceptos generales sobre calidad

Pequeña anécdota sobre las instituciones

Aseguramiento de Calidad de Proceso y de Producto (PPQA): Cobertura



La calidad son todos los aspectos de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades manifestadas o implícitas. Sin embargo hay una parte de calidad relativa a las personas que es subjetivo a lo que ellos esperan del software.

Hay varios factores que pueden afectar al software:

1. Atraso en las entregas

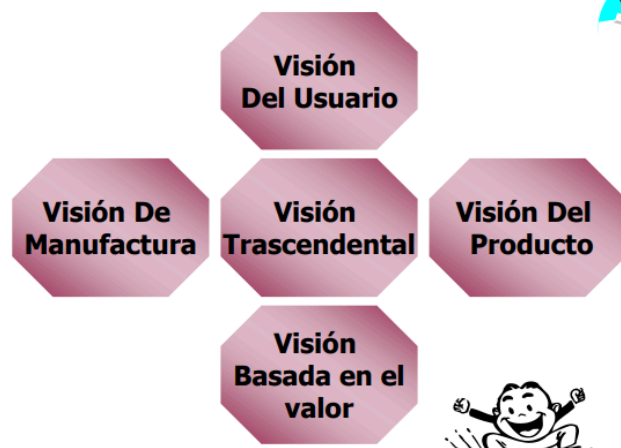


INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

2. Costos excesivos
3. Ambigüedad en los objetivos y requerimientos
4. Poco entendimiento del negocio o mercado
5. Trabajo fuera de hora

Formalmente se puede definir la **CALIDAD** como el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características simplistas esperadas del desarrollo de software profesional.



Visiones

1. **Visión del usuario:** Esto tiene que ver con las expectativas que tiene el usuario para con el producto y si lo satisface.
2. **Visión del producto:** Se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.
3. **Visión del proceso:** Es si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar
4. **Visión del valor:** Encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible al menor costo, generando mayor ganancia.
5. **Visión trascendental:** Esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar.

Aseguramiento de calidad

Es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad y en el proceso de fabricación, a la introducción de proceso de calidad. El espíritu del aseguramiento de calidad de software es actuar como medida preventiva a diferencia del testing que llega tarde. El equipo de QA debe ser independiente del equipo de desarrollo para que pueda tener una perspectiva objetiva del software. La planeación de calidad es el proceso de desarrollar un plan de calidad para un proyecto. El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valoran.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Principios

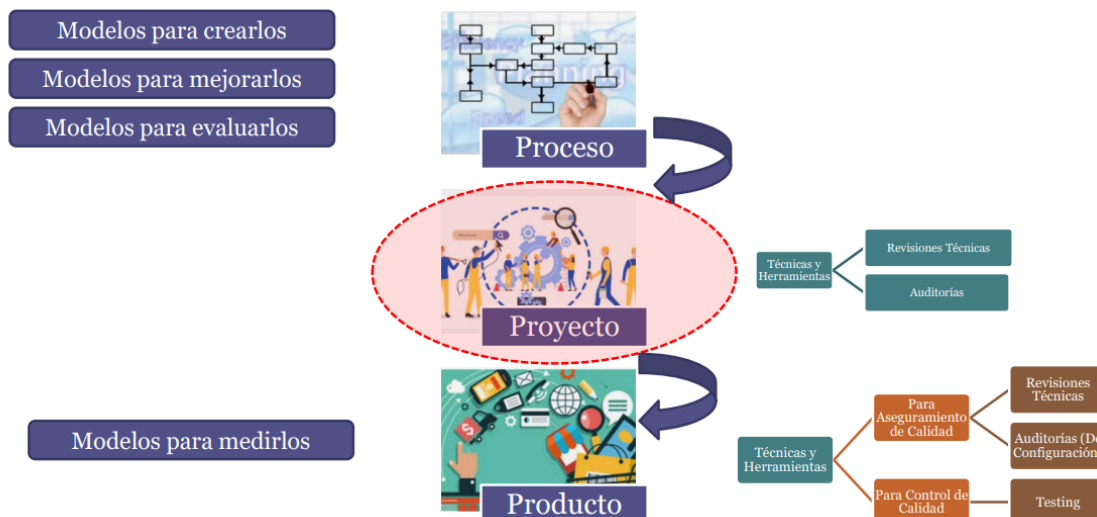
1. La calidad no se inyecta, se embebe desde el momento 0.
2. Es una responsabilidad de todos los involucrados en el proyecto.
3. Las personas son la clave para lograrlas.
4. Se necesita sponsor a nivel gerencial, pero se puede empezar por uno.
5. Se debe liderar con el ejemplo
6. No se puede controlar si no se mide, se necesitan métricas.
7. Simplicidad, se empieza por lo básico
8. Debe planificarse el aseguramiento de calidad
9. El aumento de las pruebas no aumenta la calidad.
10. Debe ser razonable para mi negocio.
11. Prevención mejor que corrección.

La gestión de la calidad

Tiene 3 intereses fundamentales, a nivel organización que establece estándares, a nivel proceso que verifica que los procesos planteados se cumplan y a nivel proyecto que se ocupa de establecer un plan de calidad.

- Calidad en Uso (ISO 25010: 2011)
- Calidad de producto (ISO 25010:2011)
- Calidad de Datos (ISO 25012:2008)

Calidad en el Software

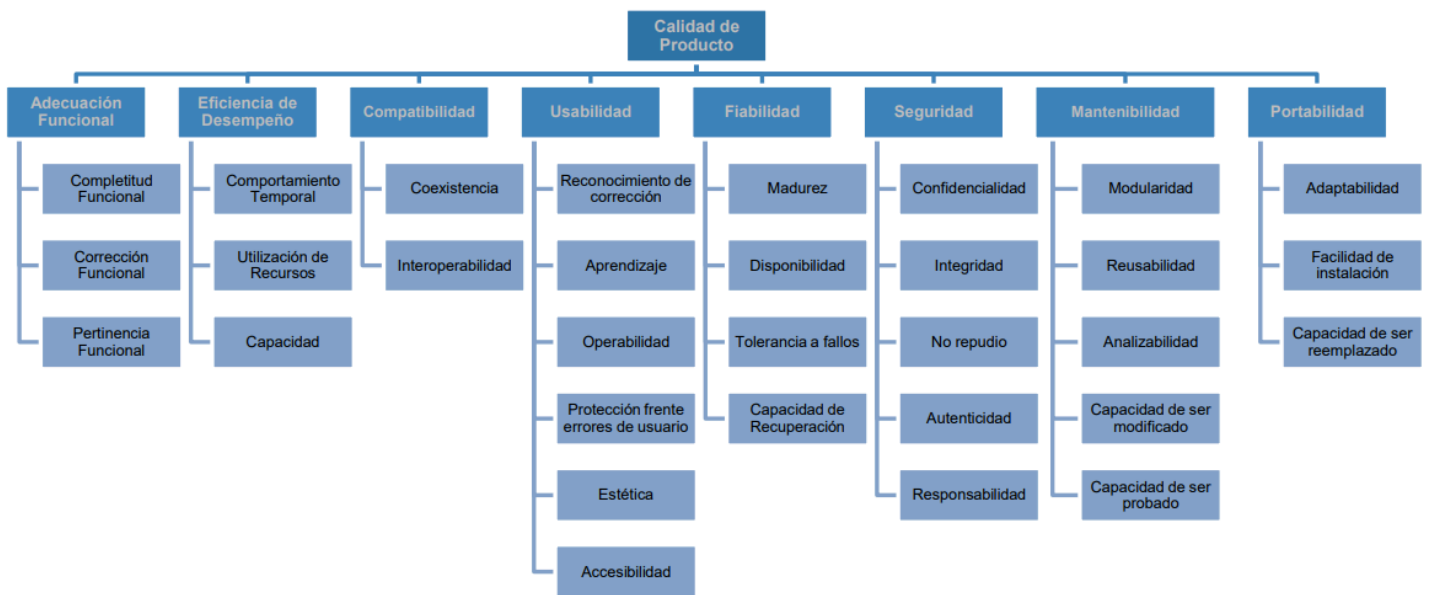


Espero no se hayan olvidado el sublime cuadro que debíamos aprender en DSI para los requerimientos no funcionales y todo eso que se olvidaron.

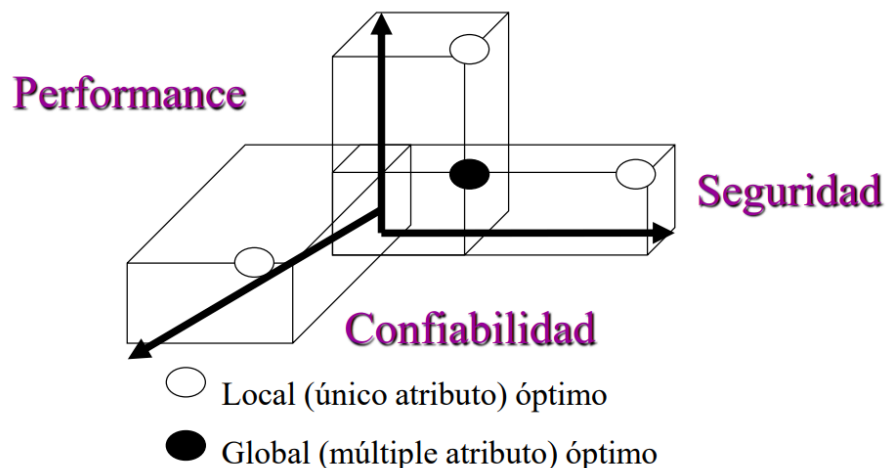


INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Modelo de Barbacci / SEI



Administración de la Calidad del Software

Concerniente con asegurar que se alcancen los niveles requeridos de calidad para el producto de software. Implica la definición de estándares y procesos de calidad apropiados y asegurar que los mismos sean respetados. Debería ayudar a desarrollar una “cultura de calidad” donde la calidad es vista como una responsabilidad de todos y cada uno.

Reporte del grupo de aseguramiento de calidad (GAC)

- No debería reportar al gerente de proyecto
- No debería haber mas de una posición entre la gerencia de primer nivel y el GAC
- Cuando sea posible, el GAC debería reportar a alguien altamente interesado en la calidad de nuestro software



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Debe estar separados del desarrollo para no estar condicionados

Actividades asociadas

- Aseguramiento de calidad
 - Establecer estándares y procedimientos organizacionales de calidad
- Planificación de Calidad
 - Selecciona los procedimientos y estándares aplicables para un proyecto en particular y los modifica de ser necesario
 - Un plan de calidad define los productos de calidad deseados y como serán evaluados , y define los atributos de calidad más significativos.
 - El plan de calidad debería definir el proceso de evaluación de la calidad.
 - Define cuáles estándares organizacionales deberían ser aplicados , como así también si es necesario utilizar nuevos estándares.
- Control de Calidad
 - Asegura que los procedimientos y estándares son respetados.
 - Esto implica el control del proceso de desarrollo para asegurar que se siguen los estándares y procedimientos.
 - Existen dos enfoques para el control de calidad:
 - Evaluaciones de Software Automáticas y
 - mediciones.
- Calidad de Procesos en la Práctica
 - Definir los procesos como estándares que podemos monitorear y administrar
 - Reportar el proceso a la administrador de proyectos y al responsable del software
- Estándares y Aseguramiento de Calidad
 - Los estándares son la clave para la administración de calidad efectiva.
 - Pueden ser estándares internacionales, nacionales, organizacionales o de proyecto.
 - Los Estándares de Producto definen las características que todos componentes deberían exhibir, ej. estilos de programación común.
 - Los Estándares de Procesos definen cómo deberían ser implementados los procesos de software.
- Revisiones de Calidad
 - Este es el principal método de validación de la calidad de un proceso o un producto.
 - Un grupo examina parte de un proceso o producto y su documentación para encontrar potenciales problemas.
 - Existen diferentes tipos de revisiones con diferentes objetivos
 - Inspecciones para remoción de defectos (producto)
 - Revisiones para evaluación de progreso (producto y proceso)
 - Revisiones de Calidad (producto y estándares).



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Principales Modelos de Calidad existentes

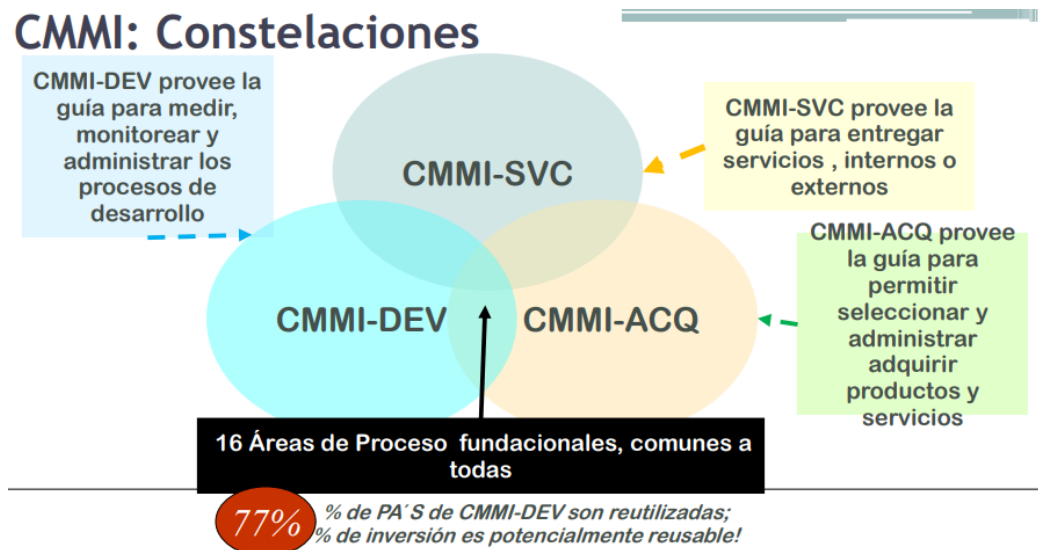
Introducción

Como nuestra brillante cátedra de ISW decidió abiertamente no dar este tema en ninguna clase como en general ninguno de la unidad 4 no me queda otra que ver las filmas con fotos del botón bumbula, el resumen de otros y armar una idea cohesiva como para safar. Lamento que no pueda hacer énfasis en ningún tema de esta unidad 4 pero repito, no se dio, ni se va a dar en mi año :D.

CMMI

CMMI (**C**apability **M**aturity **M**odel **I**ntegration) es un modelo de mejora de los procesos que fue creado en el lejano 1984. Es tan atrás en el tiempo que la Meles aún no había comenzado a dar clases en la UTN. Actualmente creo que se utiliza, la bibliografía es confusa porque dice que la versión vigente 1.3 es del 2010 pero que está la 3.0 del 2023.

Este modelo no es una “Norma” como la ISO y no emite certificados por lo que únicamente evalúa mediante profesionales reconocidos por el SEI como Lead Appraisers. Sus prácticas se basan en el conocido mapa de las 3 constelaciones que son parte de un proceso de mejora continua mediante la evaluación continua.



Nuestro modelito consiste en la utilización de 5 niveles o etapas por las cuales pasa un proceso para garantizar la calidad de lo que estamos produciendo.

1. Nivel 1

- Identificar el alcance del trabajo
- Realizar el trabajo

2. Nivel 2

- Política Organizacional para planear y ejecutar
- Requerimientos, objetivos o planes
- Recursos adecuados
- Asignar responsabilidad y autoridad



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- e. Capacitar a las personas
- f. Administración de Configuración para productos de trabajo elegidos
- g. Identificar y participar involucrados
- h. Monitorear y controlar el plan y tomar acciones correctivas si es necesario
- i. Objetivamente monitorear adherencia a lo procesos y QA de productos y/o servicios
- j. Revisar y resolver aspectos con el nivel de administración más alto

3. Nivel 3

- a. Mantener un proceso definido
- b. Medir la performance del proceso

4. Nivel 4

- a. Establecer y mantener objetivos cuantitativos para el proceso
- b. Estabilizar la performance para uno o más subprocesos para determinar su habilidad para alcanzar logros

5. Nivel 5

- a. Asegurar mejora continua para dar soporte a los objetivos
- b. Identificar y corregir causa raíz de los defectos

Gráficamente sería:



Niveles de madurez por etapas





INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Diferencias contra métodos ágiles

CMMI	MÉTODOS ÁGILES
→ Medir y mejorar el proceso [Mejores Procesos ↓ Mejor Producto]	→ Respuestas a clientes → Mínima sobrecarga → Refinamiento de Requerimientos - Metáforas - Casos de negocio
→ Características de las personas - Disciplinados - Siguen reglas - Aversión al riesgo	→ Características de las personas - Comfortable - Creative - Risk Takers
→ Comunicación - Organizacional - Macro	→ Comunicación - Person to Person - Micro
→ Gestión de Conocimiento - Activos de proceso	→ Gestión de Conocimiento - Personas

CMMI	MÉTODOS ÁGILES
→ Mejora Organizacionalmente - Uniformidad - Nivelación	→ Mejora en el Proyecto - Tradición Oral - Innovación
→ Capacidad/Madurez - Éxito por Predictibilidad	→ Capacidad/Madurez - Éxito por darse cuenta de oportunidades
→ Cuerpo de Conocimiento - Cruzando dimensiones - Estandarizado	→ Cuerpo de Conocimiento - Personal - Evolucionando - Temporal
→ Reglas de Atajo - Desalentadas	→ Reglas de Atajo - Alentadas



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

CMMI	MÉTODOS ÁGILES
→ Comités	→ Individuos
→ Confianza del Cliente	→ Confianza del Cliente
- En la Infraestructura del Proceso	- Sw funcionando, Participantes
→ Cargado al frente	→ Conducido por Pruebas
- Mover a la derecha	- Mover a la izquierda
→ Alcance de la vista [Involucrado, Producto]	→ Alcance de la vista [Involucrado, Producto]
- Amplio	- Pequeño
- Inclusivo	- Focalizado
- Organizacional	
→ Nivel de Discusión	→ Nivel de Discusión
- Palabras	- Trabajo en mano
- Definiciones	
- Duradero	
- Exhaustivo	

CMMI	MÉTODOS ÁGILES
→ Descriptivo	→ Prescriptivo
→ Cuantitativo	→ Cualitativo
- Número científicos y duros	- Conocimiento tácito
→ Universalidad	→ Situacional
→ Actividades	→ Producto
→ Estratégico	→ Táctico
→ "¿Cómo lo llamaremos?"	→ "Sólo hazlo!"
→ Gestión de Riesgos	→ Gestión de Riesgos
- Proactiva	- Reactiva

Similitudes

- **Meta:** Organizaciones de alto desempeño
- Ambas planean
- Ambas son CMMs (Consultant Money Makers)
- Ambas tienen reglas [Reglas = Requerimientos del proceso]
- La violación tiene serias repercusiones
- 'SEPG' (Grupo de proceso de ingeniería de software) & 'Política de Proceso'
- Ninguno es completo
- No nuevas ideas
- Basadas en la experiencia
- Ninguno es aplicable a "cualquier proyecto"

Si se están preguntando por el temario que es la ISO - SPICE, por lo que dice internet es lo mismo y son ese bello cuadrado que está en gestión de calidad previamente visto en DSI.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Wiki:

El **ISO/IEC 15504**, también conocido como *Software Process Improvement Capability Determination*, abreviado **SPICE**

Nivel 0: Incompleto

Nivel 1: Realizado

Nivel 2: Gestionado

Nivel 3: Establecido

Nivel 4: Predecible

Nivel 5: En optimización

Básicamente es lo mismo que vimos hasta ahora con CMMI nomas que las ISO si son un estándar que llevan consigo certificaciones.

Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad

Introducción

(Otro tema que no vi en clases)

Los objetivos de las SQA (Software Quality Assurance) son el control apropiado del software asegurando el cumplimiento de los estándares y procedimientos que hacen al proceso. La idea es lograr que disminuyan la cantidad de defectos en el producto o servicio informando a la gerencia o encargado para poder solucionarlos. En 5to año vemos las de seguridad y tienen una idea parecida que es la identificación de riesgos o problemas que comprometan a la confidencialidad, integridad o disponibilidad del nuestro software.

Existen diversos tipos de auditorías, lo más recomendable es que un externo al proceso y equipo es quien realice dicha auditoría para obtener una mirada objetiva de lo que está sucediendo. Muchas auditorías se plantean de manera periódica para tener un control de las actividades que se realizan en una organización, por otro lado, tenemos aquellas que se hacen después de una implementación o nuevo producto para medir el rendimiento. Para aquellos que se están preguntando, si en general se usa para mostrarle a la gerencia que estas haciendo “X” cosa bien y que deberían estar muy orgulloso de vos (*palmaditas en la espalda). También se usan como forma de mostrarle a terceros la confiabilidad de tu empresa, como si seguis ciertos estándares estas a la vanguardia de la calidad.

En general asocian este tema a la *Calidad™* que vemos en la materia porque es la forma que reafirmamos la existencia de la misma. Tener un control y retroalimentación es la manera más eficaz para corregir nuestras formas y garantizar la sostenibilidad de nuestro software.

Roles en una auditoría:

- **Auditado:** Participa en la auditoría y es quien propone la fecha de la auditoría, entrega evidencia, contesta las dudas del auditor, propone planes de acción. Generalmente es el líder de proyecto.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- **Auditor:** Puede ser una persona y deben estar fuera del proyecto que se está auditando. Puede ser el grupo de aseguramiento de calidad, que es el grupo que le da soporte a estos tipos.
- **Gerente de SQA:** Es quien prepara el plan de auditoría, calcula su costo, asigna recursos, resuelve las no conformidades entre auditor y auditado.

Tipos de Auditorías

Auditoria de proyecto

Responsable de ver que el proyecto se haya ejecutado con el proceso que se definió. Esto bajo la premisa de que en un proyecto se debe realizar lo que define el proceso, obviamente adaptado a lo que sirve en ese contexto del proyecto particular. Lo que se hace es tomar evidencias de lo que se está haciendo y contrastarlo con lo documentado en la ERS. arquitectura, diseño y etc.

En las auditorías de proyecto se lleva a cabo lo establecido en el “Plan de Aseguramiento de Calidad de Software” o “PACS” para los compinches (Creo que es la palabra que usan los jóvenes). Este plan debe indicar a las personas responsables que inspecciones, revisiones, documentación y pruebas que deberían realizarse.

Auditoria de configuración funcional

Valida que el proyecto cumpla con los requerimientos, la auditoría funcional compara el software que se ha construido con los requerimientos de software especificados en la ERS. El propósito de la auditoría funcional es asegurar que el código implementa solo y completamente los requerimientos y las capacidades funcionales descritas en la ERS. El responsable de QA deberá validar si la matriz de rastreabilidad está actualizada.

Auditoría de configuración física

Valida que el ítem de configuración, cumpla con la documentación técnica que lo describe. Compara el código con la documentación de soporte para evitar inconsistencias en nuestro versionado ya que es posible que una actualización o parche pasase sin registrar debidamente. En el PACS es donde se dice quien es el visera/vigilante/policía que realiza esta auditoría.

Auditoria externas de aseguramiento de calidad

Son auditorías realizadas por un personal externo a la organización, a quienes se encargan de realizar las auditorías de calidad. Esto permite evaluar a los miembros de QA que después evalúan al resto de la empresa.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Proceso de Auditorías

Procesos de auditorías

1. Planificación y preparación

- a. **No son sorpresa.** Las Entidades solicitan la auditoría y junto con el auditor definen la fecha, el alcance y objetivo, acordando los participantes de esta.
- b. Se debe definir métricas de auditoría, nunca se puede auditar todo junto y mucho menos debe estar desconectado de un propósito.
- c. En general las empresas entran en un estado de **“alerta”** donde todos se visten bien para la ocasión, se bañan (sobre todo programadores), se controla más los procesos y en general se actúa de manera correcta.

2. Ejecución

- a. El auditor escucha lo que la gente dice que hace y luego ve la documentación, pide evidencia. Te caen los MIB (Men In Black para los jóvenes) y te registran hasta la ropa interior, en general ese día lleva los sin agujeros.

3. Análisis y reporte de resultados

- a. El auditor realiza el reporte de resultados y el auditado analiza la respuesta, puede o no estar de acuerdo con algunas prácticas y se deja asentado el documento final. Se evalúan los resultados se hace una reunión de cierre y se hace entrega del reporte final

4. Seguimiento y Retroalimentación

- a. Se analiza la desviación para luego preparar un plan de acción que se envía al auditor para que lo revise y apruebe. En función del acuerdo entre las partes puede que el auditor haga seguimiento de las desviaciones que encontró hasta que considere que han sido resultante

Resultados de una auditoría

1. Buenas prácticas: Cuando es algo superado o definido que tenemos que hacer

- a. Ha establecido un sistema efectivo
- b. Ha desarrollado un alto grado de conocimiento y cooperación interna
- c. Ha adoptado una práctica superior a cualquier otra que se haya visto

2. Desviaciones: Cualquier cosa que no se hizo o que no se cumplió el proceso establecido

- a. Cualquier desviación que resulta en la disconformidad de un producto respecto de sus requerimientos
- b. Falta de control para satisfacer los requerimientos
- c. Cualquier desviación al proceso definido o a los requerimientos documentados que cause incertidumbre sobre la calidad del producto, las prácticas o las actividades.

3. Observaciones: Prácticas dudosas o con riesgos asociados.

- a. Opinión acerca de una condición incumplida
- b. Práctica que debe mejorarse



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- c. Condición que puede resultar en una futura desviación.

Métricas de auditoría

- Esfuerzo por auditoría
- Cantidad de desviaciones
- Duración de auditoría

Herramientas de auditorías

- Checklist
 - Como si fuera las compras, anotas todas las cosas que se deberían cumplir. Fecha de la auditoría
 - **Contiene:**
 - Lista de auditados (identificando el rol)
 - Nombre del auditor
 - Nombre del proyecto
 - Fase actual del proyecto (si aplica)
 - Objetivo y alcance de la auditoría
 - Lista de preguntas
- Muestreo
 - Se analizan las salidas de los procesos mediante una recolección de datos al azar para obtener una media de resultados.
- Revisión de registros
 - En caso de poderse se puede hacer sobre el total de muestra y revisar todo lo que ocurre en el proceso.
- Herramientas automatizadas
 - Para evitar que se haga papel, como en DSI, el resto del mundo encontró unos aparatos “electrónicos” (algunos dirán digitales) que permiten procesar más información y hacerlo de manera organizada. Ojalá algún día la tecnología alcance a la UTN.

Calidad de Producto:

Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.

(¿Creo que este tema ya fue visto en calidad?)



Técnicas y Herramientas para la realización de revisiones técnicas del software.

Técnicas de revisión técnica del software

1. Revisiones por pares (Peer Review):

- Consiste en que otro desarrollador o miembro del equipo revisa el código o artefactos de software.
- Se centra en identificar errores y sugerir mejoras.
- Puede ser formal o informal.
- Ejemplo: revisión de código antes de hacer un merge a la rama principal.

2. Revisiones formales:

- Reuniones estructuradas donde un moderador guía el proceso.
- Involucra al autor del código, revisores, y posibles stakeholders.
- Documentación detallada del proceso y los resultados.

3. Walkthrough:

- El autor guía al equipo a través del código o diseño.
- Se utiliza para discutir el enfoque y recibir retroalimentación sin buscar específicamente errores.

4. Inspección de software:

- Proceso formal con roles asignados (moderador, autor, lector, etc.).
- Incluye la identificación de defectos, causas y soluciones.
- Puede implicar métricas como densidad de defectos.

5. Pruebas estáticas:

- Revisión de código sin ejecutarlo, para encontrar problemas como errores de sintaxis, fallos de lógica o incumplimientos de estándares.

6. Análisis de calidad del código:

- Evaluar el código contra métricas específicas como mantenibilidad, complejidad, acoplamiento y cohesión.

7. Revisión de diseño:

- Se enfoca en verificar si el diseño cumple con los requisitos funcionales y no funcionales.
- Busca mejorar la eficiencia, escalabilidad y mantenibilidad.

Herramientas para revisiones técnicas

1. Herramientas de gestión de versiones:

- **GitHub, GitLab, Bitbucket:** Integran revisiones de código en el flujo de trabajo (pull requests con comentarios).

2. Herramientas para análisis estático de código:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- **SonarQube:** Identifica problemas de calidad del código y vulnerabilidades de seguridad.
 - **ESLint, TSLint:** Análisis para JavaScript/TypeScript.
 - **Checkstyle:** Para Java.
 - **Pylint:** Para Python.
3. **Integración continua (CI):**
- **Jenkins, CircleCI, Travis CI:** Automatizan pruebas y análisis de calidad durante el desarrollo.
4. **Sistemas de seguimiento de problemas:**
- **Jira, Trello, Azure DevOps:** Documentan problemas encontrados durante revisiones técnicas.
5. **Herramientas colaborativas para revisiones:**
- **Crucible:** Herramienta específica para revisiones de código.
 - **Review Board:** Plataforma para revisiones colaborativas de código.
 - **Phabricator:** Herramienta de gestión y revisión de código.
6. **Plataformas de gestión de calidad:**
- **TestRail, PractiTest:** Usadas para documentar y rastrear revisiones relacionadas con pruebas.
7. **IDEs con herramientas de revisión:**
- **IntelliJ IDEA, Visual Studio, Eclipse:** Integran análisis estático y sugerencias de mejora.
8. **Modeladores de diseño y arquitectura:**
- **Enterprise Architect, Lucidchart, Visual Paradigm:** Ayudan en la revisión de diagramas de diseño y arquitectura.

Testing en ambientes Ágiles.

Introducción a molestar a los desarrolladores

Como notarán hay unidades no vistas en clases y tampoco en resúmenes que deje a la merced del lector. Sin embargo este tema si fue dado por la matrona Meles, es más le dio énfasis a esto cosa de que no lleguemos con los demás temas y lo leamos nosotros como los humildes corderos que somos. Este tema es testing, osea el proceso de buscar errores en las demás personas sin haber hecho nada, básicamente ser mujer (tranquilos snowflake se llama humor).

Siendo serios, el testing es el **proceso mediante el cual se somete a un software o un componente de software a condiciones específicas con el fin de determinar y demostrar si el mismo es válido o no en función de los requerimientos especificados**. Es una actividad destructiva 😊 cuyo objetivo es encontrar defectos, cuya presencia es asumida de antemano en el código. Es una actividad costosa, por lo que es necesario lograr un nivel de cobertura óptima teniendo en cuenta el costo-beneficio.

Error Vs Efecto



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Error

VS



Defecto

- El error es una falla o equivocación encontrada en un producto de software en un momento dado que fue corregido sin salir de una misma etapa (actualidad).
- Un defecto es un error que se trasladó a etapas posteriores de su origen (1943-actualidad).

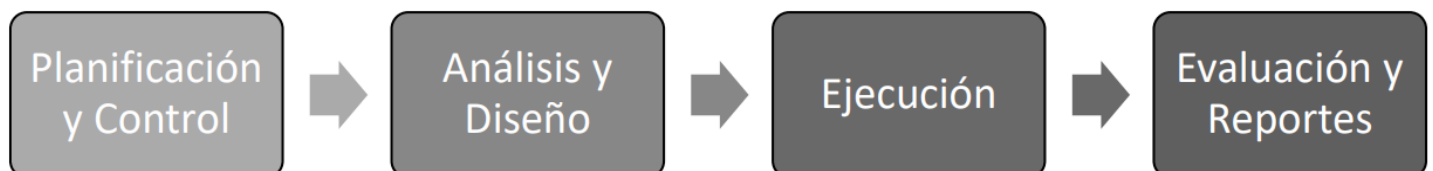
Severidad

- 1 – Bloqueante
- 2 – Crítico
- 3 – Mayor
- 4 – Menor
- 5 - Cosmético

Prioridad

- 1 – Urgencia
- 2 – Alta
- 3 – Media
- 4 – Baja

En las filminas tenemos un conjunto de cosas como el proceso de testing:

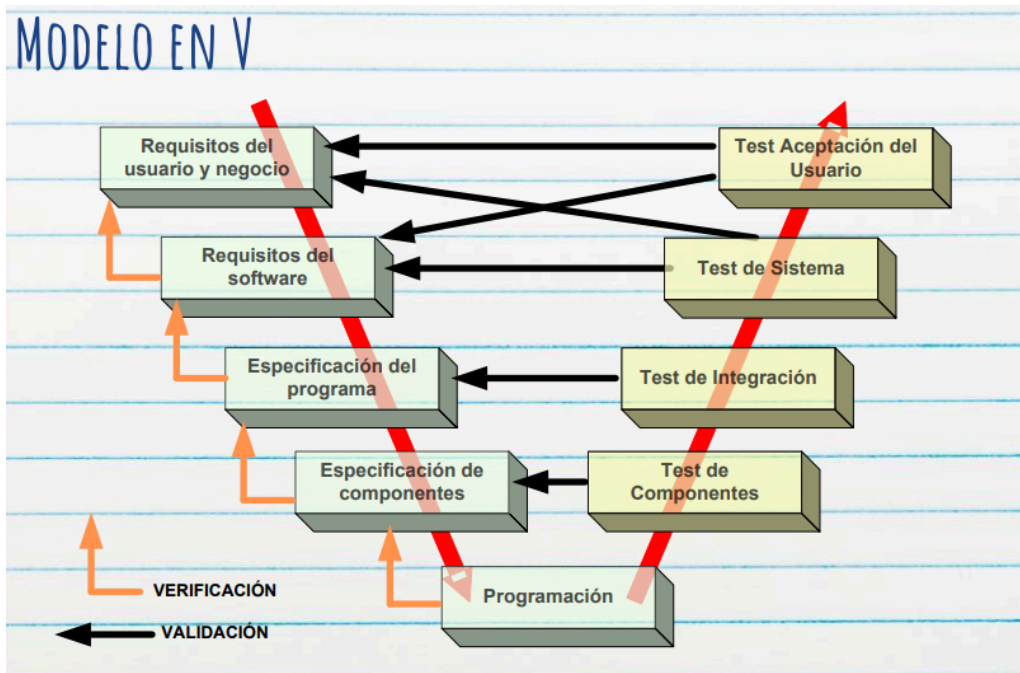


Proceso del Testing Planificación y Control



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Planificación y control

La planificación de las pruebas es la actividad de verificar que se entiende las metas y los objetivos del cliente por sus partes interesadas. También entender qué pruebas vamos a estar realizando con un plan de testing.

Construcción del Test Plan:

- Estrategia de Testing
- Riesgos y Objetivos del Testing
- Recursos
- Criterio de Aceptación

Controlar:

- Revisar los resultados del testing
- Test coverage y criterio de aceptación
- Tomar decisiones

Análisis y Diseño

Revisión de la base de pruebas

- Verificación de las especificaciones para el software bajo pruebas
- Evaluar la testeabilidad de los requerimientos y el sistema
- Identificar los datos necesarios
- Diseño y priorización de los casos de las pruebas
- Diseño del entorno de prueba



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Ejecución

- Desarrollar y dar prioridad a nuestros casos de prueba
- Crear los datos de prueba
- Automatizar lo que sea necesario
- Creación de conjuntos de pruebas de los casos de prueba para la ejecución de la prueba eficientemente.
- Implementar y verificar el ambiente.
- Ejecutar los casos de prueba
- Registrar el resultado de la ejecución de pruebas y registrar la identidad y las versiones del software en las herramientas de pruebas.
- Comparar los resultados reales con los resultados esperados.

Evaluación

- Evaluar los criterios de Aceptación
- Reporte de los resultados de las pruebas para los stakeholders.
- Recolección de la información de las actividades de prueba completadas para consolidar.
- Verificación de los entregables y que los defectos hayan sido corregidos.
- Evaluación de cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.

Principios del Testing

- El testing es una actividad destructiva que encuentra defectos cuya presencia se asume.
- Se testea con una actitud negativa tratando de demostrar que algo es incorrecto
- El testing exitoso es aquel que encuentra defectos
- El costo del testing está entre el 30 al 50 % del valor del producto
- El testing pone en evidencia defectos, pero no agrega calidad ni garantiza que el producto no tiene errores.
- Un desarrollo exitoso puede llevar un testing no exitoso
- El testing es necesario siempre
- Se parte de la suposición de que siempre tendrá defectos por encontrar, ya que es una característica inherente de los productos desarrollados por equipos de personas.
- Puede empezar antes de la codificación porque necesita de los requerimientos para armar los casos de prueba
- El testing NO asegura que se tenga un producto de calidad, ni que el proceso por el que se desarrolló sea de calidad
- Agrupamiento de defectos: los defectos suelen agruparse en un conjunto limitado de módulos o áreas, las pruebas deben ser en conjuntos acotados.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Niveles de Testing

Pruebas Unitarias

En programación, una prueba unitaria, PU, o test unitario (del inglés: unit test) es una forma efectiva de comprobar el correcto funcionamiento de las unidades individuales más pequeñas. El objetivo de las pruebas unitarias es aislar cada parte del programa y mostrar cómo probarlo por separado. Las mismas fomentan los cambios para mejorar la estructura (refactorizar), simplificar la integración con el resto del aplicativo, mejorar la calidad del código, también sirven como documentación de cómo utilizarlo y en general modularizar los errores en partes más pequeñas de detectar.

En estas pruebas verificamos que los nombres que se usan son correctos (y siguen lo pactado en el diseño), que recupera los valores correctamente, si llega al mismo estado final desde un estado en concreto y se comporta siempre de la misma manera. Algo importante a destacar es que estas pruebas se pueden automatizar para lograr resultados concretos y poder verificar los cambios surtan efecto de manera inmediata.

Como anécdota recuerdo en una materia que se nos habló de la arquitectura orientada al testing, que es una forma de desarrollar código pensando íntegramente en la automatización de las pruebas.

Pruebas Integración

Un “chiste” que puso la meles



▶ Efecto de sonido risa de ardilla

Las pruebas de integración evalúan los componentes de una aplicación en un nivel más amplio que las pruebas unitarias. Las pruebas unitarias se usan para probar componentes de software aislados, como métodos de clase individuales. Las pruebas de integración confirman que dos o más componentes de una aplicación funcionan juntos para generar un resultado esperado, lo que posiblemente incluya a todos los componentes necesarios para procesar por completo una solicitud.

Estas pruebas más amplias se usan para probar la infraestructura de la aplicación y todo el marco, lo que a menudo incluye los siguientes componentes:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

1. Base de datos
2. Sistema de archivos
3. Dispositivos de red
4. Canalización de solicitud-respuesta

En general es la respuesta de nuestro software como un todo y en general detonan las vulnerabilidades o fallas entre capas o componentes.

Pruebas de Versión o Sistema

Te dejo que uses la imaginación... Exacto se prueba todos los componentes de una versión, apreciación mía, van de la mano de con la creación de las líneas base o versionado. Quiere decir que estas pruebas son largas y exhaustivas ya que de pasar el software se encuentra en una versión “estable” y puede ser presentado.

Pruebas de Aceptación Usuario

Este es el más común en las retro o review, es que el PO o cliente revise que esos son los requerimientos que pide y se los cumple como corresponde. En general esto puede hacer que te den los Story Points que desees para sumar a tu velocidad o si te quedas patitas en la calle.

Estrategias de Caja Negra

Las pruebas de caja negra, también conocidas como pruebas funcionales, son un tipo de pruebas de software basadas en el grado de conocimiento de los procesos - **en este caso nulo** - que se dispone al momento de realizar un conjunto de pruebas. No quiere decir que no entendamos que hace nuestro software, pero si es una prueba que se hace desde el desconocimiento técnico.

La generación de un caso de prueba puede realizarse a partir de la estructura de un Documento ERS (Especificaciones de Requisitos de Software) para cada una de las funcionalidades que un software deberá cumplir. Puesto que cada caso de prueba de caja negra estará relacionado con cada funcionalidad del software, también son conocidas como pruebas funcionales.

- Basado en especificaciones
 - Partición de Equivalencias
 - Análisis de valores límites
- Basados en la experiencia
 - Adivinanza de Defectos
 - Testing Exploratorio

Basado en especificaciones: Partición de Equivalencias

Dos Pasos

1. Identificar las clases de equivalencia (Válidas y no Válidas)



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- a. Rango de valores continuos
- b. Valores Discretos
- c. Selección simple
- d. Selección múltiple

2. Identificar los casos de prueba

Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas	
CLASES DE EQUIVALENCIA DE ENTRADA				
Usuario logueado	1	Usuario logueado con permisos de RE	2	Usuario no logueado
			3	Usuario sin permisos de RE
Fecha y hora de inicio	4	Fecha y hora con formato DD/MM/YYYY HH:MM y menor a la fecha actual + 5 días	5	Fecha y hora con formato inválido
			6	Fecha y hora mayor a la fecha actual + 5 días
			41	Fecha inválida
			7	No ingresa valor
Temperatura	8	Número entero >= 30 y <= 50	9	Número no entero
			10	Número entero menor a 30 o mayor a 50
			11	Otro valor
			12	No ingresa valor

CLASES DE EQUIVALENCIA DE SALIDA				
Inicio de Templado programado	29	Mensaje de fecha y hora de inicio y fecha y hora de fin programadas	30	Mensaje de error de Usuario no logueado
			31	Mensaje de error de usuario sin permisos
			32	Mensaje de error de fecha y hora de inicio incorrecta
			33	Mensaje de error de temperatura incorrecta
			34	Mensaje de error de duración incorrecta
			35	Mensaje de error de aspecto no seleccionado
			36	Mensaje de error de textura no seleccionada
			37	Mensaje de error indicando que la máquina no está en reposo
			38	Mensaje de error indicando que la máquina está apagada
			39	Mensaje de error indicando que la máquina no está cargada
			40	Mensaje indicando error en cálculo de fecha y hora de finalización.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Este es un ejemplo de la guía donde vemos un ejercicio, para simplificar las clases equivalencia tienen una entrada determinada para una salida planteada. Quiere decir que si tenemos un campo de usuario logueado con permisos vamos a tener una sola entrada y 2 posibilidades (No logueado, Sin permisos). En la salida ponemos todas las posibilidades que tenemos en esta user story. Finalmente cargamos su caso de prueba.

Id del Caso de Prueba	Clases de eq	Prioridad (Alta, Media, Baja)	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
1	1,4,8,13,18,21,24,29	Alta	Iniciar Templado con éxito	El usuario "WillyW" tiene sesión iniciada y permisos de Responsable de Elaboración El aspecto "Opaco" se encuentra cargado y disponible La textura "Suave" se encuentra cargada y disponible La fecha y hora actuales son 31/12/2018 23:50:00 La máquina está encendida, en reposo y cargada.	1. El RE selecciona la opción "Iniciar Templado" 2. El RE ingresa fecha de inicio 05/01/2019 23:49:00 3. El RE ingresa duración 120 minutos 4. El RE selecciona aspecto "Opaco" 5. El RE selecciona textura "Suave" 6. El RE ingresa temperatura 50 7. El RE selecciona la opción "Confirmar"	7. El sistema muestra la fecha y hora programada de inicio 05/01/2019 23:49 y la fecha y hora de fin calculada 06/01/2019 01:49

En la primera columna ponemos el caso de prueba y establecemos un ID único e irrepetible. En las clases de Equivalencia establecemos con numeración a que escenario hacemos referencia, en este caso no puse todas las clases pero tendrían esos números que ven. Le asignamos una prioridad, un nombre, sus precondiciones que asumimos como realidad y escapan al caso de uso, que pasos tenemos que seguir y el resultado (una clase de salida). Destaquemos que en este caso es únicamente el camino feliz de la situación donde se cumplen todas las clases de equivalencia, la ausencia de una como entrada implicaría que debe dar un mensaje de error.

Basado en especificaciones: Análisis de valores límites

Es una variante de la partición de equivalencias, en vez de seleccionar cualquier elemento como representativo de una clase de equivalencia, se seleccionan los bordes de una clase.

El análisis del valor límite es un tipo de prueba funcional, se ocupa de verificar que cada función del software cumple los requisitos y especificaciones. En el caso de las pruebas de contorno, esta funcionalidad incluye el modo en que el software trata las distintas entradas.

Pros



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

- Mejor calidad del software
 - Con tantas cosas que verificar, algunos defectos pueden pasar desapercibidos. Las pruebas de contorno demuestran la funcionalidad de las áreas del software que tienen más probabilidades de contener errores, lo que conduce a una mejor construcción del software y, en última instancia, a una aplicación más fiable y estable.
- Mayor cobertura de las pruebas
 - Es muy útil en las pruebas de software porque ayuda a reducir el número de casos de prueba necesarios para una cobertura completa de las pruebas. El análisis de valores límite garantiza que los valores importantes y que cada valor pueda probarse más a fondo.
- Detección precoz (como yo) de defectos
 - Las pruebas de valor límite forman parte de un enfoque que da prioridad a la detección temprana de defectos. Detectar los errores en una fase temprana del proceso permite a los equipos de desarrollo ahorrar tiempo y dinero, sin mencionar el hecho de que es mucho más fácil poner remedio a los errores en las primeras fases del desarrollo.
- Eficiencia
 - La reducción de las entradas a todas menos las que tienen más probabilidades de causar problemas puede ahorrar mucho tiempo a los equipos de pruebas, tanto en la redacción como en la ejecución de los casos de prueba.

Contras

- Alcance limitado
 - Trabaja en los límites o bordes de las entradas de datos válidas. En general, ignora las entradas intermedias razonando que estarán bien si las entradas válidas de los bordes lo están. Sin embargo, no hay que descartar que algunos de estos valores que no se han comprobado puedan plantear problemas.
- Demasiado simplista
 - El análisis de límites consiste en simplificar las cosas. Aunque esto funciona para reducir los casos de prueba, el enfoque es menos adecuado para dominios muy complejos con múltiples límites, interacciones o dependencias. De hecho, puede tener dificultades para gestionar situaciones complejas, lo que significa que es necesario explorar otras técnicas para obtener una cobertura adecuada.
- Supuestos
 - Cualquier proceso que intente aumentar la eficacia corre el riesgo de pasar por alto errores concretos. Se centra en los límites de una zona de distribución. Para ello, debe hacer suposiciones sobre otros insumos que se encuentran a ambos lados de los valores límite. Los responsables de las pruebas deben encontrar un equilibrio entre eficacia y cobertura, lo que supone un ligero riesgo si sólo se utilizan las pruebas de límites.
- Basarse en especificaciones y requisitos precisos
 - La eficacia depende de la calidad y exactitud de las especificaciones y la documentación de requisitos. Cualquier error no comprobado en estos documentos puede repercutir en



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

las pruebas de valores límite y provocar errores específicos que no se comprueban ni descubren hasta las últimas fases críticas del desarrollo.

- Dependencia de las clases de equivalencia
 - Para que sea exhaustivo es necesario tener un buen conocimiento de las clases de equivalencia. Establecer estas clases con precisión requiere experiencia y cierta información sobre la aplicación.

Basados en la experiencia: Adivinanza de Defectos

La adivinación de errores es un método heurístico o informal de prueba de caja negra que se basa en la creatividad, el conocimiento y la visión del probador para generar casos de prueba que probablemente expongan errores o fallas en el sistema. La adivinación de errores no sigue ninguna regla, estándar o técnica predefinida, sino que utiliza el propio juicio e intuición del probador para anticipar dónde y cómo el sistema podría salir mal. Se puede aplicar en cualquier etapa del proceso de prueba, desde las pruebas unitarias hasta las pruebas del sistema, y se puede combinar con otros métodos, como el análisis de valores límite, la partición de equivalencia o la prueba de tabla de decisiones.

Por lo que yo entiendo es tocar por tocar buscando un error de manera genérica, creo que probablemente este término lo asumimos con el siguiente tipo de prueba. En plano consiste en descubrir error por tocar, azar y con la experiencia que hayamos tenido antes, tiene un enfoque no funcional además.

Basados en la experiencia: Testing Exploratorio

El exploratorio es un tipo de prueba que todo el mundo habla porque es lo que más se usa en los inútiles de mierda sin conocimiento. Es como que hasta un mono podría hacerlo por eso son tan populares en las mili pili infumables que conforman DSI-ISW. Se centra, como su nombre lo indica, en la exploración de la aplicación usando el sentido común. A diferencia del anterior, si documentamos, si documentamos, buscamos darle secuencia y sentido a todo lo que estamos probando. Igualmente se basa en la experiencia de los usuarios para poder entender qué está pasando y si haríamos cambios más allá de lo funcional como puede ser para cierto tipo de carga agregar un drag and drop (arrastrar y tirar). El propósito general es ser un pasito más de lo que logramos en la adivinanza.

Estrategias de Caja Blanca

Este tipo de pruebas es para gente racista.

Se basan en el análisis de la estructura interna del software o un componente del software, la brillante ingeniera Meles no pone mucho en esta parte. Si le sirve de ejemplo para las pruebas de seguridad las pruebas que son de caja blanca son aquellas que la persona que hace la tarea tiene conocimientos acerca de lo que ocurre en el software. Osea que no solo verifica la entrada y salida sino que además verifica que el paso a paso interno sea correcto no solo desde las ventanas de front convencionales.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Pueden existir diferentes elementos a los cuales debemos ejecutar cobertura, estas son:

- **Sentencia:** Son los elementos tangibles, ya sea la ejecución de una función o la muestra en pantalla de un mensaje o una pantalla en si.
- **Decisión:** Esto se aplica en cada condicional, si tenemos un IF tenemos que hacer esto, que consiste en probar que cada condicional pueda dar un TRUE o un FALSE, sin que necesariamente se prueban todas las condiciones dentro de este.
- **Condición:** Este deriva del anterior, se basa en los if nuevamente, tenemos que probar todas las condiciones dentro de cada condicional, no solamente si entra o no a sus resultados.
- **Decisión/Condición:** Este se da en casos particulares, donde tomamos como prima la teoría del libro que es independiente de la tecnología en los que se apliquen los casos de prueba, por ende

¿Cuánto Testing es Suficiente?

Esta es una pregunta con 2 grandes miradas, la primera es la de Tano Martinoli, que plantea:

“Si sos bueno programando lo haces a la primera bien y chau, el análisis no sirve de nada que tanto”
- Tano “La maquina del sexo” Martinoli -

Por otro lado tenemos la de mi ANALista Bosetti:

“No puedo Gena entrar a disc, estoy haciendo Testing todo el dia pero no avanzamos en nada”
- Jc Bosetti todo los dias -

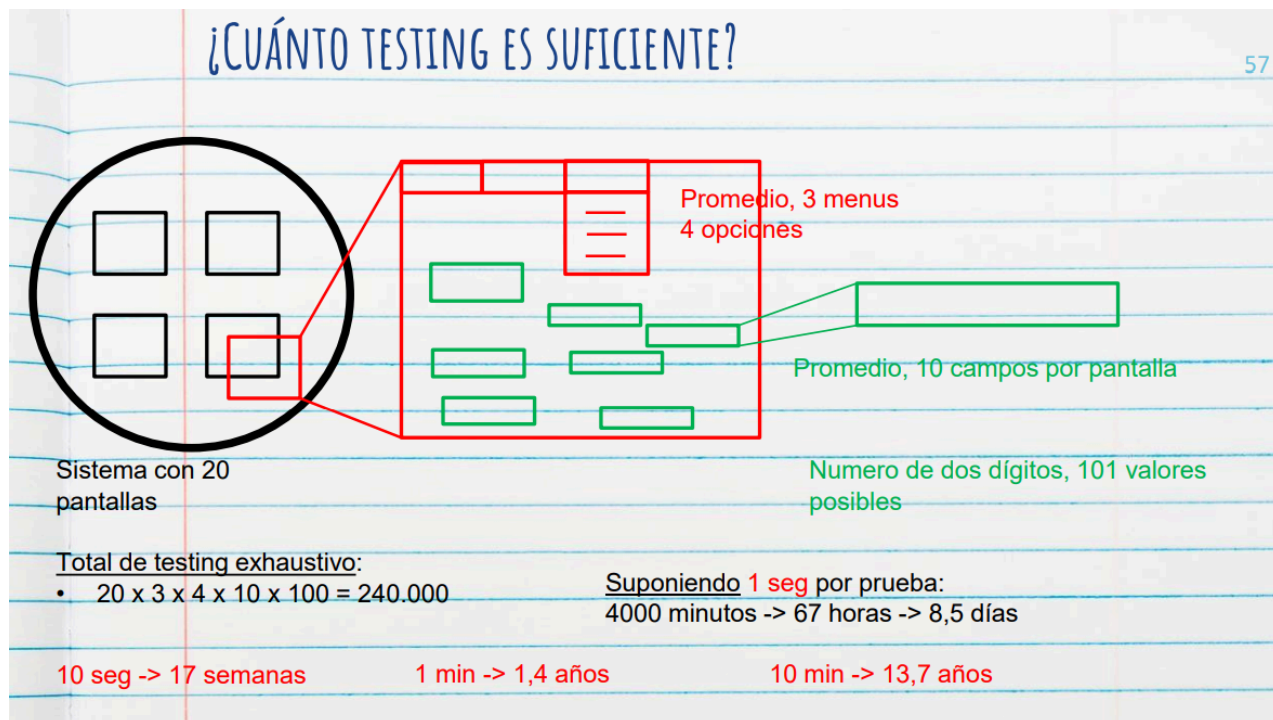
Ambas miradas representan los extremos poco sanos, por un lado alguien que solo hace desarrollo no entiende que debemos cumplir con una cierta “calidad” de software en el cual nuestro programa debe ser funcional y que además cumpla con lo pactado con el cliente. Por el otro lado tenemos a JC que tiene una obsesión poco sana de estar en 15mil reuniones y estar haciendo mil pruebas sin que tengan mucho goyete porque parece que las cosas no se hicieran.

La cátedra plantea lo mismo que te dice la Meles a cada pregunta de DSI; “depende”, que no es fallada esta aseveración. No debemos dejar a nuestro software a la buena de Dios pero tampoco podemos estar todo el dia de reunioncitas probando cosas, como vemos en la siguiente imagen de las filminas:



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO



Acá lo que se establece de parámetro es primero el nivel de confianza que tenemos en la aplicación, que riesgos corremos en caso de que no funcione y la capacidad (tiempo y complejidad) que tenemos para probarlo. Testear todo de manera exhaustiva es imposible (y caro), por ende tenemos que poner prioridades a que queremos verificar al 100% que funcione y que cosas se dejan para que el cliente las pruebe por nosotros. Es factible que si no podemos probar todo exhaustivamente solo lo hagamos con aquellas funcionalidades que involucran operaciones complejas (generalmente monetarias), mientras que cosas de soporte que se usan 1 o 2 veces al año las pruebe el desarrollador en su casa a oscuras.

Hablando de desarrolladores, una cualidad del testing es molestar a la gente que lo crea ya que se ve como un ataque, crítica o hasta que dudamos de la capacidad de nuestro equipo. Esto es inevitable ya que esta actividad es destructiva y provoca resentimiento en las personas. Pero como hice yo cada vez que me dejan, **me guardo el rencor en lo más profundo del alma para que cuando un día lo tenga todo poder mirar hacia abajo en forma de humillación,** aceptamos la crítica y formamos una actitud de retroalimentación hacia el hecho que nos afectó para así mejorar como desarrolladores (y personas).

Tipos de Testing

Smoke Test

En ingeniería de software y pruebas de software, las pruebas de humo (smoke testing en inglés) son un tipo de pruebas funcionales que consisten en una revisión rápida de un producto de software para comprobar su inicio y que no tiene defectos evidentes que interrumpan la operación básica del mismo. Son pruebas que pretenden hacer una evaluación inicial de la calidad de un



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

producto de software previo a una recepción formal, ya sea al equipo de pruebas (quien ejecutará una batería completa de comprobaciones) o al usuario final.

El nombre es por analogía a las pruebas rudimentarias en ingeniería electrónica, en las que se comprueba que el encendido de un circuito no causa humo ni chispas.

(Biblio: La Wiki)

Testing Funcional

Las pruebas se basan en funciones y características (descrita en los documentos o entendidas por los testers) y su interoperabilidad con sistemas específicos. Se basa en los requerimientos y procesos de negocio que debe cumplir nuestro software como parte de “que hace”.

Testing No Funcional

Son pruebas que recorren aspectos ajenos al “que hace”, como pueden ser cosas de qué pasa si varios usuarios cargan la misma factura al mismo tiempo. Otro ejemplo es cuando tenemos pruebas de mantenimiento donde verificamos que ande nuestro programa en todas las plataformas, servidores y se carguen en los repositorios.

Algunos ejemplos de la cátedra:

- Performance Testing
 - Cuando el enunciado del segundo parcial de DSI decía; “debe responder en menos de 3 segundos” tenemos que hacer este tipo de pruebas. También cuando queremos ver la capacidad en usuarios, velocidad y procesamiento de nuestro sistema bajo situaciones “reales”.
- Pruebas de Carga
 - Probamos que pasa si cargamos un Bulk (gran conjunto de datos)
- Pruebas de Interfaz de usuario
 - Asociado al UI/UX donde vemos si se entiende la aplicación de manera intuitiva.
- Pruebas de Stress
 - Le damos un laburo de 9 hs de lunes a sábado con un salario mínimo y vemos como reacciona. Parecido al de carga tiene que ver cómo nuestro software responde de manera constante a ciertas interacciones, nada más que en vez de ser en rafaga son en un largo periodo.
- Pruebas de mantenimiento
 - Como comente, tiene que ver en general con la implementación y cómo se guardan-ejecutan nuestros programas.
- Pruebas de fiabilidad
 - Cuál es la tasa de error en operaciones riesgosas.
- Pruebas de portabilidad
 - Parecidas a las de mantenibilidad son las que hacemos cuando Windows lanza su versión de NET X incompatible con la anterior.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

TDD

Test Driven development (TDD) es un paradigma en el cual escribimos código de la manera más contraintuitiva, pensando en las pruebas y sus pruebas. Esto hace que la estructura del código sea diferente a lo convencional siendo orientado a pruebas primero (Test First Development) y Refactorización (Refactoring)

Para que funcione el desarrollo guiado por pruebas, el sistema que se programa tiene que ser lo suficientemente flexible como para permitir que sea probado automáticamente. Cada prueba será suficientemente pequeña como para que permita determinar unívocamente si el código probado pasa o no la verificación que ésta le impone.



BIBLIOGRAFÍA



Dale man desde cuando dudan de mi palabra?

Resumen de JC:

<https://fern-wishbone-d50.notion.site/Unidad-1-Ingenier-a-en-Software-en-el-Contexto-f4949d2abe064324ab5adcd359525881>

Libro de Ciclo de vida:

<https://www.javier8a.com/itc/bd1/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>

El resumen de carlitos:

[REDACTED]

WikiPedia:

https://donate.wikimedia.org/w/index.php?title=Special:LandingPage&country=AR&uselang=es&wmf_medium=sidebar&wmf_source=donate&wmf_campaign=C13_es.wikipedia.org



MIS MEMORIAS

Mil años de soledad

Esta materia la curse en mi sexto año de facultad y probablemente sea la última materia que curse antes de aprobar el proyecto final por ende no puedo hablar de esta materia como si fuera algo de 4to año. Para mi esto es un final a la Universidad, este fue mi “Last Dance” antes de terminar la carrera que implicó un esfuerzo extra en un montón de aspectos. Tengo la sensación que fue una vida desde que ingrese, ese lejano 2019, donde no habíamos tenido pandemia y aún estaba flaco.

Sinceramente tuve que trabajar por obligación (la preocupación por mis viejos es la obligación) y no tuve la oportunidades que vi en muchos compañeros de disfrutar ser joven adulto. Lo veo con amargura, aunque Fer le diga envidia sana, porque siento que fue un camino cuesta arriba donde veía a muchas personas disfrutando cosas que yo no pude y que me causan una tristeza nostálgica. Tuve por suerte la posibilidad de un laburo remoto de 6hs los últimos años de mi carrera que me dieron un impulso muy fuerte para continuar de lo cual estoy muy agradecido.

Sin embargo, dejó como testimonio que lo sacrifique todo para poder completar la carrera, deje de lado mis sueños, mi salud mental y física. Pospuse las cosas que me gustaban hacer, personas que yo amaba y hasta mi familia porque no tenía tiempo por el estudio y trabajo. Hoy estoy acá y no me queda otra que hacer la vista atrás, de la noche oscura que significó hacer todas estas cosas para poder recibirme y tristemente me duele no haber podido estar ahí. Me imaginaba terminar, rendir la última materia para salir afuera del aula donde mi familia, amigos y hasta mi novia (que cortamos) estuvieran ahí para felicitar me, para festejar con ellos una última vez. Pero la vida nunca es así, nunca es un final épico y satisfactorio sino que ordinario, un conjunto de pasos aburridos que no tienen mística por sí solo. No me malinterpreten, estoy feliz de haber terminado la carrera y de haber conocido tanta gente, pero una parte de mi se apega a ese sueño, a que ella hubiera estado ahí para festejar juntos, que todas las tristezas se hubieran esfumado el día que saque esta materia y ver a mis amigos el ultimo día.

Como dije, la vida no es así, la ultimas 2 semanas fueron una tortura para mi mente ahora estoy solo en una casa vacía, sintiendo que no fui suficiente para la gente que me rodea. Siento que nunca soy suficiente y que podría haber hecho mucho más.



INGENIERIA Y CALIDAD DE SOFTWARE

RESUMEN FINAL Y ULTRAVIOLENTO

Lamento la nota triste, los últimos resúmenes los escribí con muchas cosas que me estaban pasando por la cabeza. El 2024 fue un año que me puso a prueba, fue un conjunto de problemas que me golpearon por todos lados, todo se rompió, incluido yo.

Para no dejar las cosas con este humor les digo (muy de cornuda lo se) que nunca dejen de brillar, nunca dejen de sonreír porque la vida es este conjunto de tragedias en el cual tenemos poco tiempo para ser felices. No se si decirles que hagan como yo y sacrifiquen todo para llegar al final porque cuando uno mira atras se da cuenta que tiene 25 años donde los últimos 6 años fueron una tortura constante.

Me siento preso de las circunstancias que me rodean y tome las decisiones que tuve que tomar, yo considere que lo valía, que prefería terminar y hacer las cosas que me gustan cuando tenga la palabra “Ingeniero” delante. Esto no significa que tengas que seguir el mismo camino que yo o sentirte mal por las mismas cosas. Cada uno es artífice de su destino, el capitán de su barco y dueño de sus decisiones que en nosotros queda tomar el camino que queramos.

Para finalizar quiero dejarle un agradecimiento a mis **compañeros de grupo** que se pusieron las pilas, cuando yo no las tuve, para que podamos completar todos los trabajos prácticos. Como también a mis **padres y amigos** que siempre están ahí para mi aunque no sepa expresarlo, sigo adelante por ustedes (Amerita un brindis esto).

Y por último un agradecimiento a **usted querido lector**, estos resúmenes los hago para ayudar a mis futuros colegas porque sé que no soy el único en esta complicada aventura que llamamos Ingeniería en sistemas. Espero que hayas disfrutado leyendo mis acotaciones, deseo que al igual que este humilde caballero puedas terminar la carrera.

No te des por vencido cuando todo te esta saliendo mal, la vida no es acerca de qué tan fuerte puedas pegar, sino de que tanto puedas aguantar los golpes y seguir adelante, que tanto te puedan dañar y no rendirte.

Como siempre digo, esto no es un adiós... es un hasta siempre!

¡Gracias Totales!

“Mientras más oscura es la noche, más brillan las estrellas”
-Braum-